



THE LAB CODER

Foundation (Grove Hat)

Updated 22 Mar 2026

Content

Page	Unit	Foundation 1
3	2001	<u>Introduction to Raspberry Pi</u>
5	2002	<u>Sequencing</u>
13	2003	<u>Loop</u>
16	2004	<u>Control Statement</u>

Page	Unit	Foundation 2
22	2005	<u>Logic Operator</u>
28	2006	<u>Ultrasonic Sensor</u>
36	2007	<u>Colour Sensor</u>
40	2008	<u>Conditional Loop</u>
44	2009	<u>Project 1</u>

Page	Unit	Foundation 3
45	2010	<u>Count Loop</u>
48	2011	<u>Variable</u>
53	2012	<u>Module Control with Variable</u>
54	2013	<u>Applications</u>

Page	Unit	Foundation 2
55	2014	<u>Variables and Sensors</u>
57	2015	<u>Boolean</u>
59	2016	<u>Applications II</u>
60	2017	<u>Project 2</u>

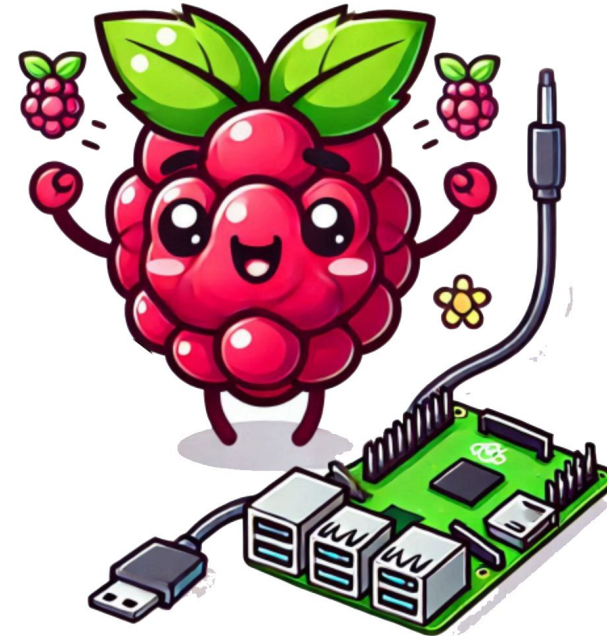
2001 Introduction to Raspberry Pi

Learning

You will be introduced to the Raspberry Pi robot and how to operate it.

In class, you will be guided to explore the following:

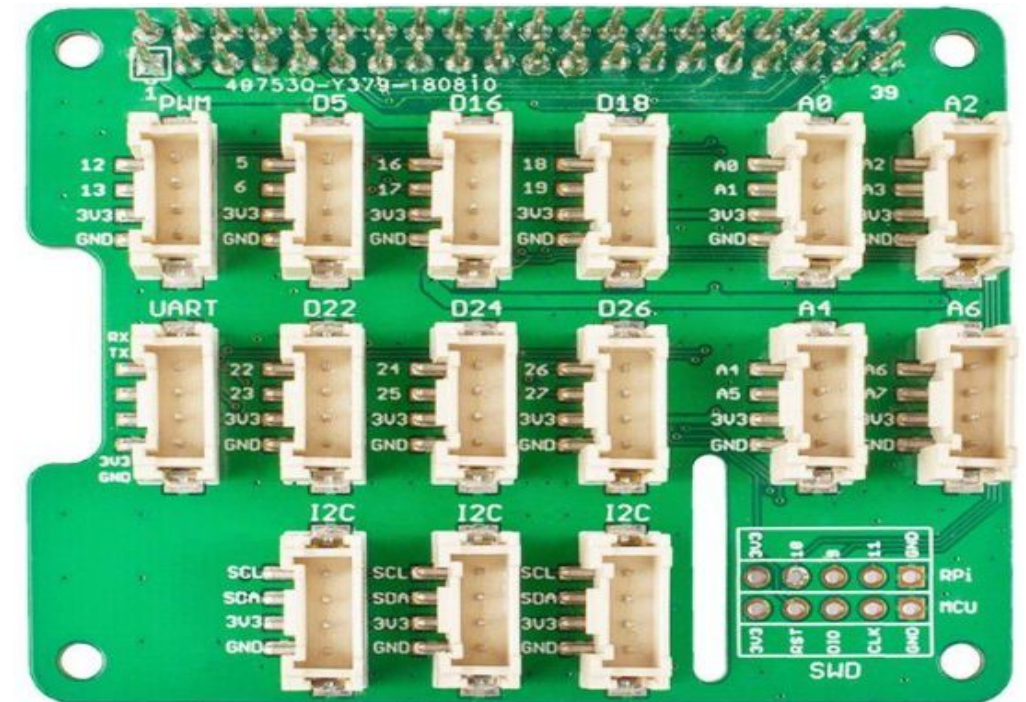
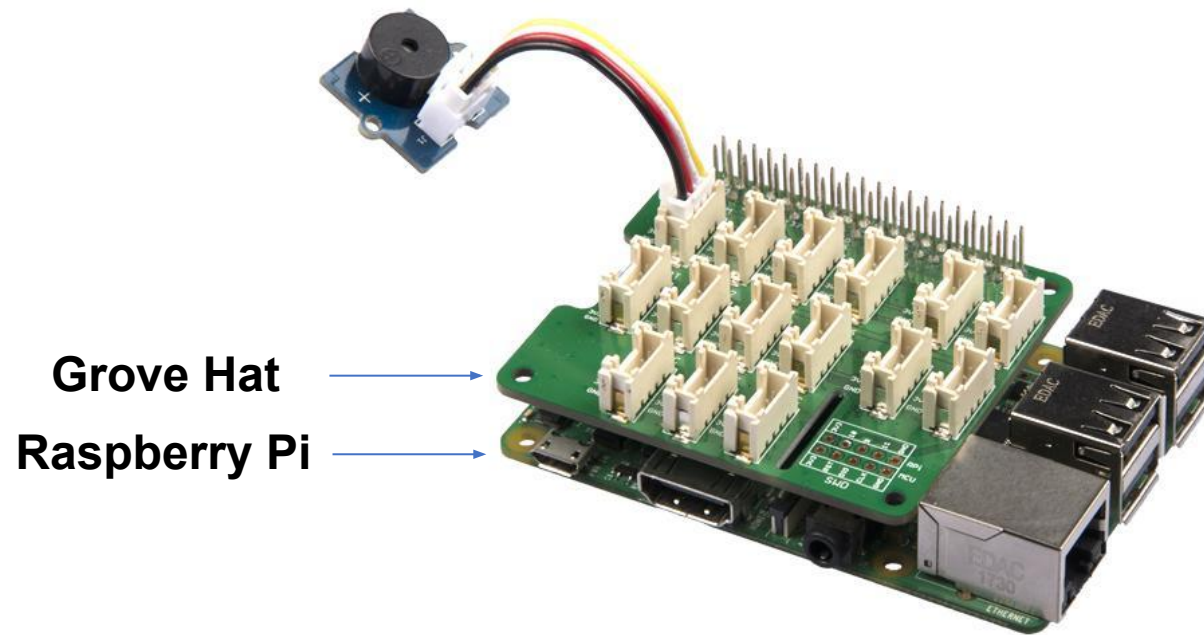
- Powering up your robot.
- Identifying the Buzzer and LCD modules.
- Testing the blocks for Buzzer and LCD.
- Using Comment block for description.



2001 Introduction to Raspberry Pi

Robot

Raspberry Pi (the brain) + Grove Hat (ports for connecting modules and sensors)



2002 Sequencing

Learning

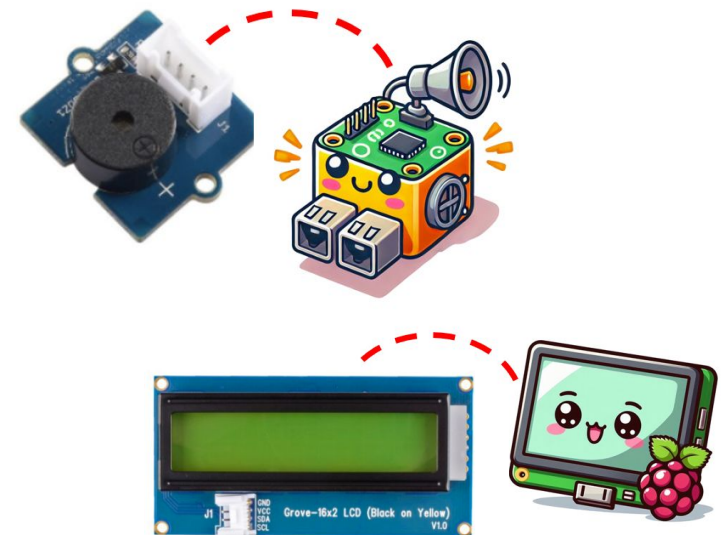
When you write code, it's like giving your robot a list of instructions. But just like following a recipe or building LEGO, **the order** of those steps really matters. That's called **sequencing**, making sure things happen in the right order.

In this unit, you'll use **Google Blockly** to control your robot's **Buzzer** and **LCD**. Blockly works like a stack of blocks: the robot reads them from the top down, one at a time.

If your blocks are in the wrong order, things might look strange or not work at all! So, we'll practice putting blocks in the right sequence to make your robot do exactly what you want.

In class, you will be guided to explore the following:

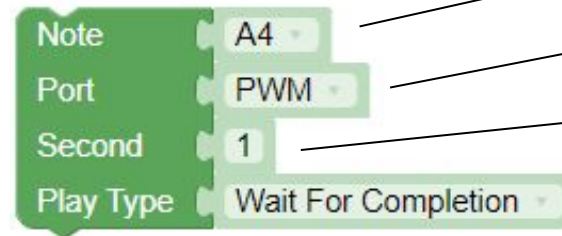
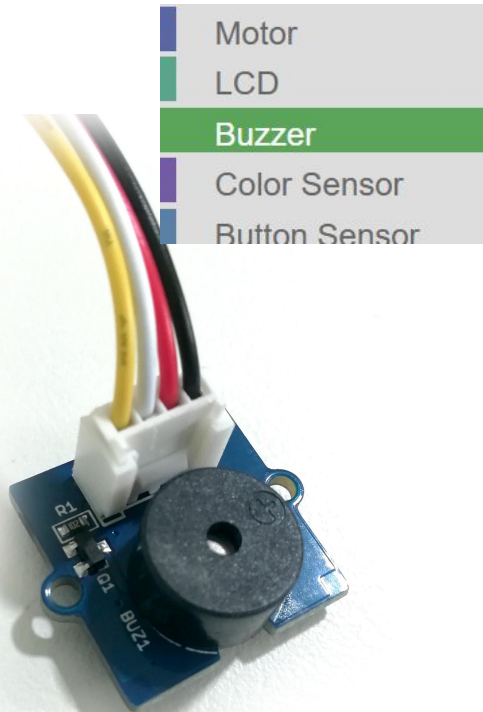
- Understand the Buzzer and LCD modules and their blocks.
- Using Wait block to insert pauses.
- Controlling pitches and making rhythms.
- Controlling display position and making animation.
- Learn how to make visual and sound happen together.



2002 Sequencing

Buzzer Sensor

For making a sound



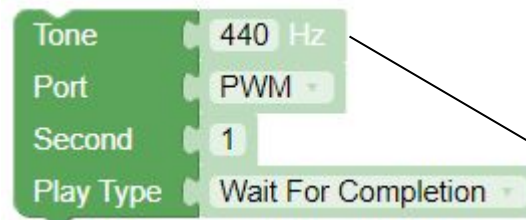
Notes to play (C4, D4, E4, F4, A4, B4). C4 is lowest.

Plug at which port

Duration of play

Set the Play Type:

- **"Wait For Completion"**
Wait until the sound ends then move to next block
- **"Play once"**
Move to next block while letting it play once. It will stop when the time is up.
- **"Loop"** (this option is not required)
Move to next block while keep looping the sound

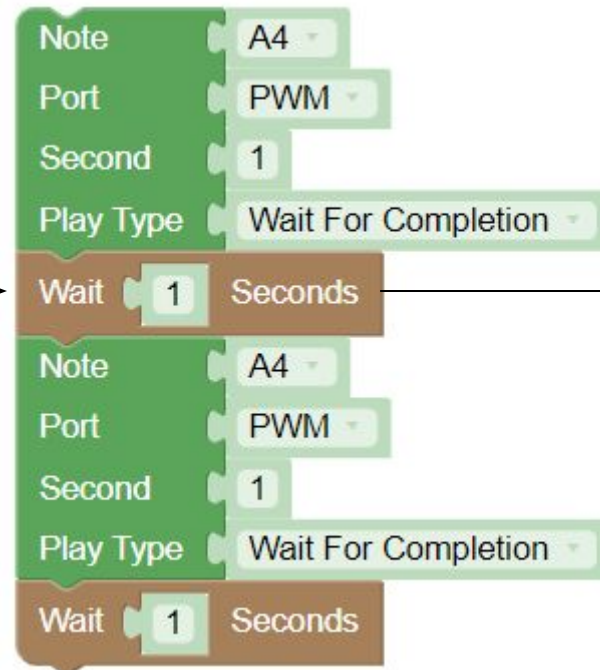
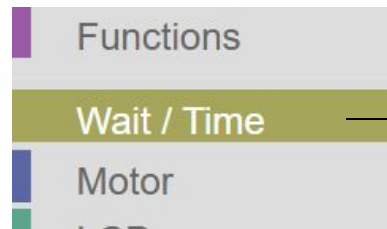
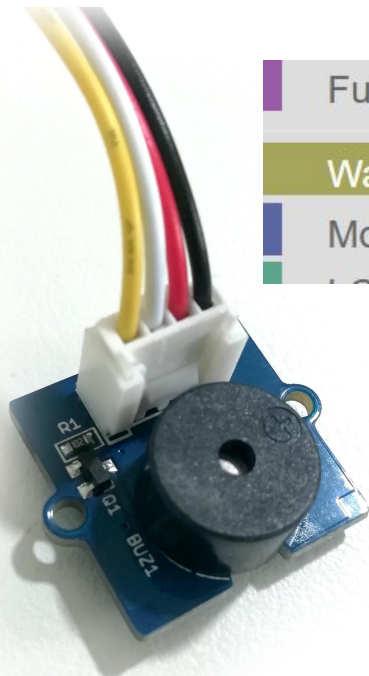


You can also use frequency instead of notes. The higher the sharper the tone.

2002 Sequencing

Buzzer Sequence

Making a **beeping** sound



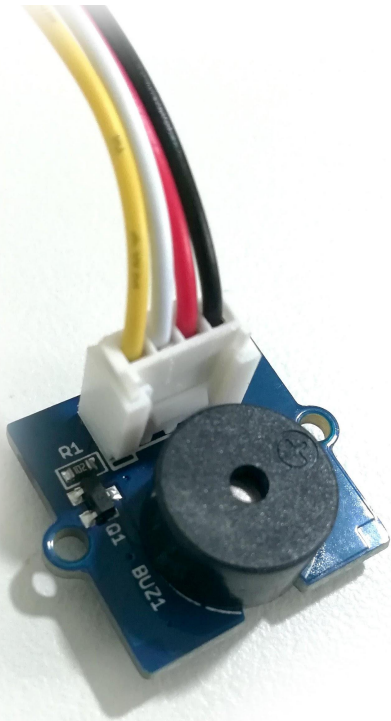
To make a **BEEPING** sound, you must include a **wait block** between the sound blocks. Without it, it will give a continuous sound.

The **speed** of beeping is affected by the both durations of the sound and the wait.

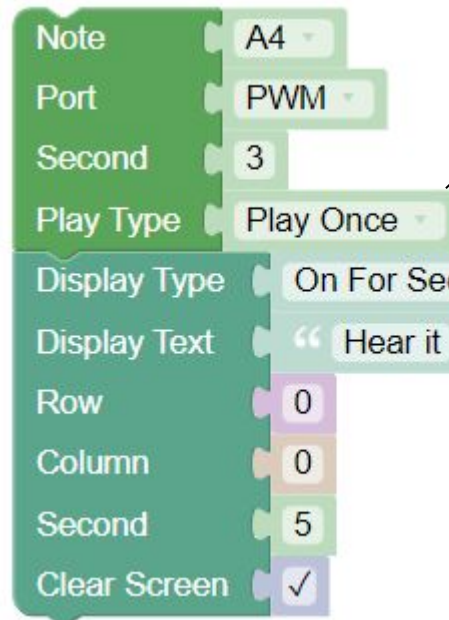
2002 Sequencing

Buzzer Sensor

Play a sound and execute other blocks concurrently



You can set the sound to play for a duration while continue to perform the subsequent tasks.



Play Type: **Play Once**

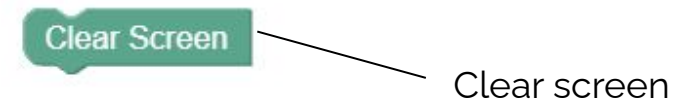
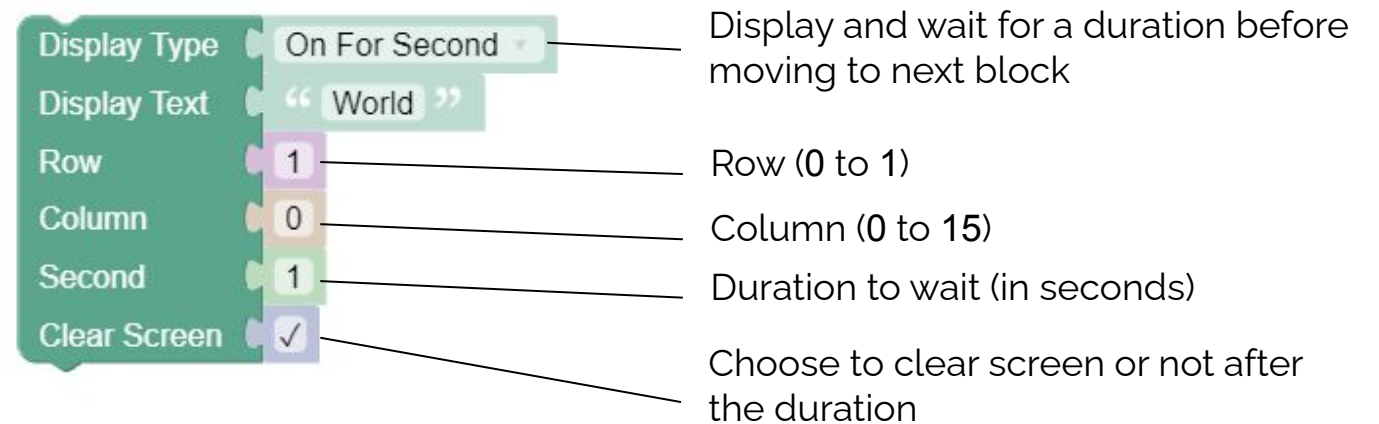
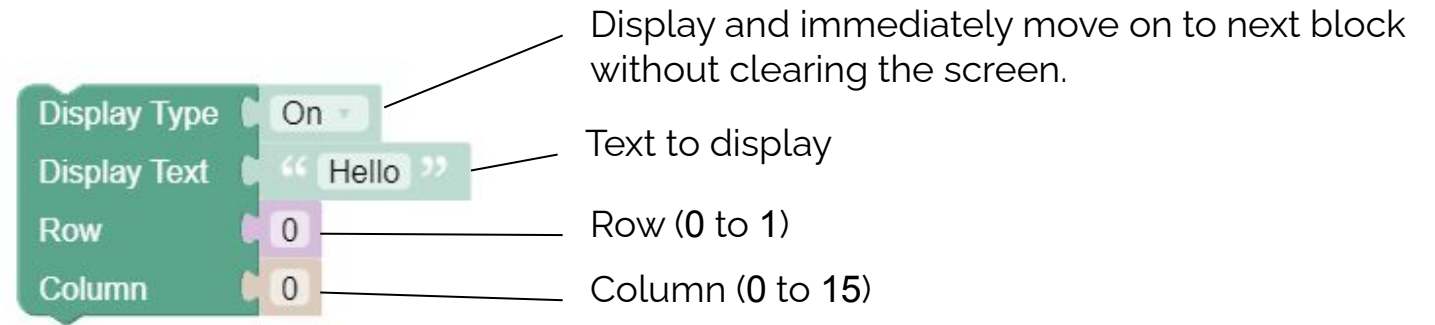
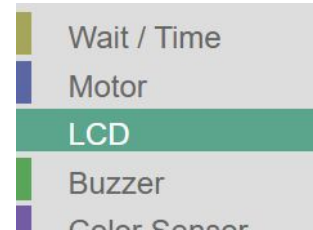
The sound will start playing and the next block will be executed immediately (it does not wait for 3 seconds)

The sound will continue playing at the background while the LCD display "Hear it". The sound will stop after 3 seconds (indicated in the sound block) while the LCD will continue to display for another 2 seconds (total 5 seconds as indicated in the LCD block).

2002 Sequencing

LCD

For displaying text



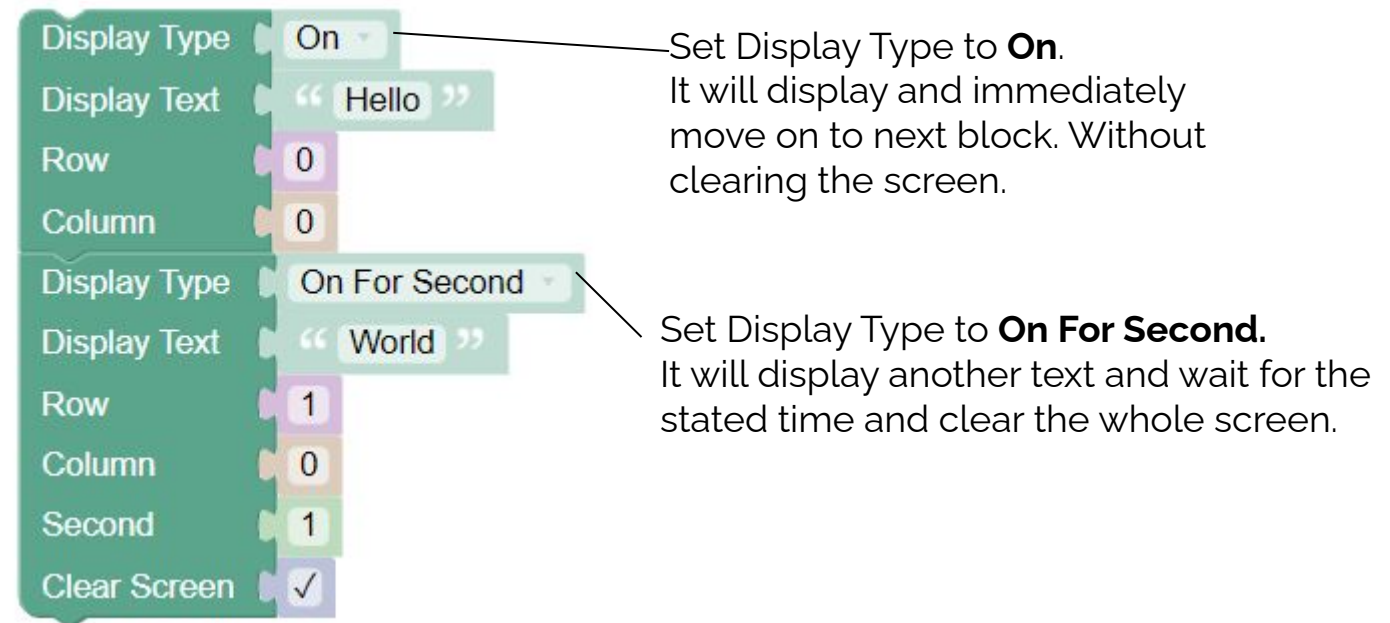
2002 Sequencing

LCD

Displaying text **concurrently**



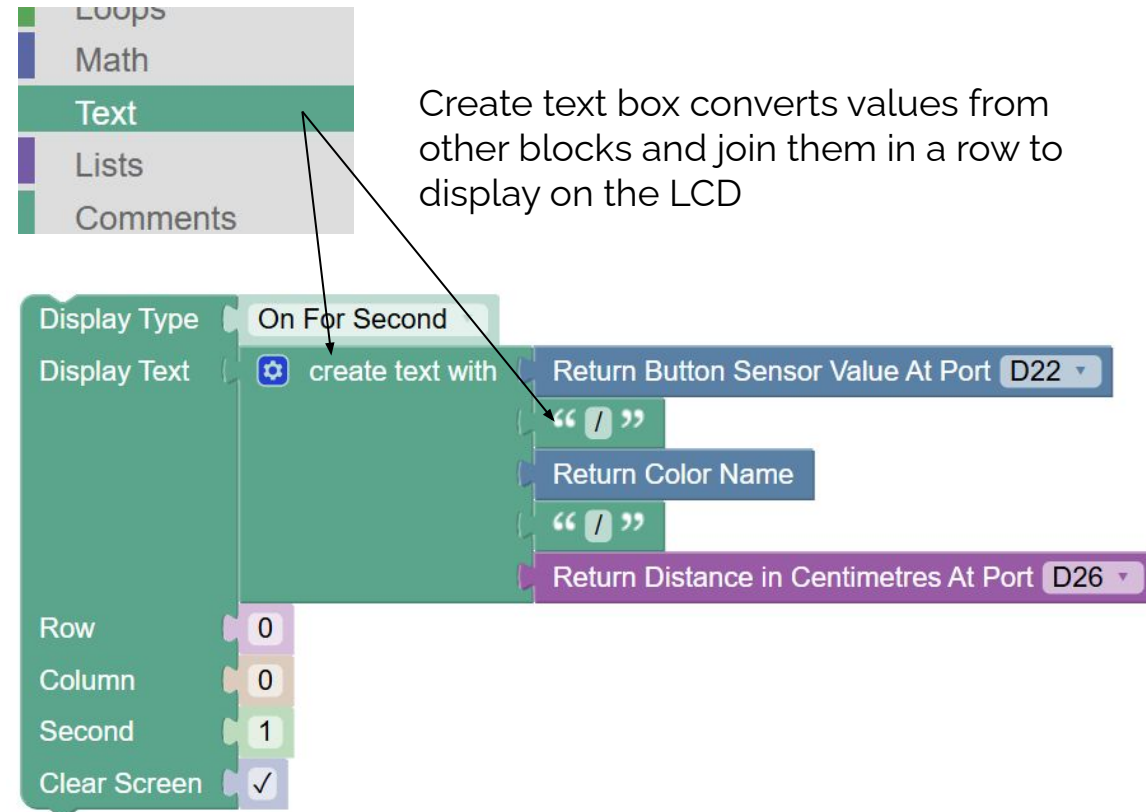
Putting the blocks in this manner will show the text concurrently (at the same time)



2002 Sequencing

LCD

For displaying values from other blocks in a row



2002 Sequencing

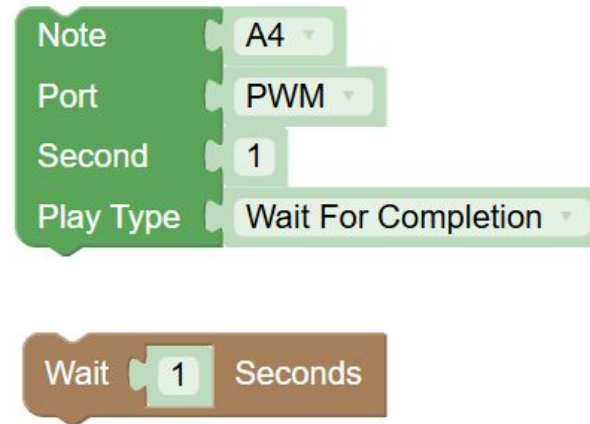
Comment

Comment blocks can be used to for writing description. They can be placed anywhere and will not be run.



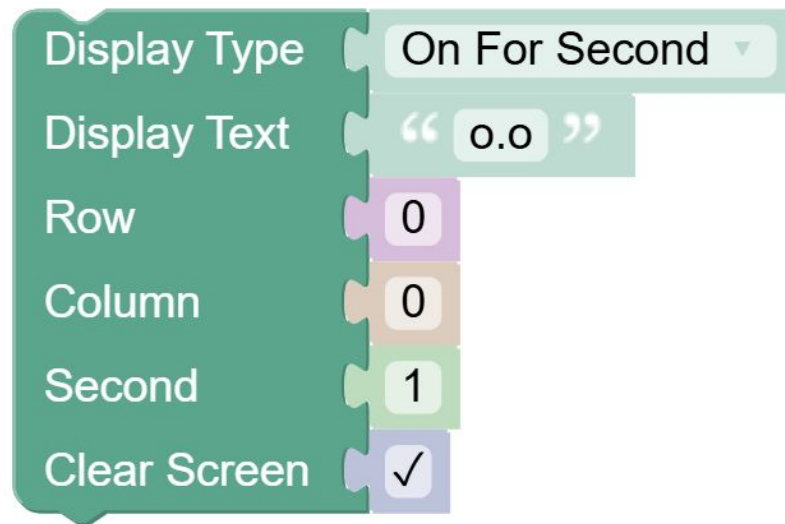
Exercise Q1

Using the blocks below, make 3 beeping sounds. You may duplicate the blocks.



Exercise Q2

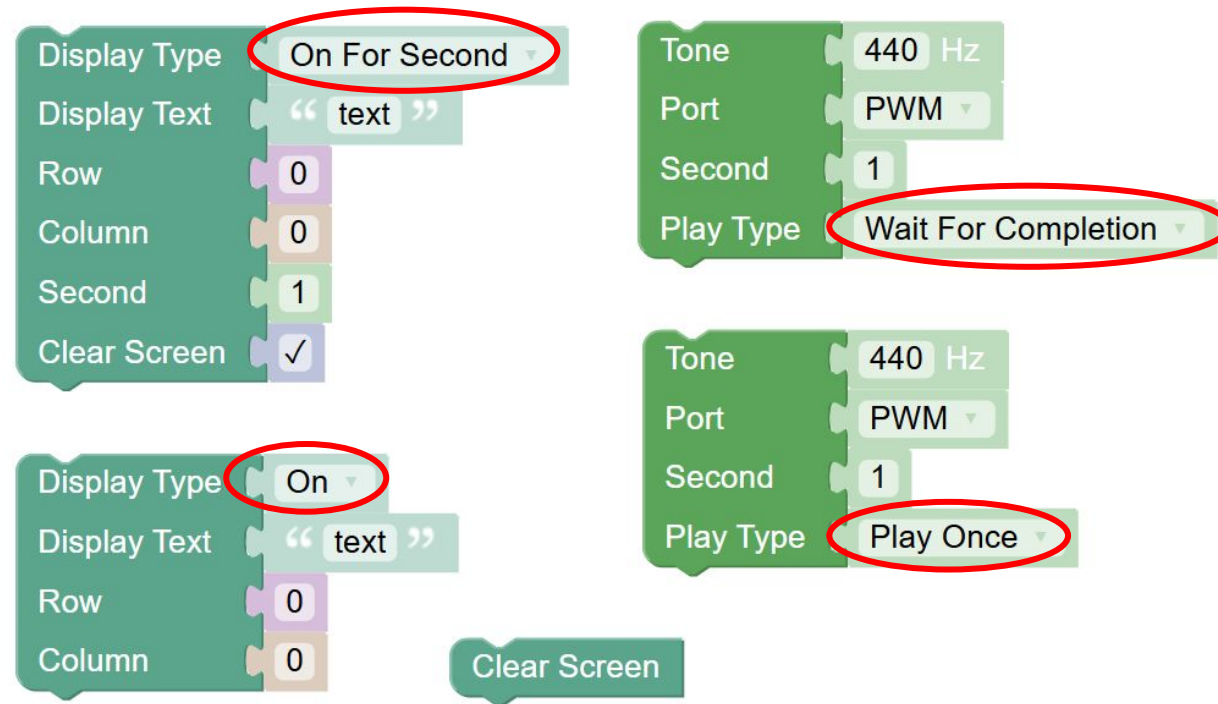
By duplicating and adjusting the values of the LCD block below, make a blinking expression that blinks 3 times as shown.



animated clip

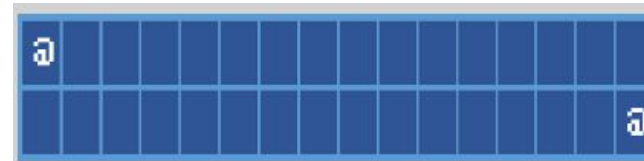
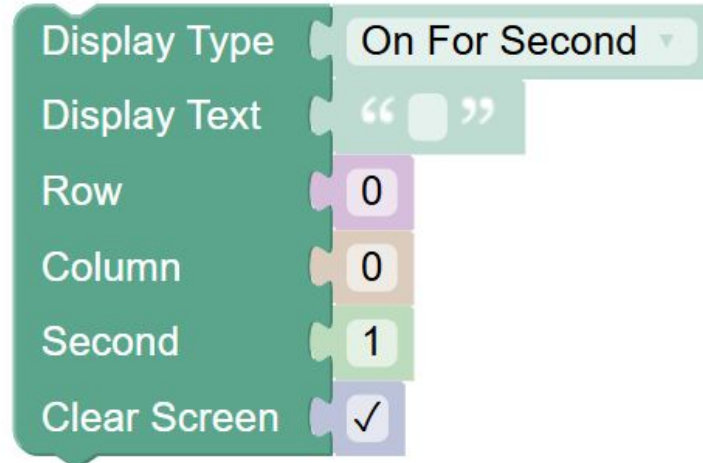
Exercise Q3

Using only the blocks below, show 2 ways to make the text and sound appear **together**, stay for one second and disappear **together**.



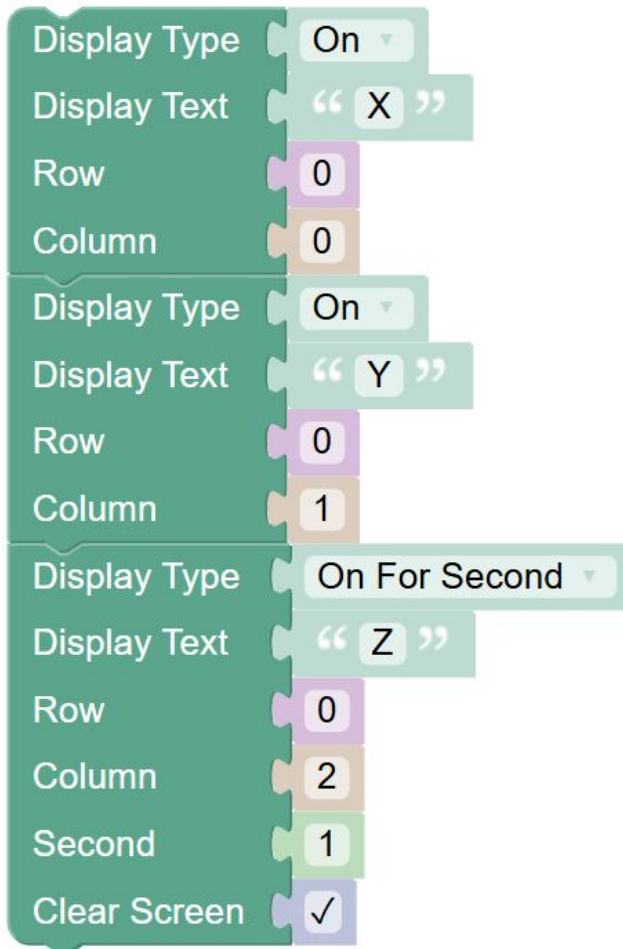
Exercise Q4

The **two** LCD blocks, display the following on the LCD.



Exercise Q5

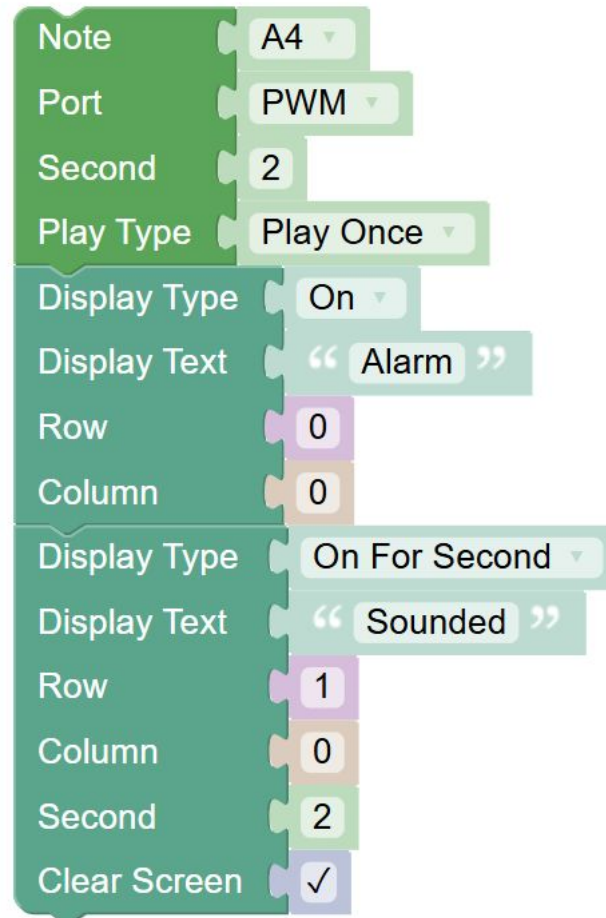
Study the code below without running. What will be displayed when it is run?



- A) X, Y and Z will appear one by one
- B) X and Y will appear first, followed by Z
- C) X, Y and Z will appear together
- D) Only Z will appear

Exercise Q6

Study the code below without running. Write down how long it take for the code to complete.

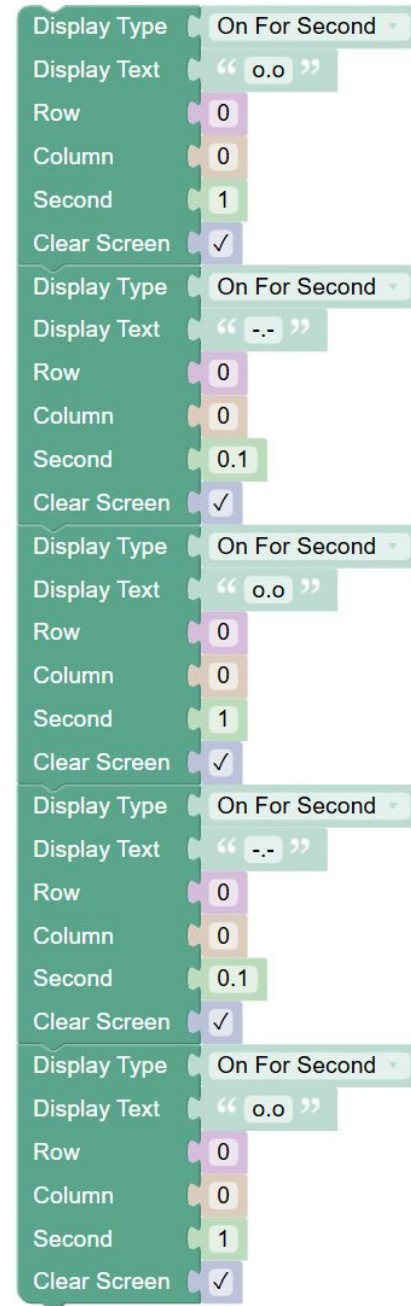


Answers

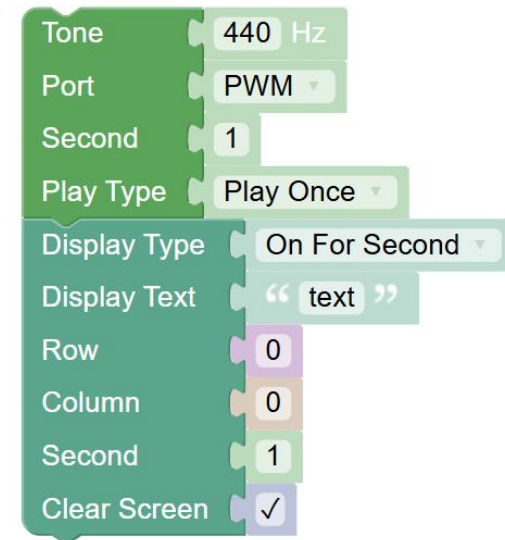
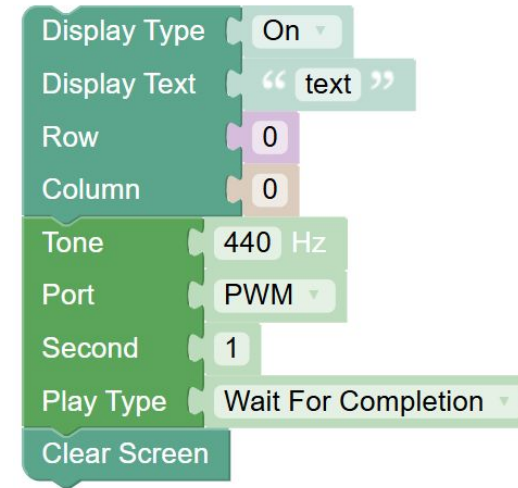
Q1:



Q2:

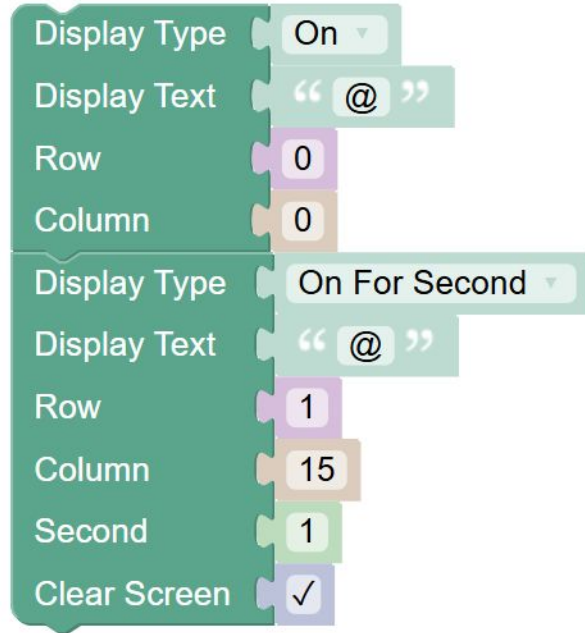


Q3:



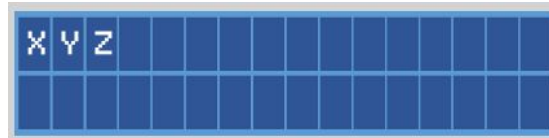
Answers

Q4:



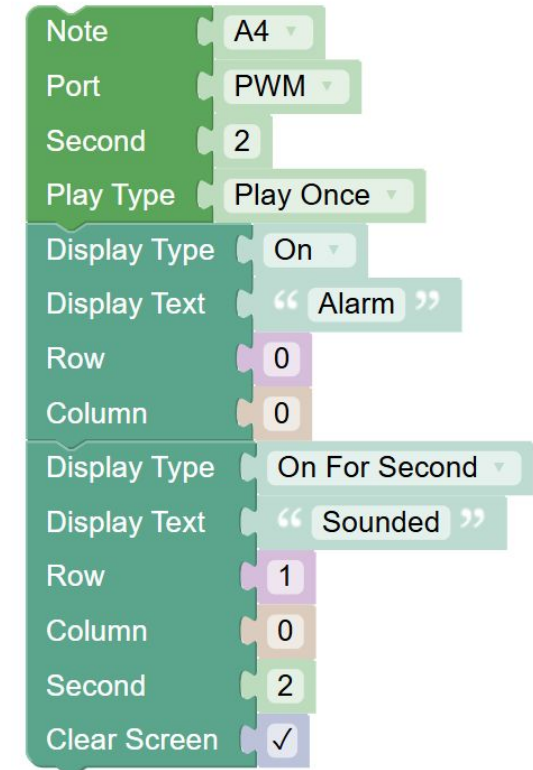
Q5: C

When the Display Type is On, X and Y will appear and immediately run the block below. It is so fast that all of them appear to be shown together. They will also disappear together after 1 second.



Q6: 2 seconds.

The Play Type for the sound block is Play Once, so it will run together with the blocks below.



2003 Loop

Learning

You might notice that some steps happen again and again. These repeating steps are called **patterns**. Instead of writing the same code over and over, you can use something called a **loop**. A loop tells the computer to repeat a set of instructions.

In this unit, you'll learn how to use loops in Blockly to make your robot repeat actions, either a certain number of times, or forever!

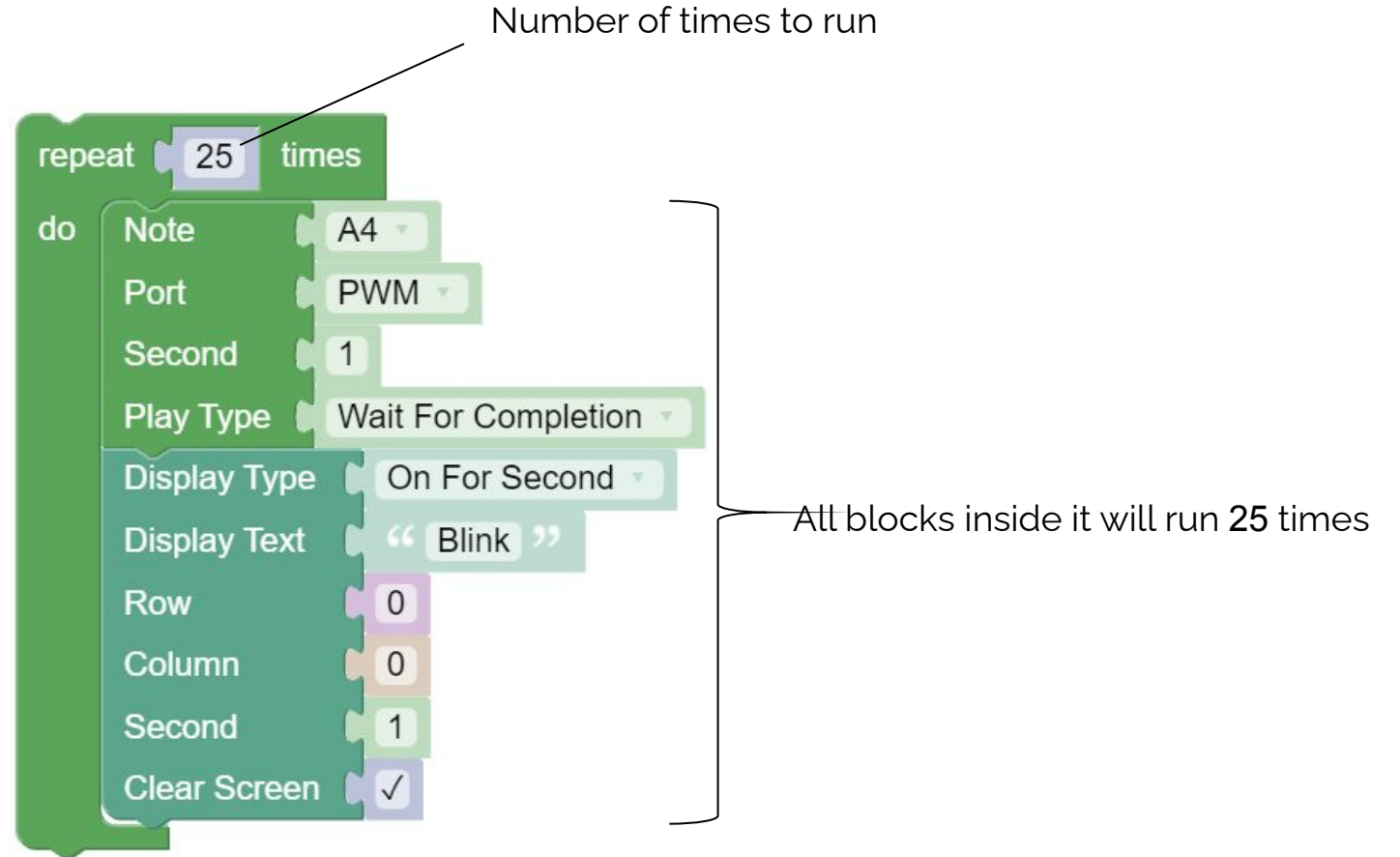
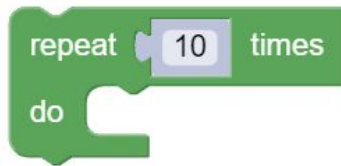
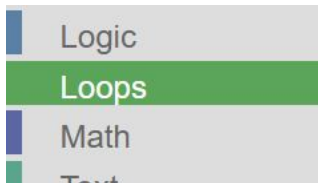
In class, you will be guided to explore the following:

- Using loops to make beeping sounds and blinking text.
- Making a program repeat forever.
- Understanding multiple loops and nested loops.

2003 Loop

Repeat

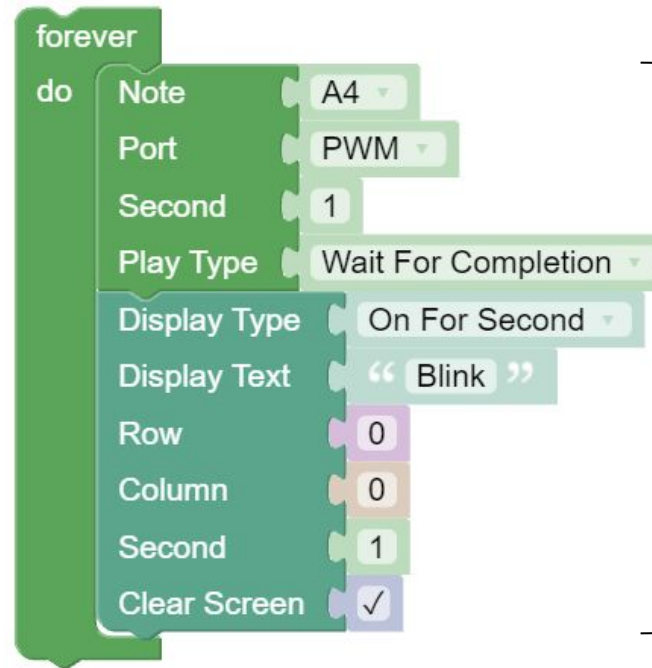
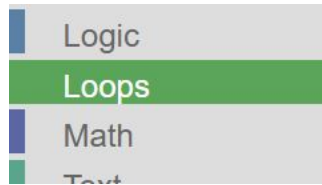
Repeat codes for a number times



2003 Loop

Repeat *forever*

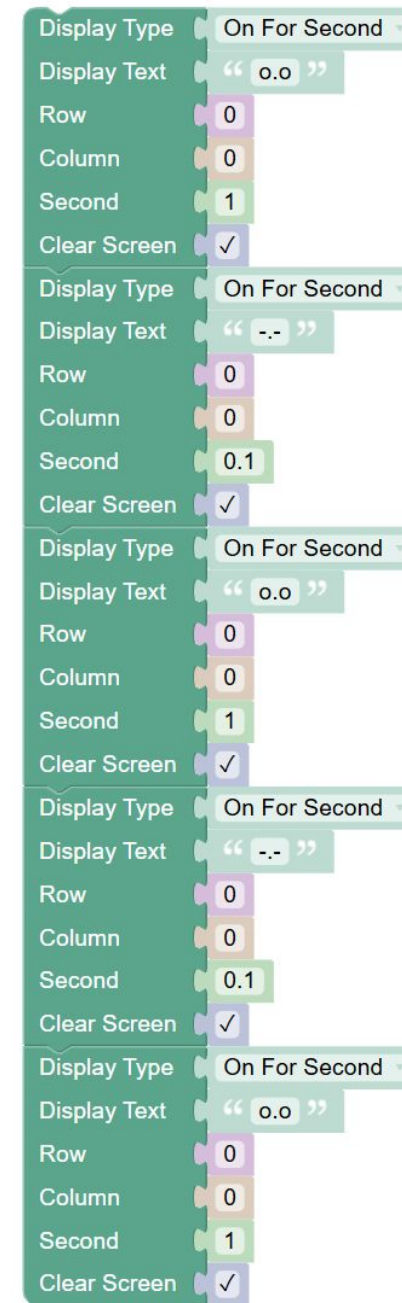
Repeat codes forever



All blocks inside it will
repeat forever

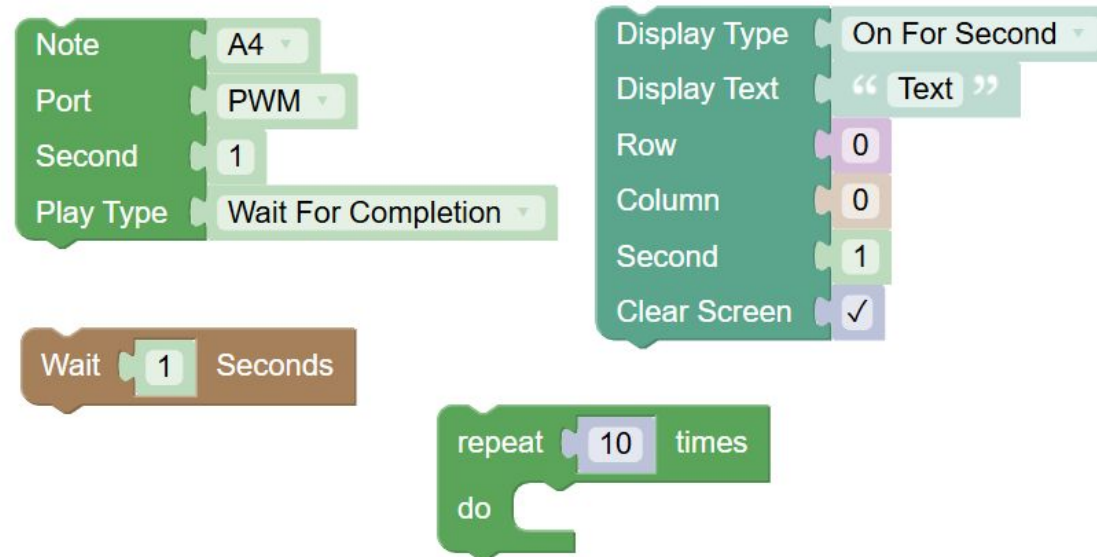
Exercise Q1

The code shows an expression that blinks three times. Use a Repeat Loop to shorten the code.



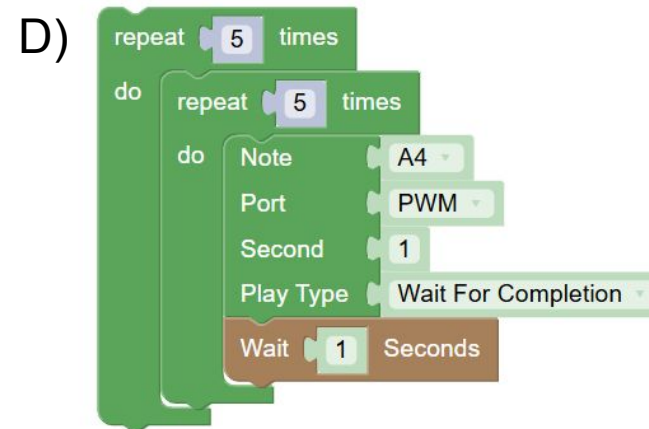
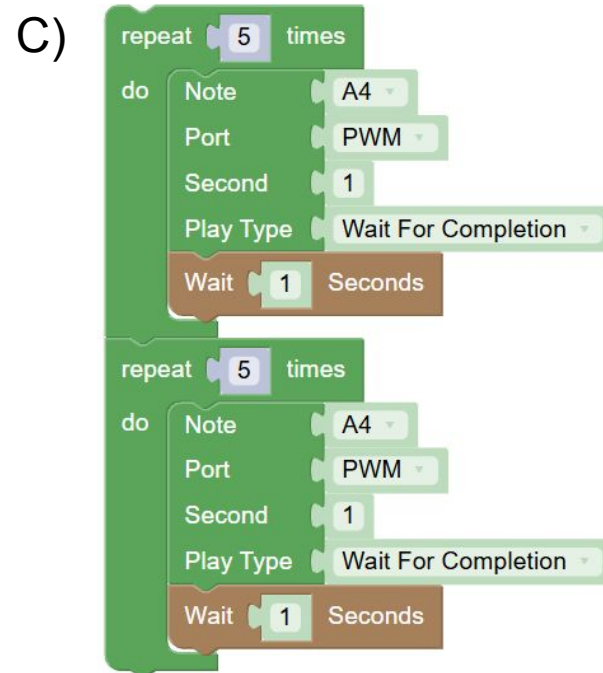
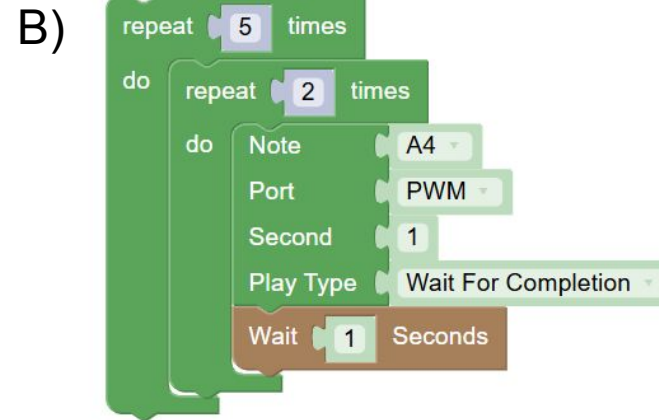
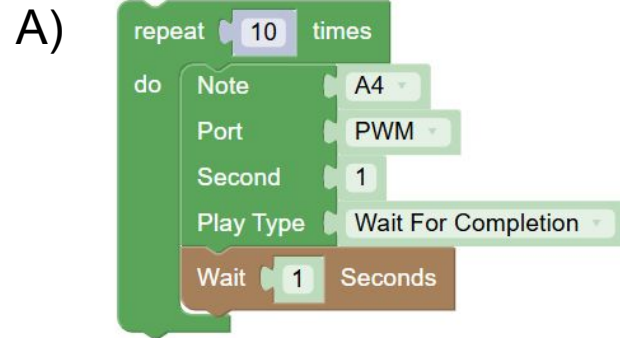
Exercise Q2

Using the following blocks, make **5 beeping sounds** followed by **5 blinking text**. You will need to duplicate some blocks.



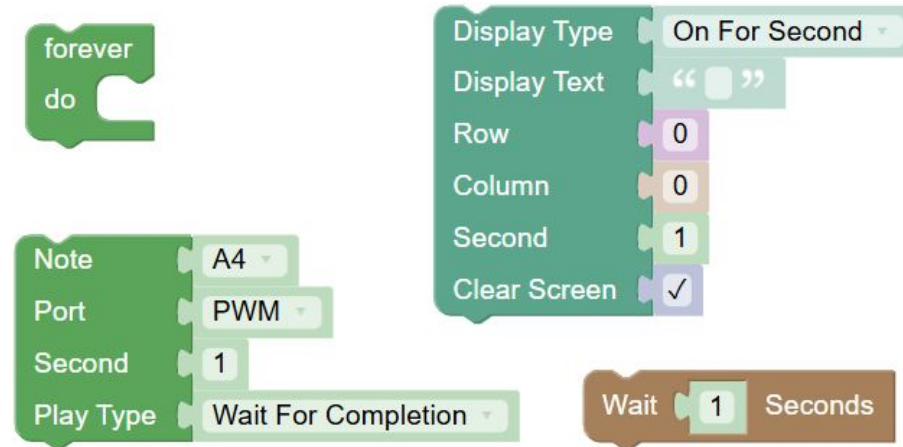
Exercise Q3

Which of the following codes make 10 beeps?



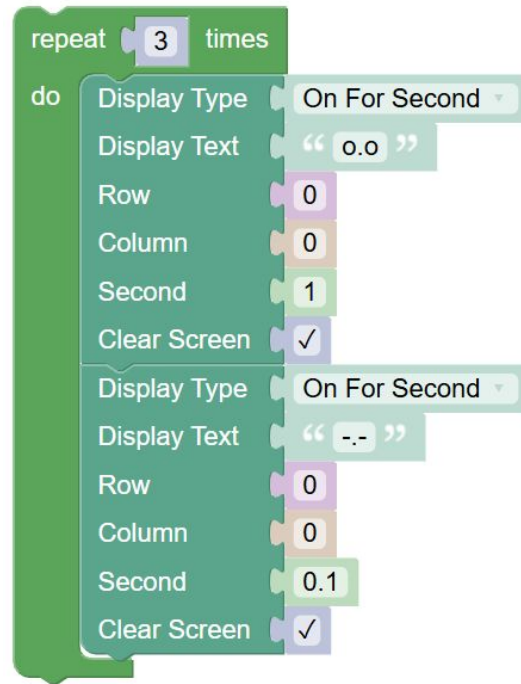
Exercise Q4

Using the following blocks, display “DANGER” on the screen while beeping continuously.

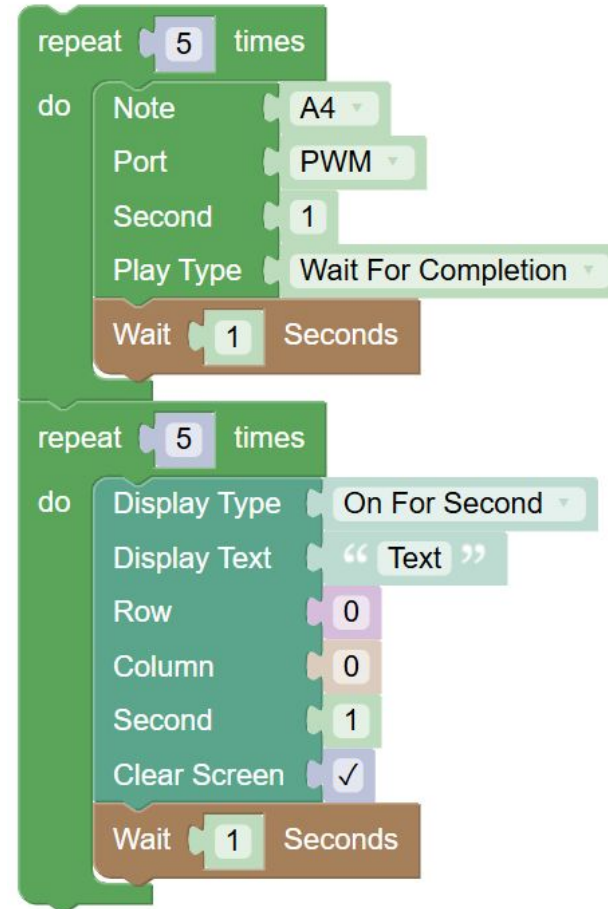


Answers

Q1:

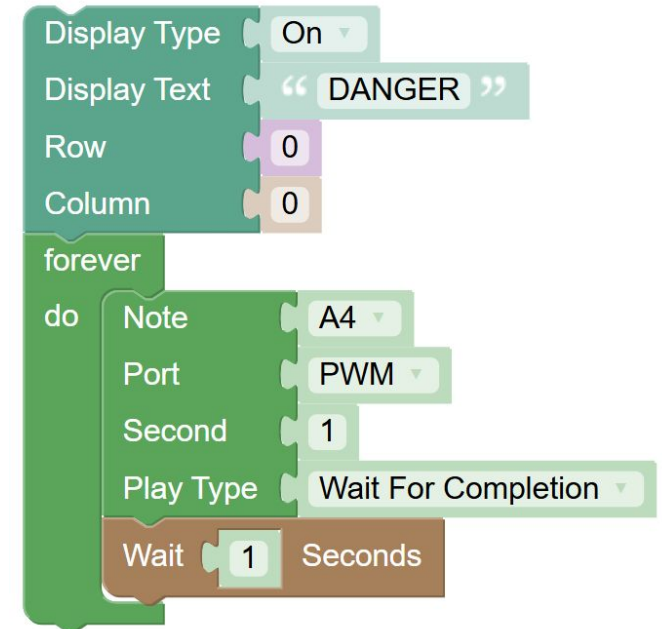


Q2:



Q3: A, B and C

Q4:



2004 Control Statement

Learning

An **If control statement** helps your code make decisions. For example, "If the button is pressed, play a sound," or "If the score is 10, show a win screen." The computer checks the **condition**, and only runs that part of the code if the condition is true.

You can build If statements in different ways, from simple checks to more complicated ones with lots of choices.

In this unit, you'll learn about the Button sensor and how to use different types of If statements to interact with it.

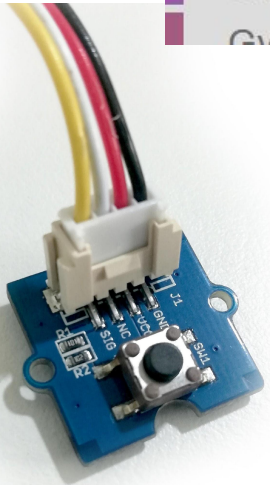
In class, you will be guided to explore the following:

- Using Button sensor to interact with the robot.
- Form conditions to check for button press/release in order to make decisions.
- Understanding different If statement structures.

2004 Control Statement

Button Sensor

To interact with the robot



Send a value to the robot:

0 if the button is not pressed

1 if the button is pressed

Return Button Sensor Value At Port D22

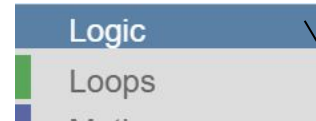
The port where the sensor is plugged
Remember to change to the correct port

Wait Until Button Sensor at Port D22 Is Pressed

If the button is currently not pressed,
wait until button is pressed

Wait Until Button Sensor at Port D22 Is Released

If the button is currently pressed,
wait until button is not pressed

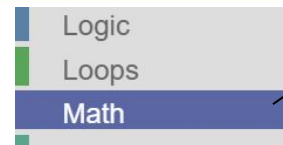


Return Button Sensor Value At Port D22 = 1

Condition to **check** if button is pressed
It is **not** setting D22 to 1

Return Button Sensor Value At Port D22 = 0

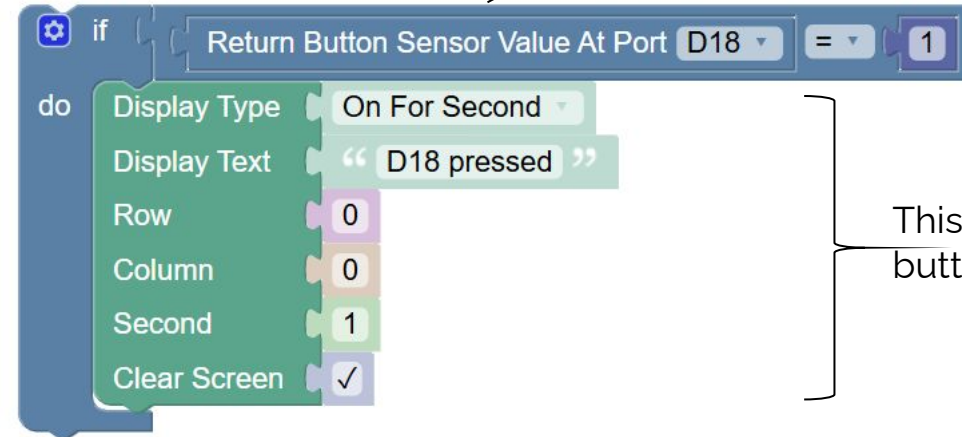
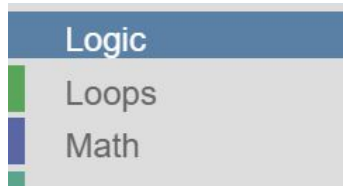
Condition to **check** if button is not pressed
It is **not** setting D22 to 0



2004 Control Statement

If

If a condition is true, then run some codes



If this condition is true,
the blocks in "do" will run.

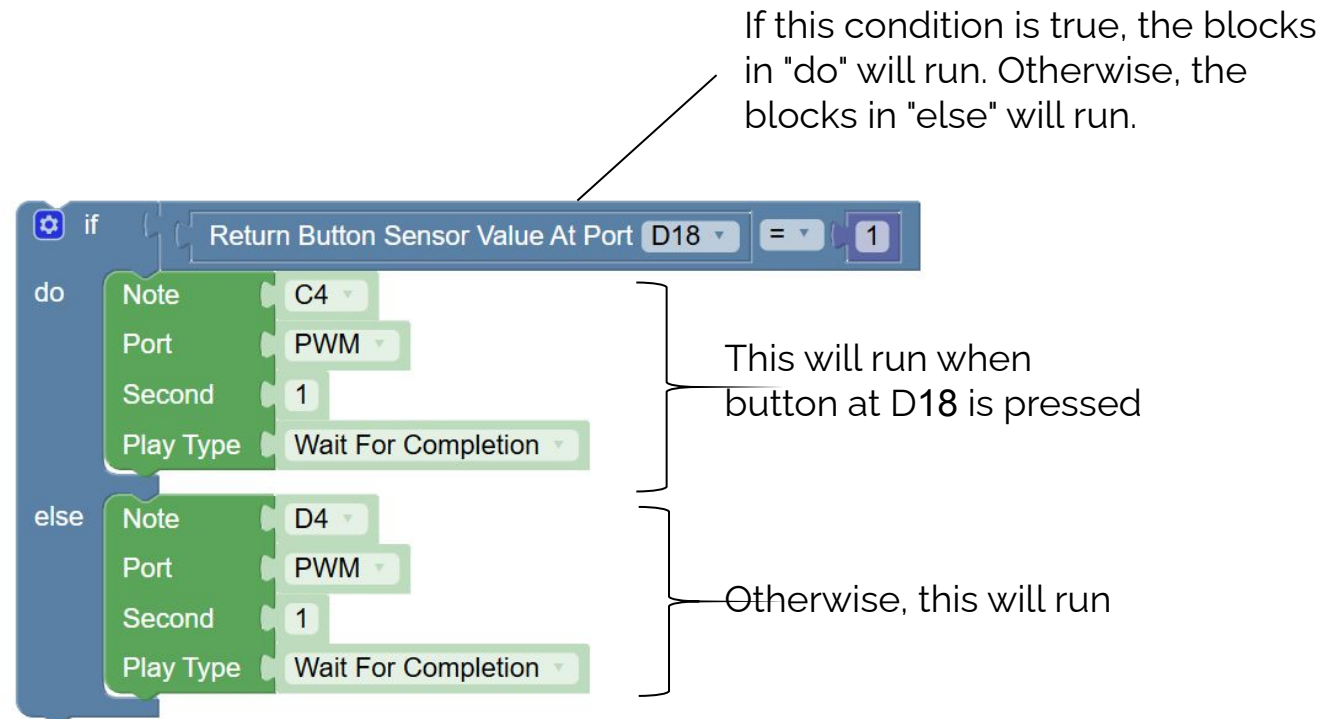
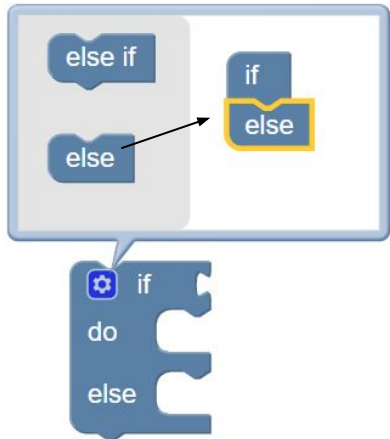
This will run when
button at D18 is pressed

2004 Control Statement

If...else

If a condition is true, then run some codes.

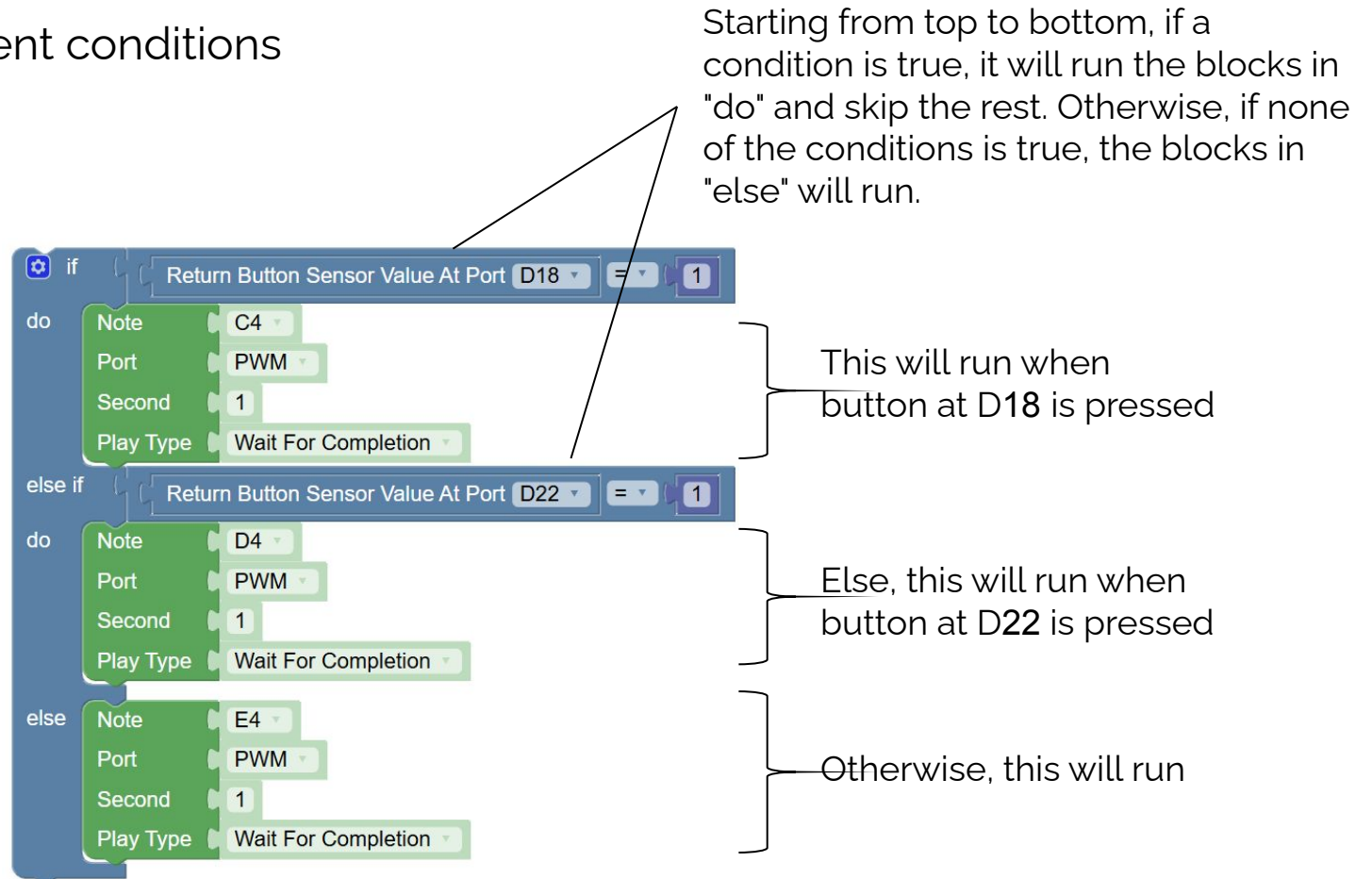
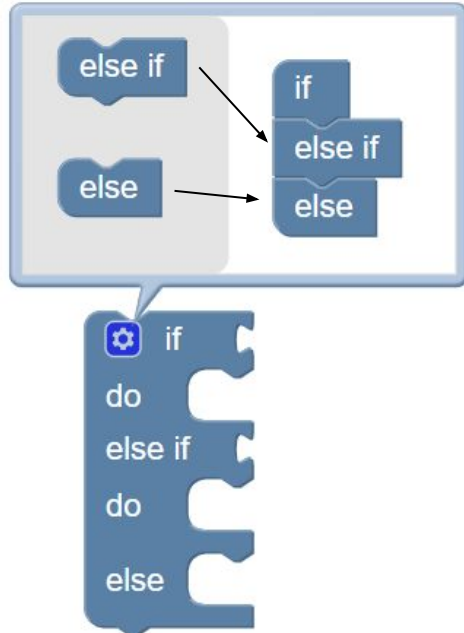
Otherwise, run some other codes.



2004 Control Statement

If...else if...else

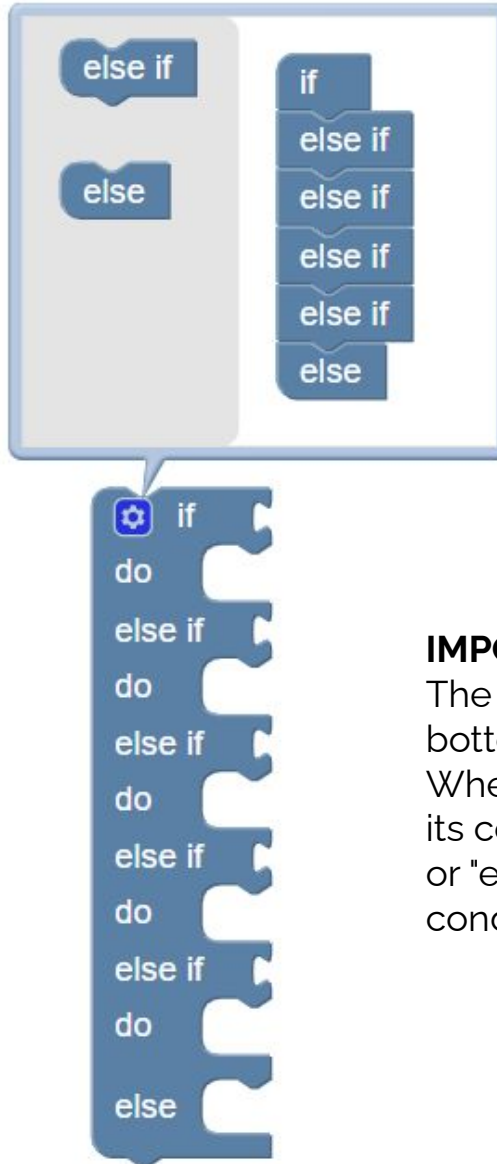
Run different codes based on different conditions



2004 Control Statement

If...else if...else

You can add many "else if"

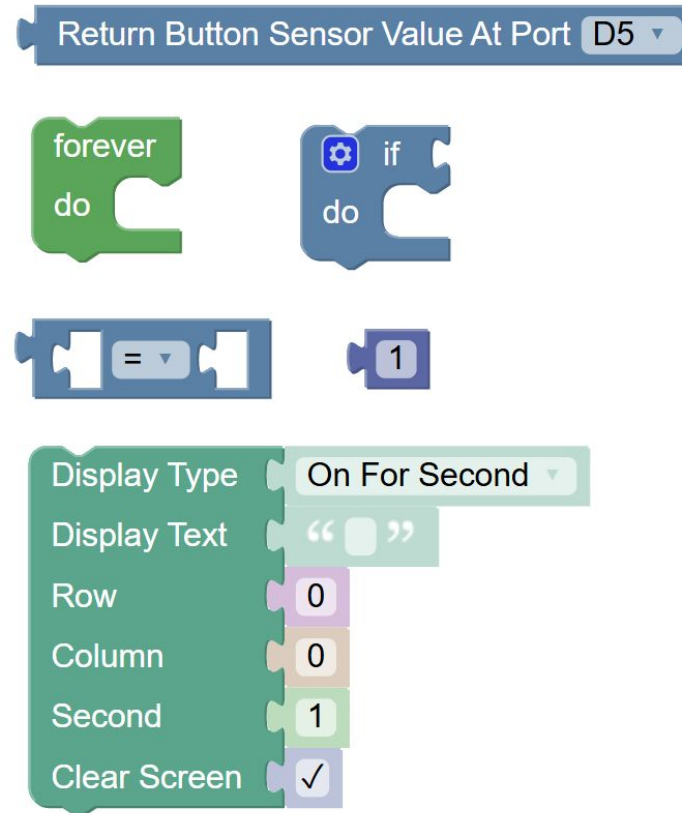


IMPORTANT:

The program will check from top to bottom to see which condition is true. When one of the conditions is true and its codes are run, the remaining "else if" or "else" will be skipped (even if the conditions in other "else if" is true).

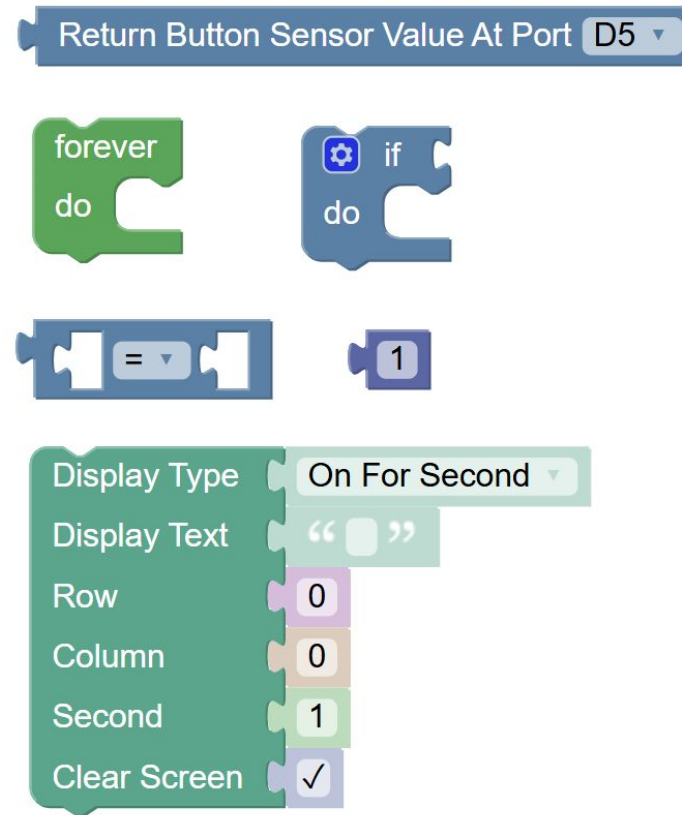
Exercise Q1

Using the blocks below, create a code that displays “**D18 pressed**” whenever **Button D18 is pressed**.



Exercise Q2

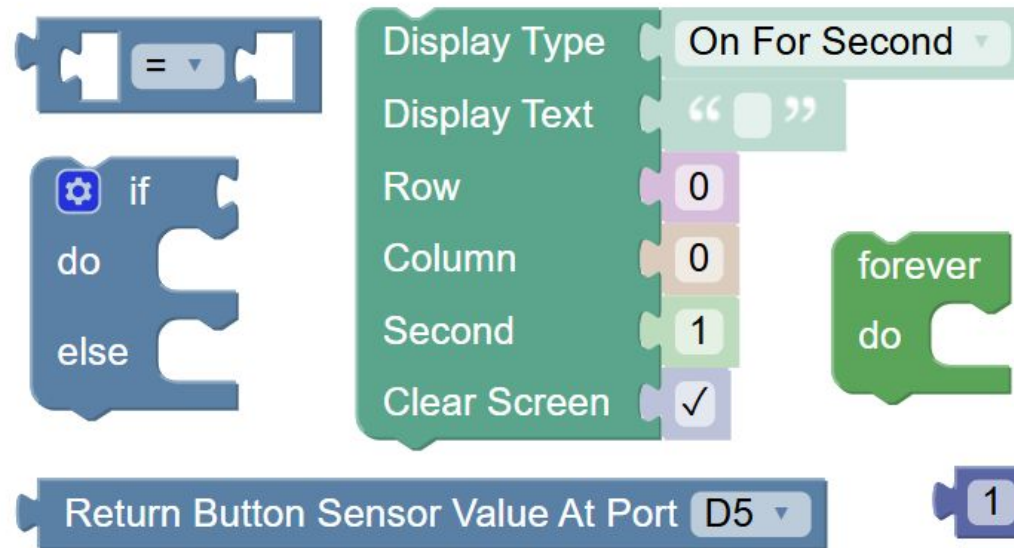
Using the blocks below, create a code that displays “**D18 not pressed**” whenever **Button D18 is NOT pressed**.



Exercise Q3

Using the blocks below (you will need to duplicate some blocks), create a code that performs the following:

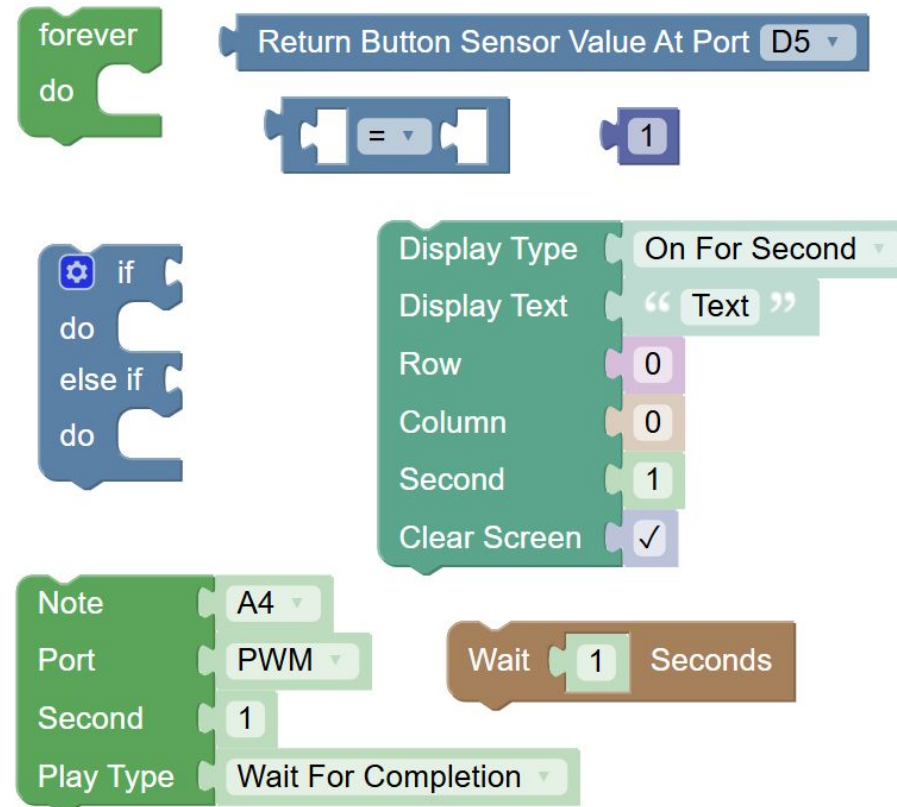
- If D22 is pressed, display “ding dong” for one second.
- If D22 is not pressed, display “press the bell”.



Exercise Q4

Using the blocks below (you will need to duplicate some blocks), create a code that performs the following:

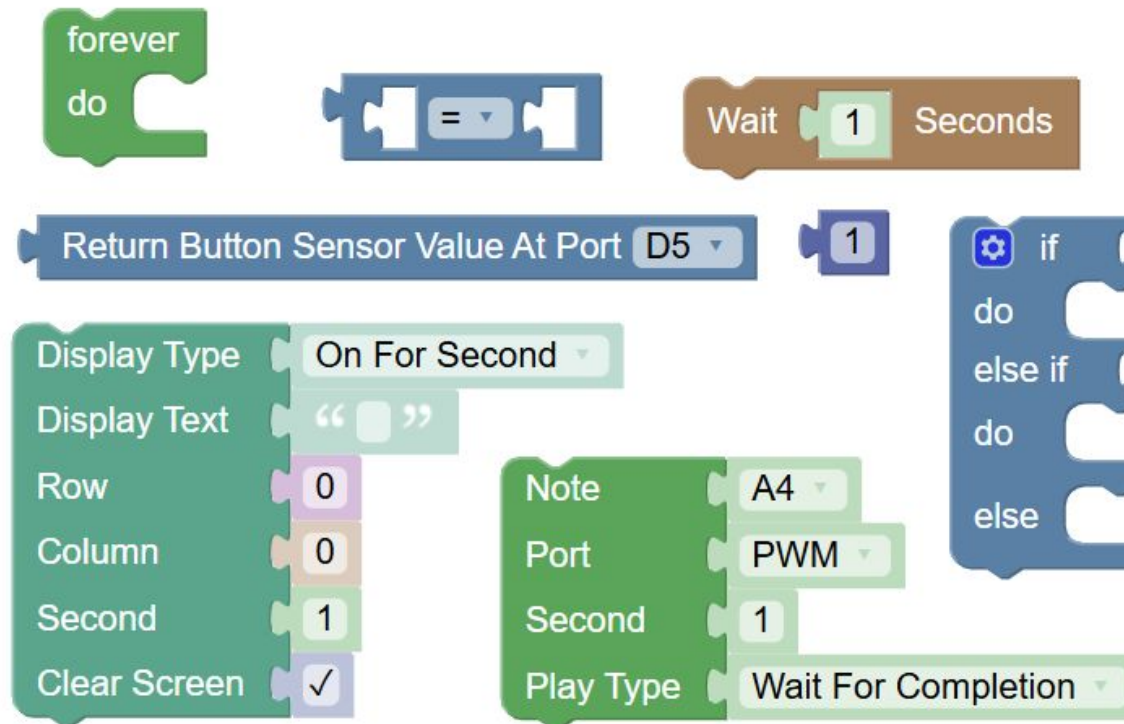
- If D22 is held on, make a beeping sound.
- If D24 is held on, display blinking text.



Exercise Q5

Using the blocks below (you will need to duplicate some blocks), create a code that performs the following:

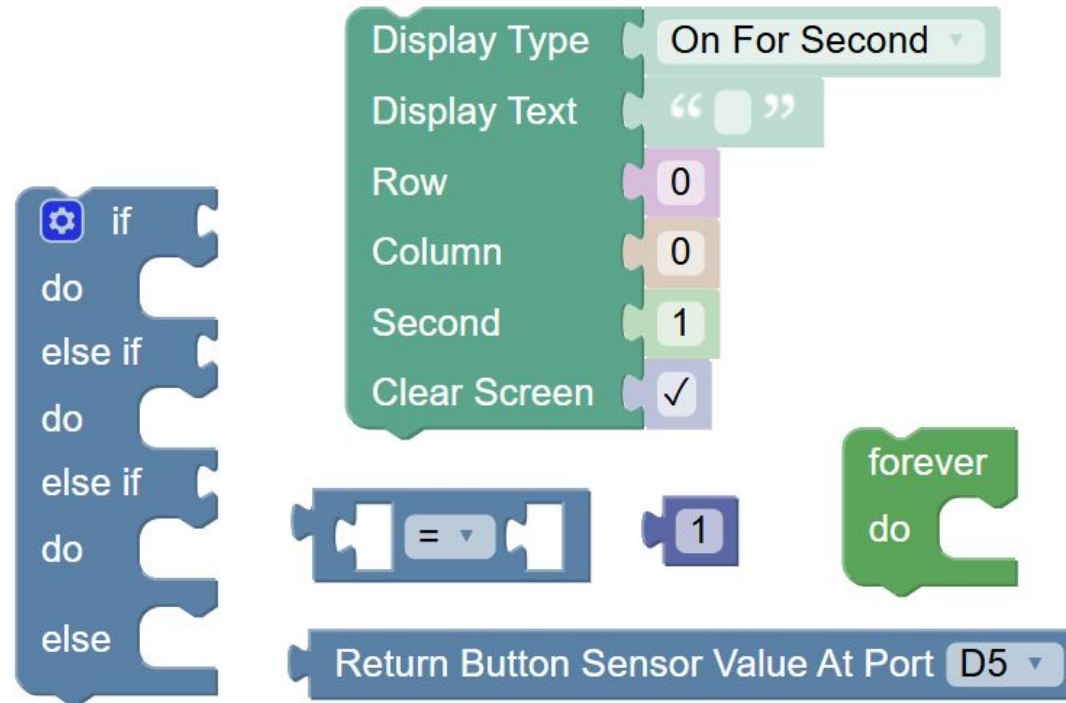
- If D22 is held on, display “D22 held”.
- If D24 is held on, display “D24 held”.
- Otherwise, make a beeping sound.



Exercise Q6

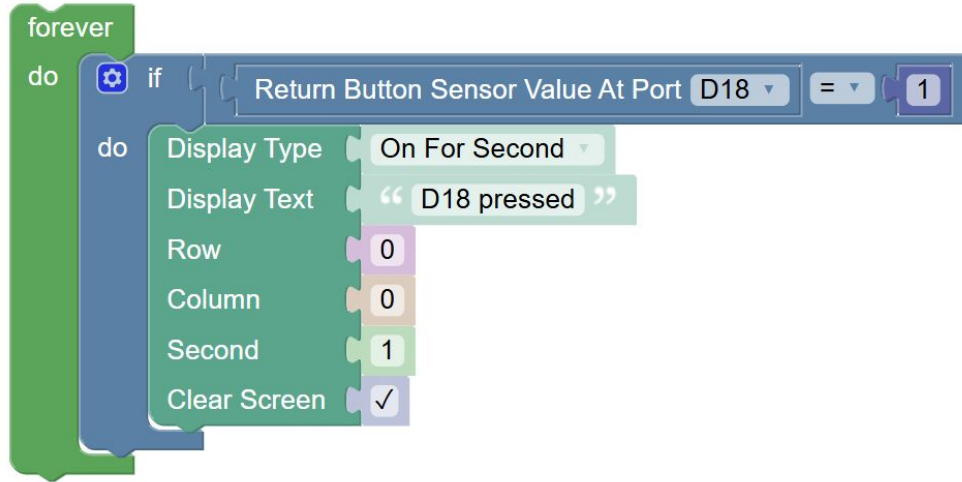
Using the blocks below (you will need to duplicate some blocks), create a code that performs the following:

- If D18 is held on, display "<"
- If D22 is held on, display "|".
- If D24 is held on, display ">"
- Otherwise, display "-".

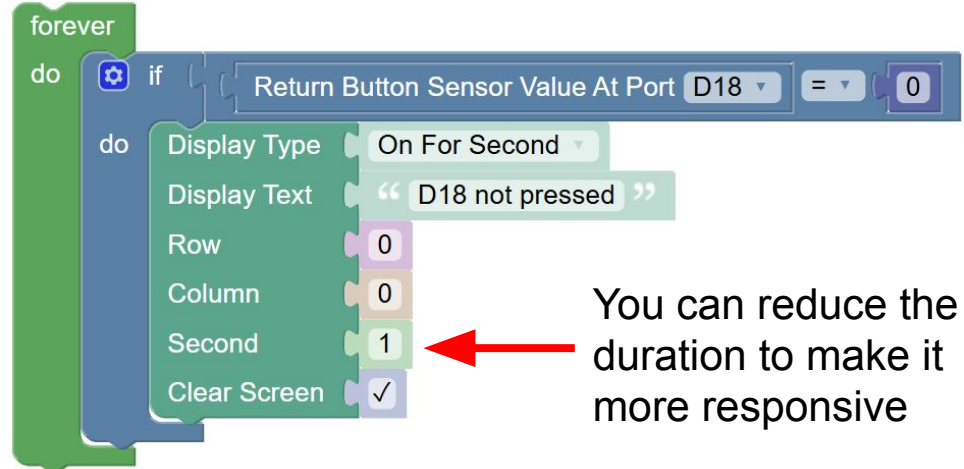


Answers

Q1:

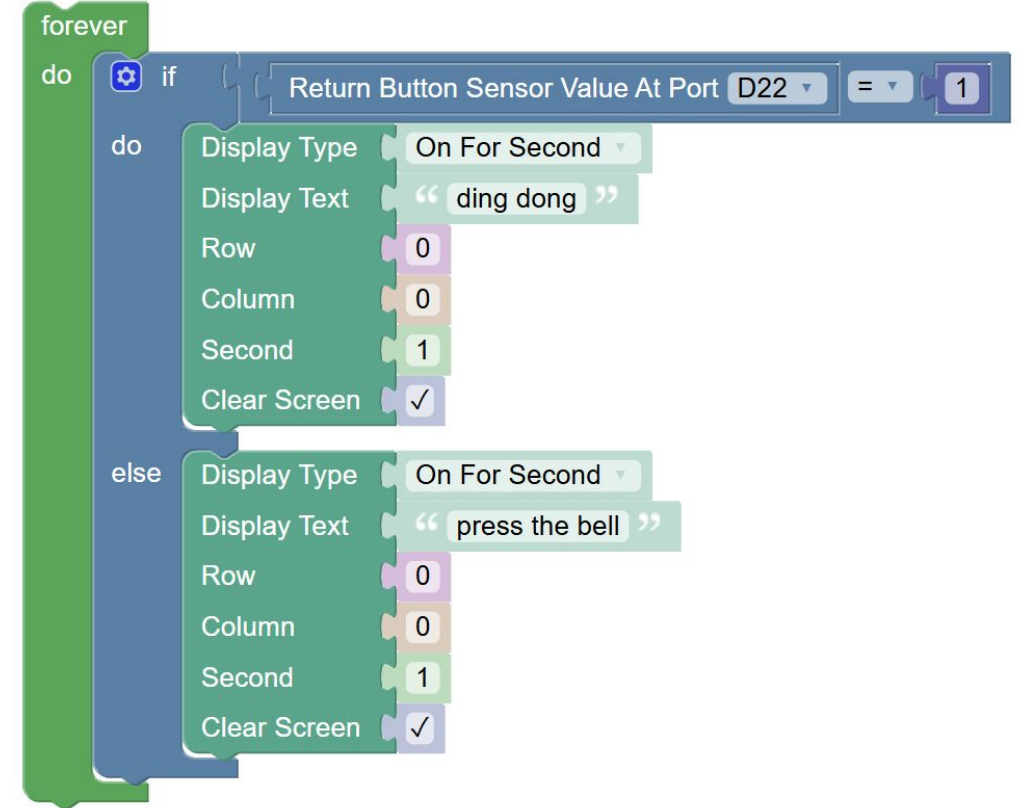


Q2:



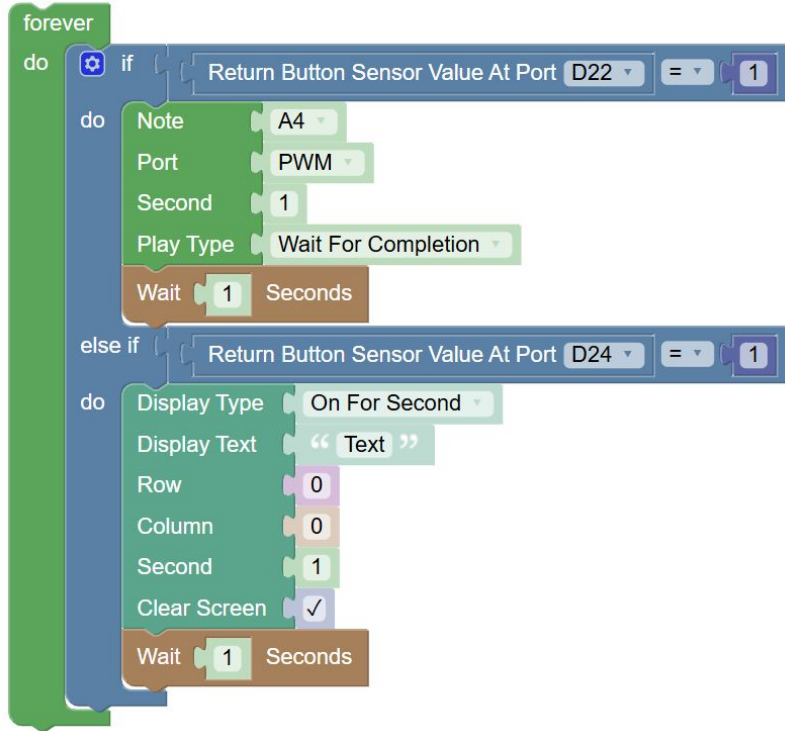
You can reduce the duration to make it more responsive

Q3:

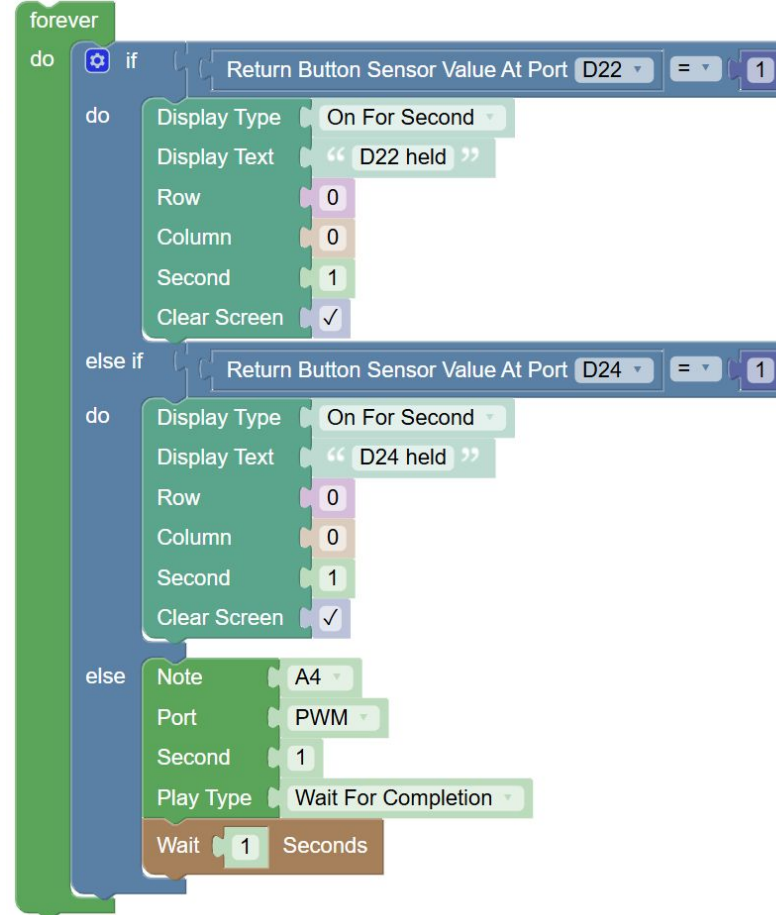


Answers

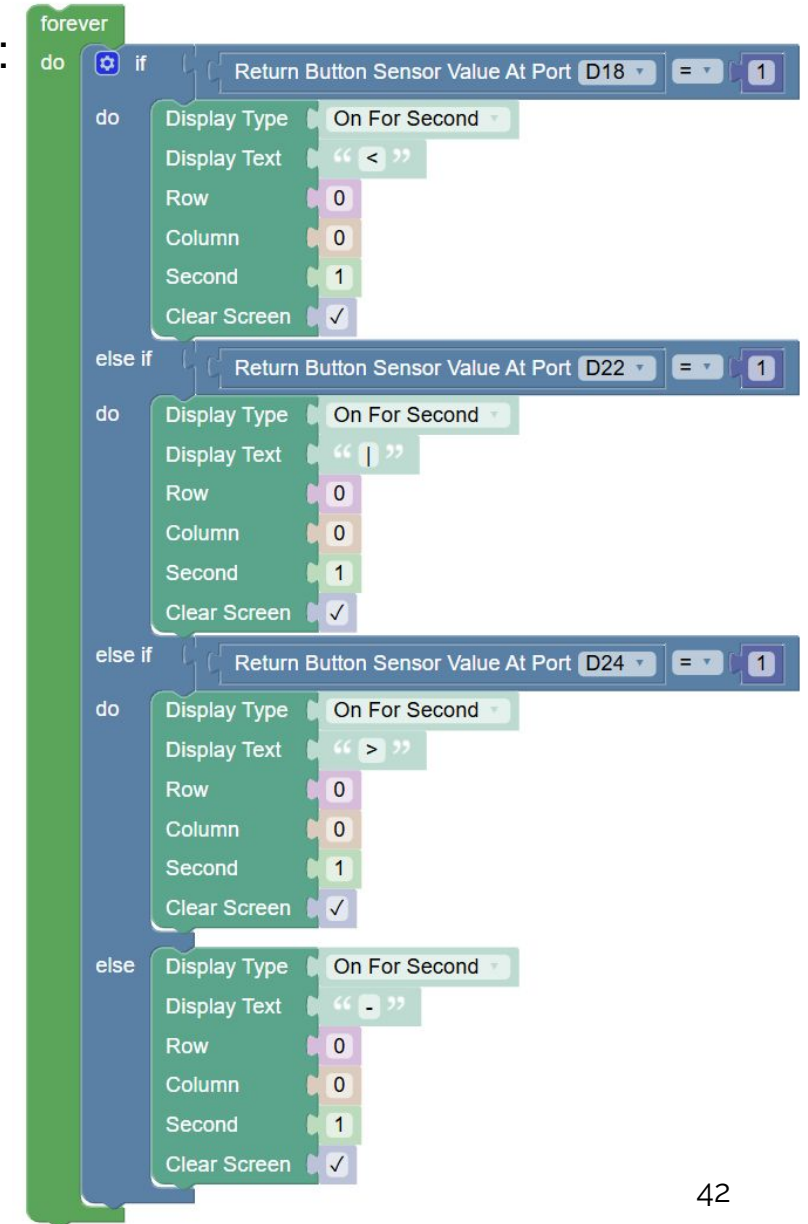
Q4:



Q5:



Q6:



2005 Logic Operator

Learning

The **AND** and **OR operators** are logic operators used to combine multiple conditions.

For **AND operator**, the combined condition is true only when **all** the individual conditions are true. For **OR operator**, the combined condition is true when **at least one** of the individual conditions is true.

In this unit, we will learn how to use logic operators to detect different button press combinations.

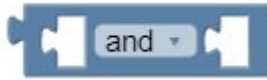
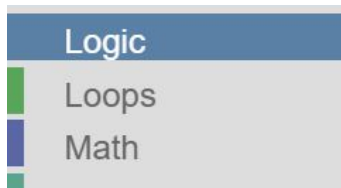
In class, you will be guided to explore the following:

- How to detect different combinations of button presses.
- How to stop looping.
- How to combine multiple logic operators.

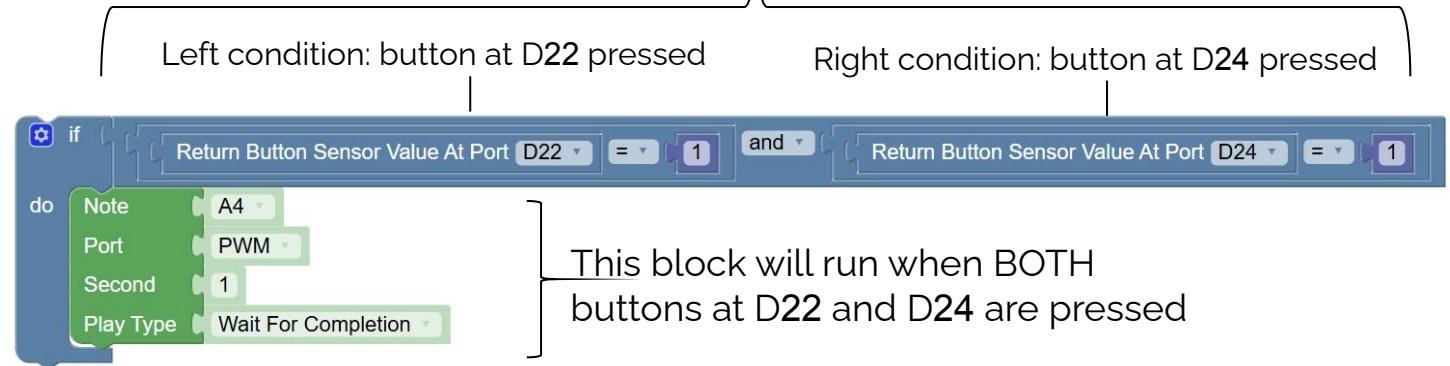
2005 Logic Operator

AND

For checking if two conditions are **BOTH** true



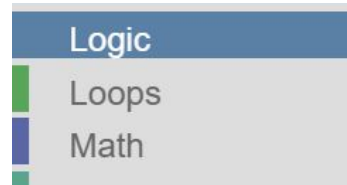
This condition will be true if the left and right conditions are **BOTH** true



2005 Logic Operator

OR

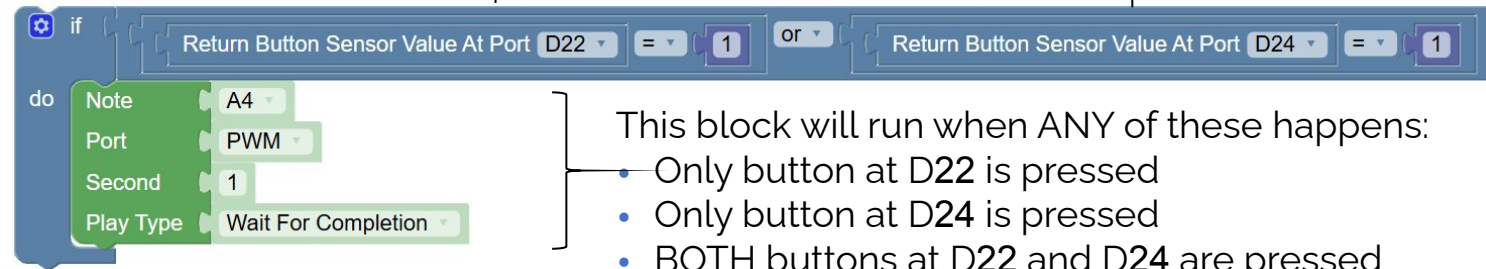
For checking if **AT LEAST ONE** of the conditions is true



This condition will be true if either the left or right or both conditions are true

Left condition: button at D22 pressed

Right condition: button at D24 pressed



2005 Logic Operator

COMBINING OPERATORS

You can check if multiple conditions are **ALL** true



You can also check if **AT LEAST ONE** of multiple conditions is true

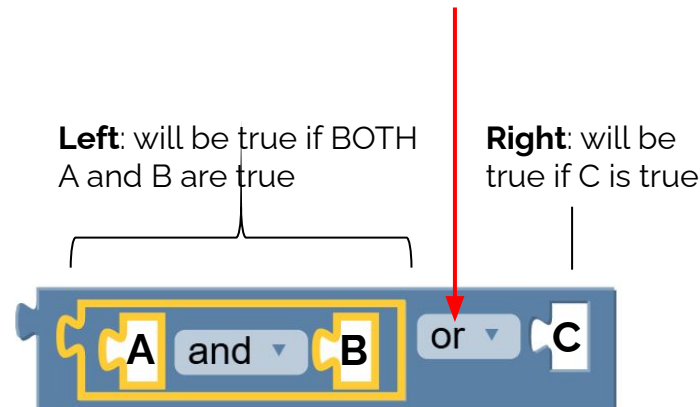


2005 Logic Operator

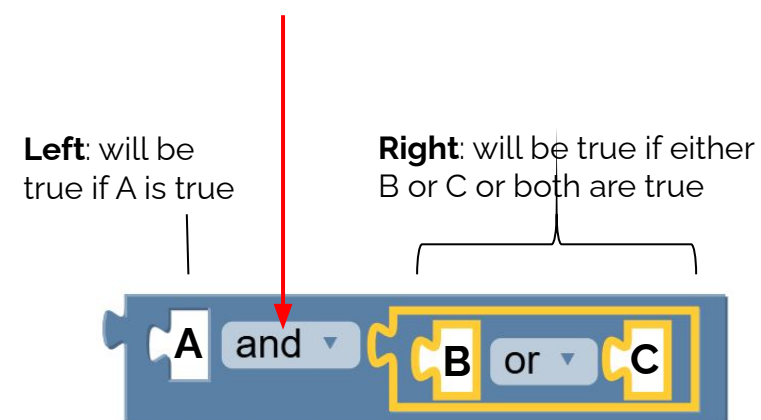
AND + OR

You can mix AND and OR blocks to form complex conditions but be careful of how the logic works

This condition will be true if either the left **or** right **or** both are true



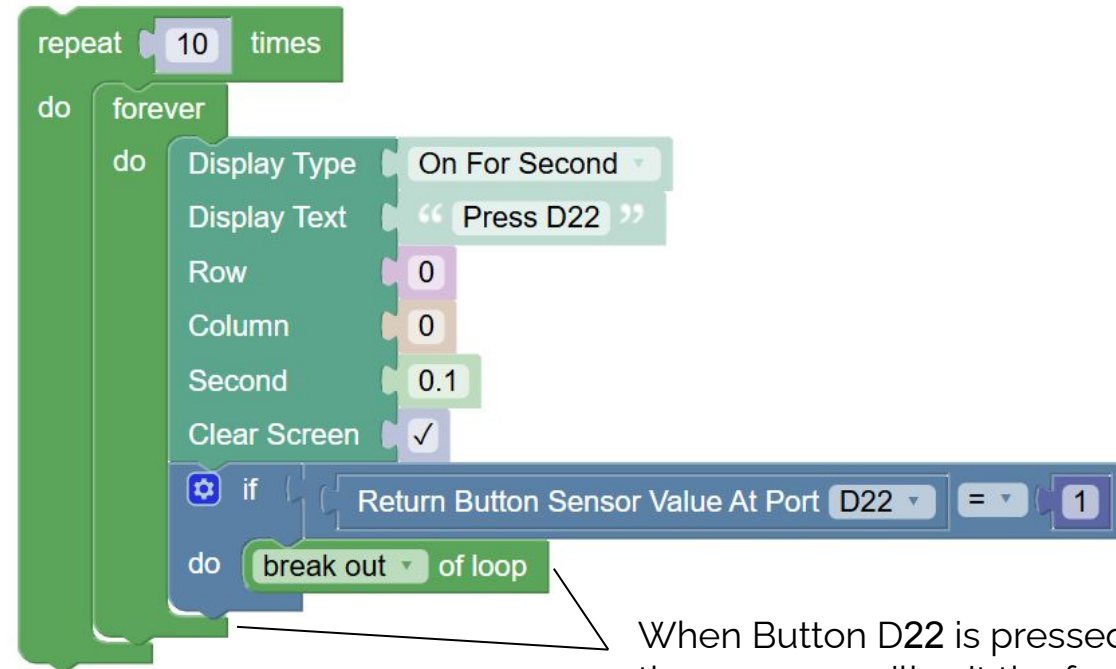
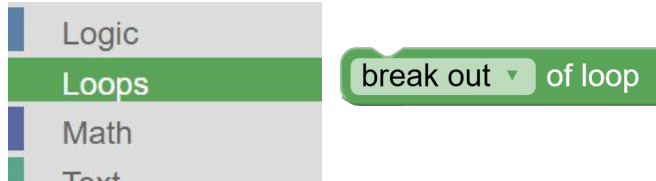
This condition will be true if both left **and** right are true



2005 Logic Operator

Break from loop

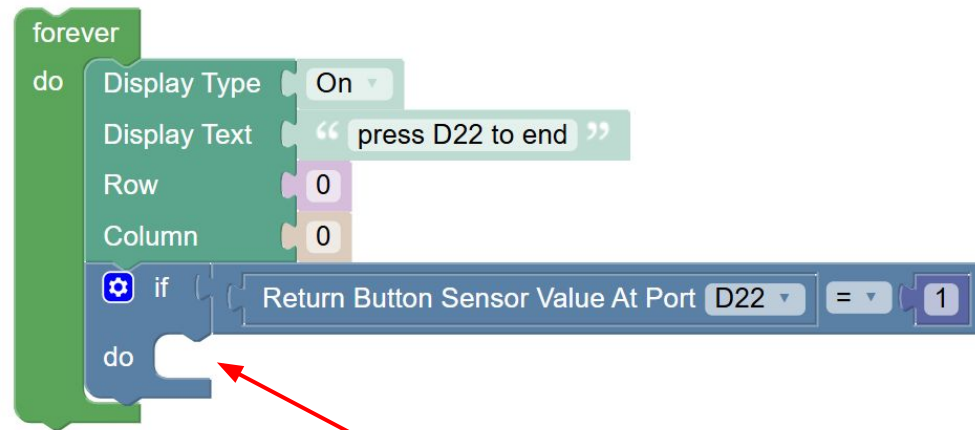
Exit from the **nearest** loop



When Button D22 is pressed, the program will exit the forever loop only and gets back into for 10 times. Hence, you need to press 10 times to end the program.

Exercise Q1

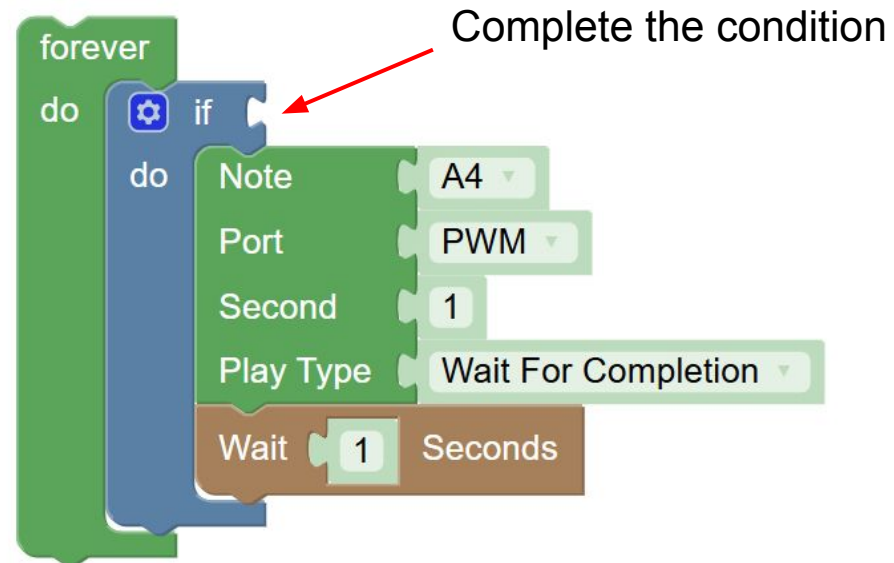
The code below shows a message. Complete the code within the IF block so that when D22 is pressed, the program will **clear the screen and stop**.



Complete the code

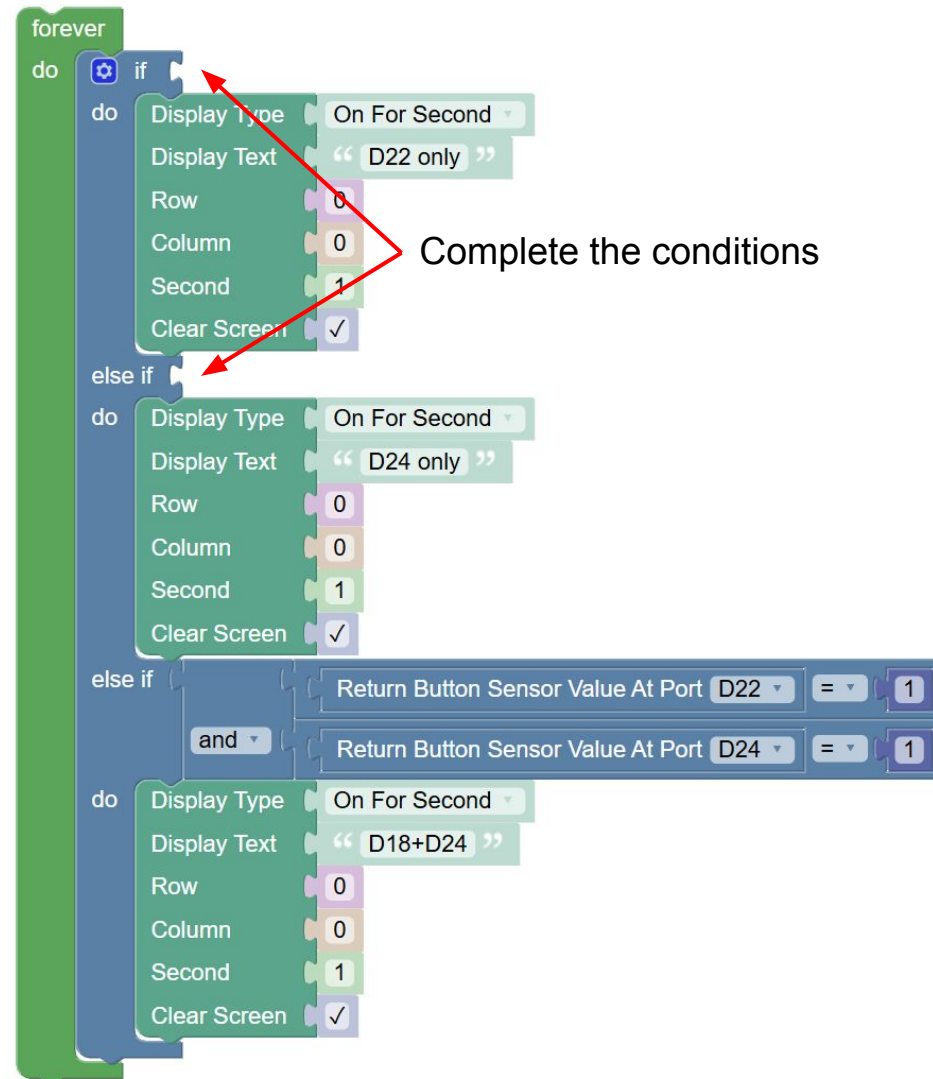
Exercise Q2

Complete the IF block condition so that when **D22 and D24 are pressed together**, it will make a beeping sound.



Exercise Q3

Complete the code below so that it displays the button(s) pressed.

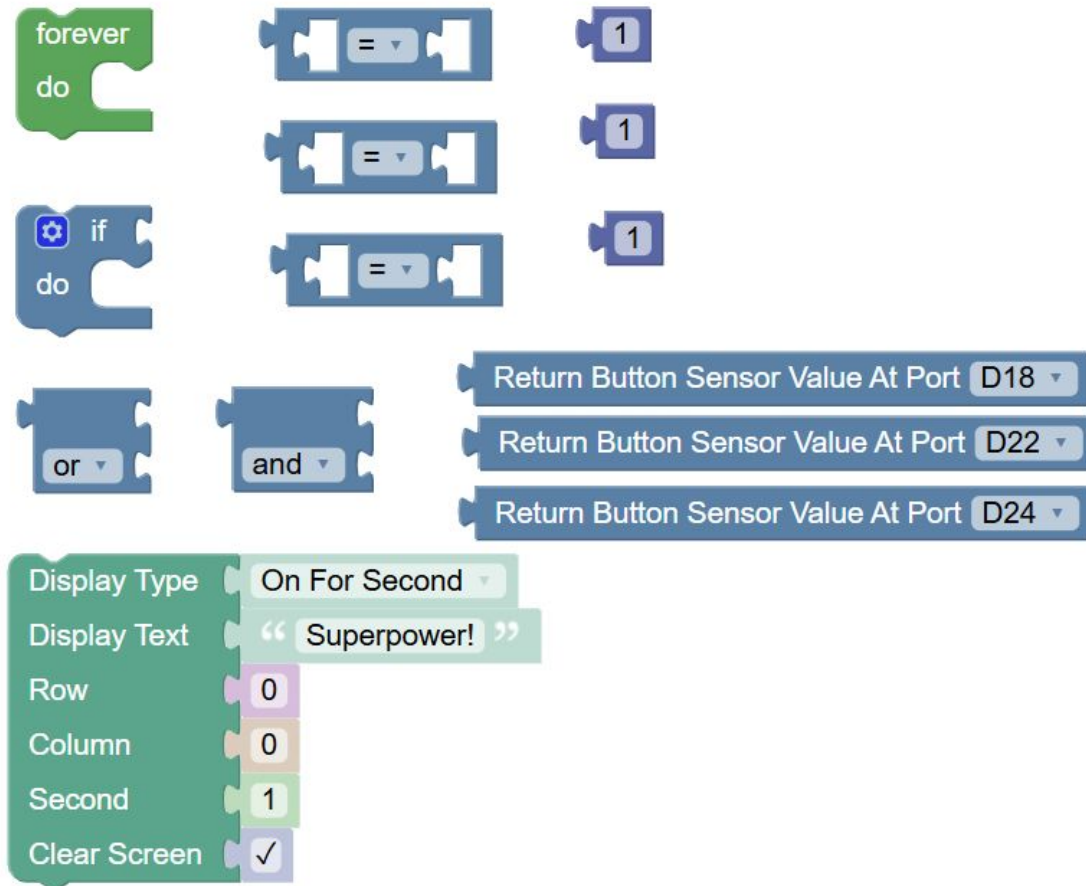


Exercise Q4

In a game program, three buttons can be used to activate a superpower.

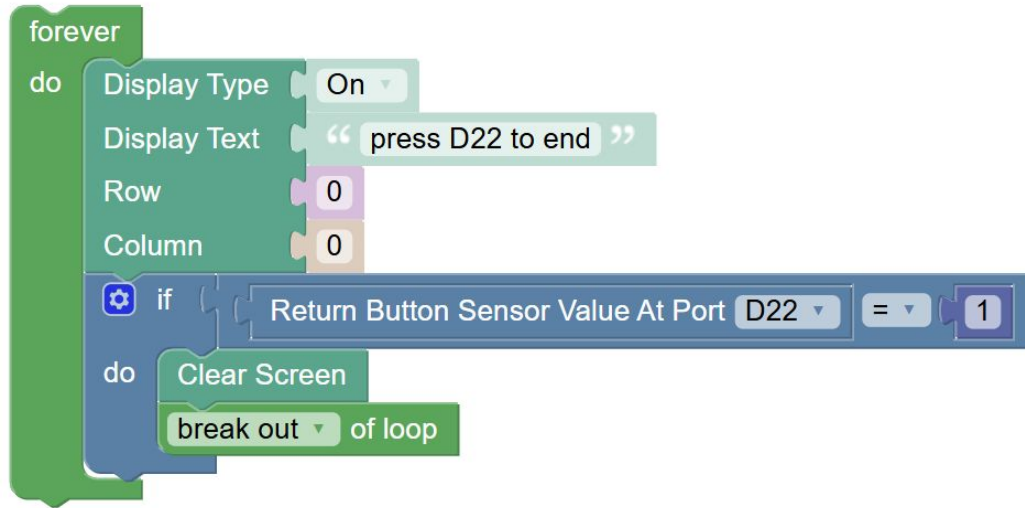
- D22, or
- D18 + D24

Use the blocks below to form the code that represents the above.

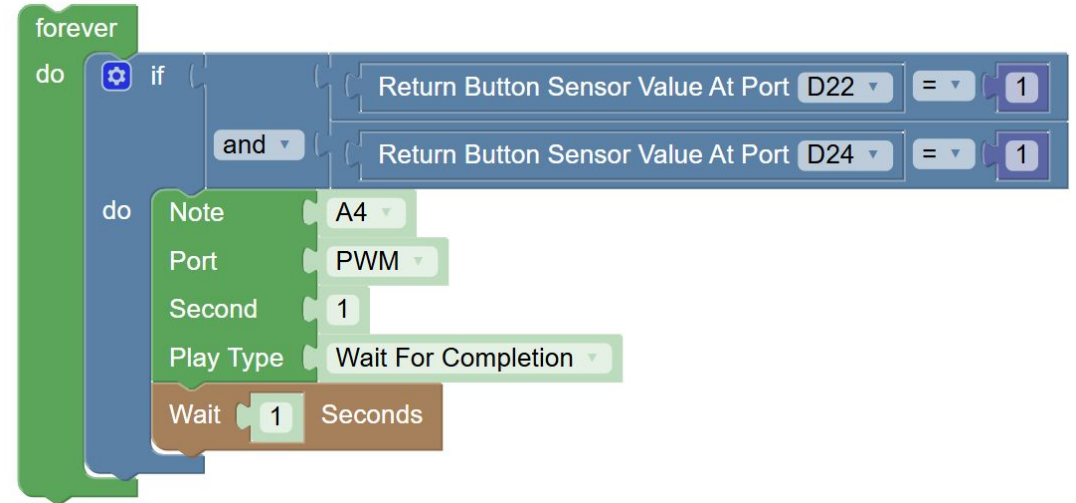


Answers

Q1:

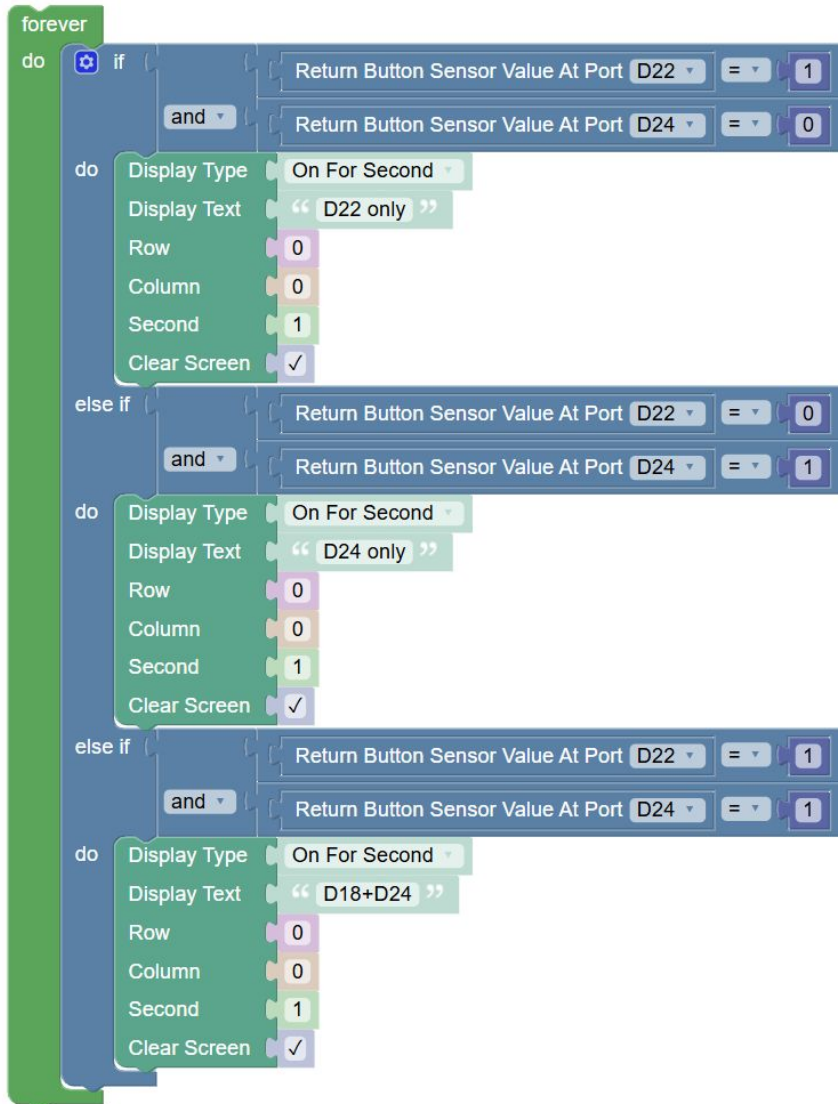


Q2:

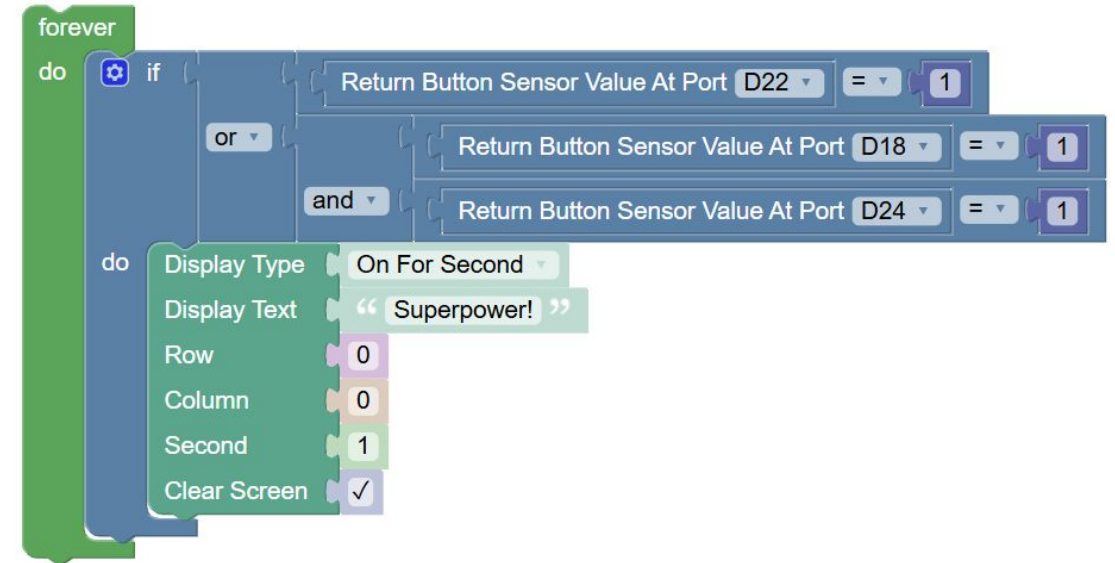


Answers

Q3:



Q4:



2006 Ultrasonic Sensor

Learning

The **Ultrasonic distance sensor** is used to detect obstacle distances. We will learn about range and how to form conditions to check for distance range using logic and comparison operators.

In class, you will be guided to explore the following:

- Understand Ultrasonic distance sensor and its blocks.
- Understand comparison operators (inequalities).
- Learn about continual and disjoint ranges.
- How to form conditions to check for ranges.

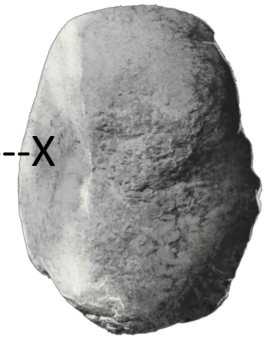
2006 Ultrasonic Sensor

Ultrasonic Distance Sensor

For detecting object in front of the sensor



It can measure distance from 0 to about 500cm



Button Sensor

Ultrasonic Sensor

Gyro Sensor

Return Distance in Centimetres At Port D26

Send the distance value to the robot

The port where the sensor is plugged

Wait Until Distance at Port D26 > 1 CM

Wait until the range condition is met
(in this example is distance less than 5 cm)

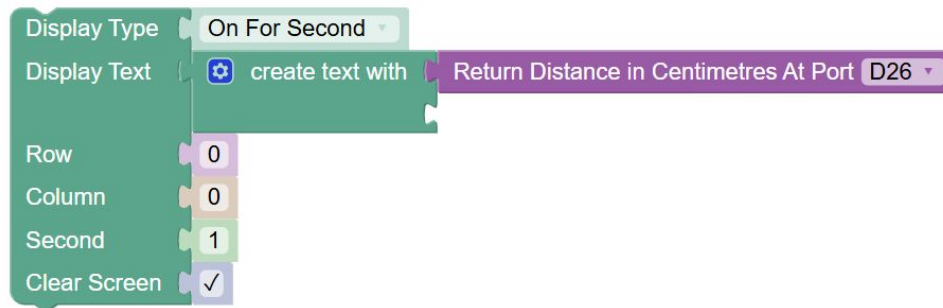
Select your comparison operator:

- = equal to
- ≠ not equal to
- < smaller than
- > greater than
- ≤ smaller than or equal to
- ≥ greater than or equal to

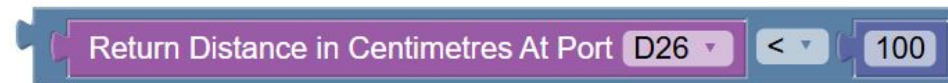
2006 Ultrasonic Sensor

Ultrasonic Distance Sensor

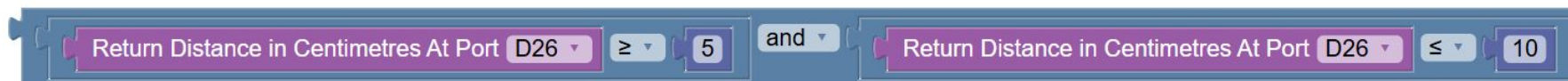
Checking range



The distance scanned often changes rapidly. Hence, instead of using the equal operator, we normally use the smaller/bigger than operators to check for ranges as shown below.



Condition to check if ultrasonic distance is less than 100cm

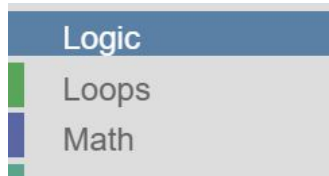


Condition to check if ultrasonic distance is from 5 to 10cm away

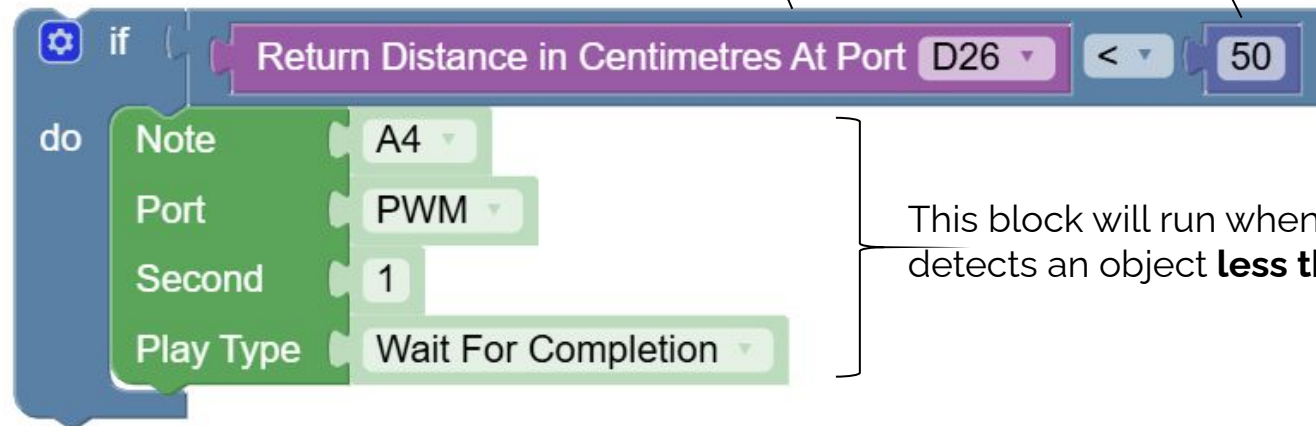
2006 Ultrasonic Sensor

Smaller than <

For checking if a value is less than another



This condition will be true if the left value is less than the right value

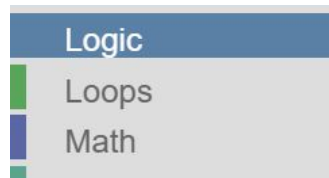


This block will run when the ultrasonic sensor detects an object **less than 50cm** away

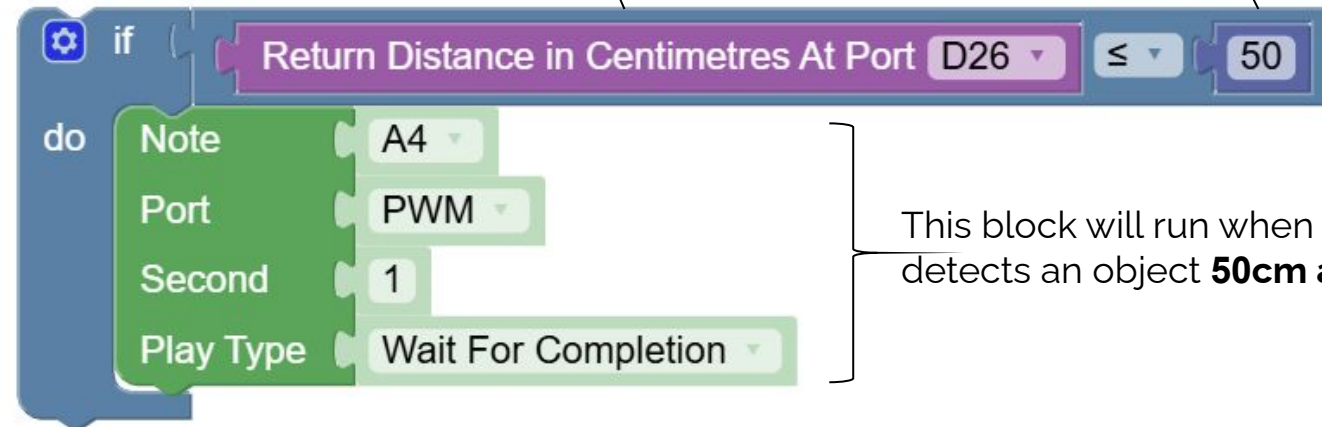
2006 Ultrasonic Sensor

Smaller than or Equal to $\leq$

For checking if a value is less than or equal to another



This condition will be true if the left value is less than or equal to the right value

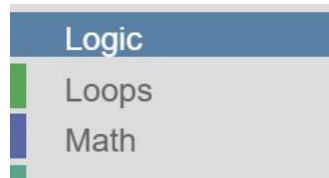


This block will run when the ultrasonic sensor detects an object **50cm away or less**

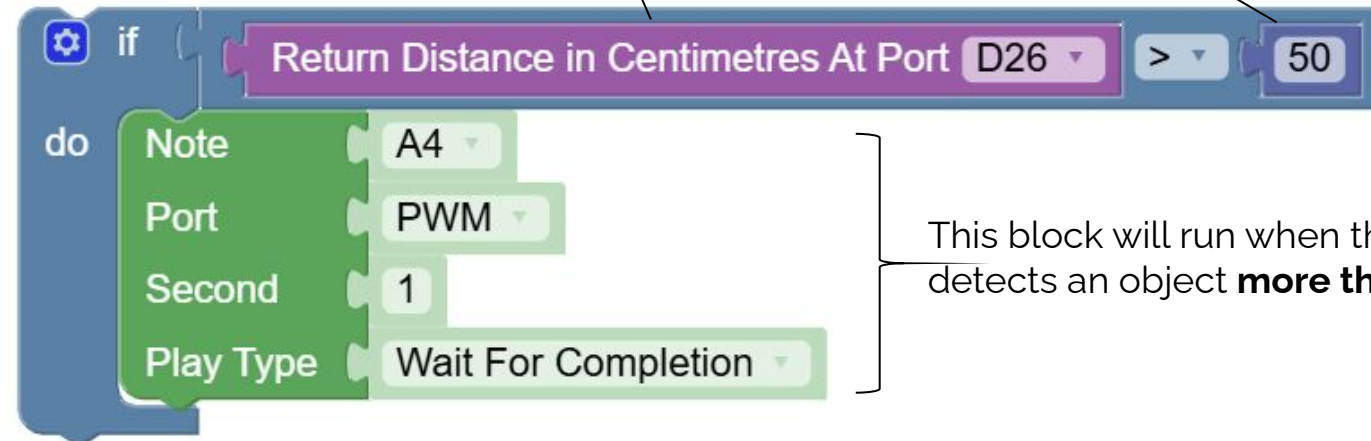
2006 Ultrasonic Sensor

Greater than >

For checking if a value is more than another



This condition will be true if the left value is more than the right value

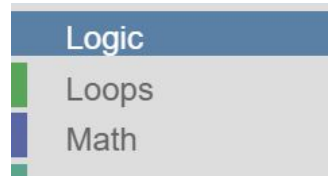


This block will run when the ultrasonic sensor detects an object **more than 50cm** away

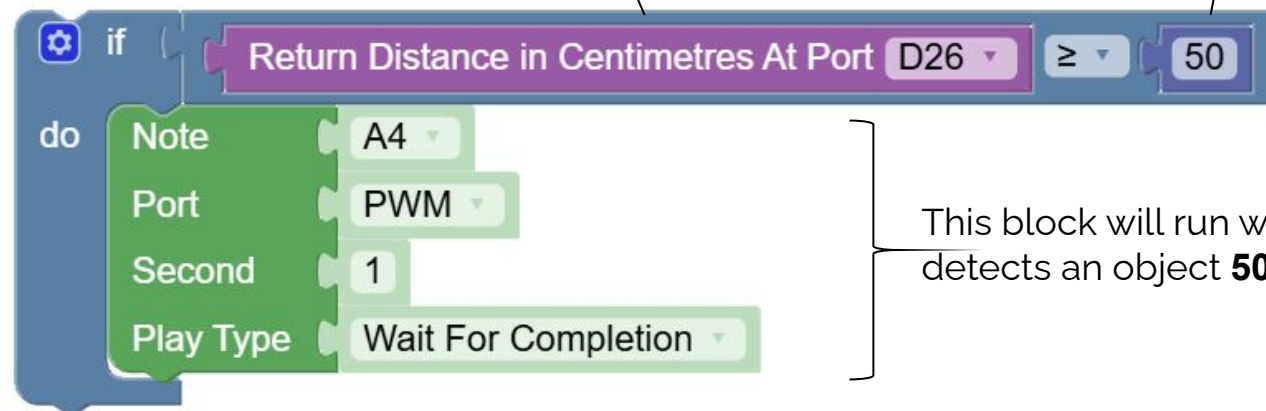
2006 Ultrasonic Sensor

Greater than or Equal to $\geq$

For checking if a value is more than or equal to another



This condition will be true if the left value is more than or equal to the right value

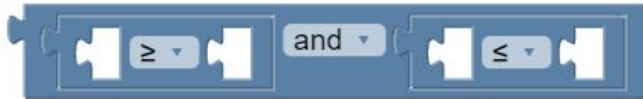


This block will run when the ultrasonic sensor detects an object **50cm away or more**

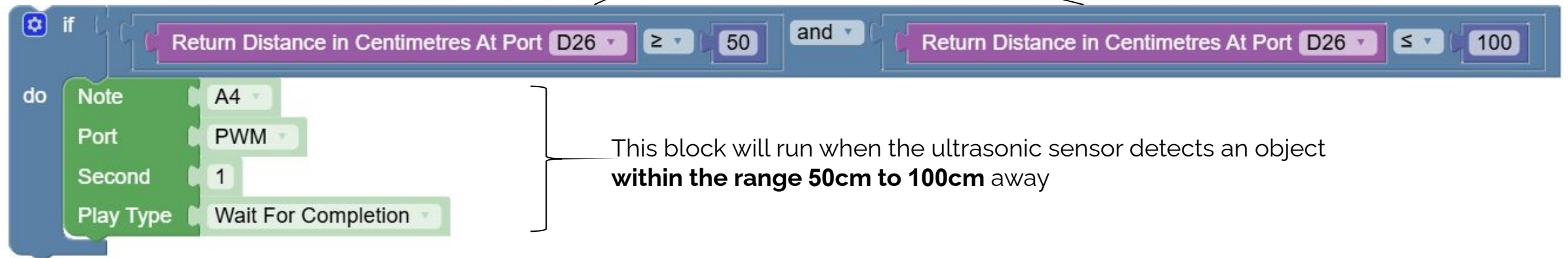
2006 Ultrasonic Sensor

Checking within a range

For checking if a value within a range



This condition will be true if both the left and right conditions are true



This block will run when the ultrasonic sensor detects an object **within the range 50cm to 100cm** away

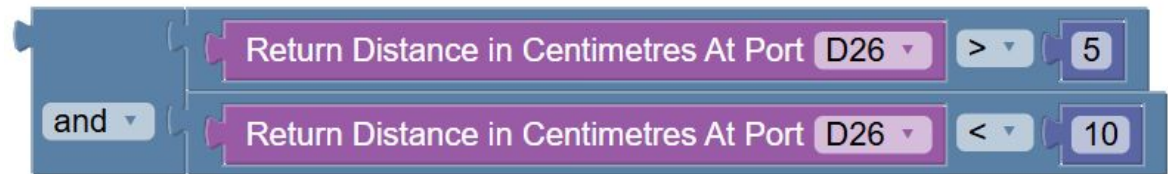
Exercise Q1

Write down **True** or **False** for the following comparisons:

- 1) $9 > 5$ _____
- 2) $9 \geq 5$ _____
- 3) $5 < 9$ _____
- 4) $5 \leq 9$ _____
- 5) $3 > 3$ _____
- 6) $3 < 3$ _____
- 7) $3 \geq 3$ _____
- 8) $3 \leq 3$ _____
- 9) $0 > -1$ _____
- 10) $0 \leq -1$ _____
- 11) $-2 < -1$ _____
- 12) $-2 \leq -1$ _____
- 13) $-4 \geq -4$ _____
- 14) $-4 \leq -4$ _____

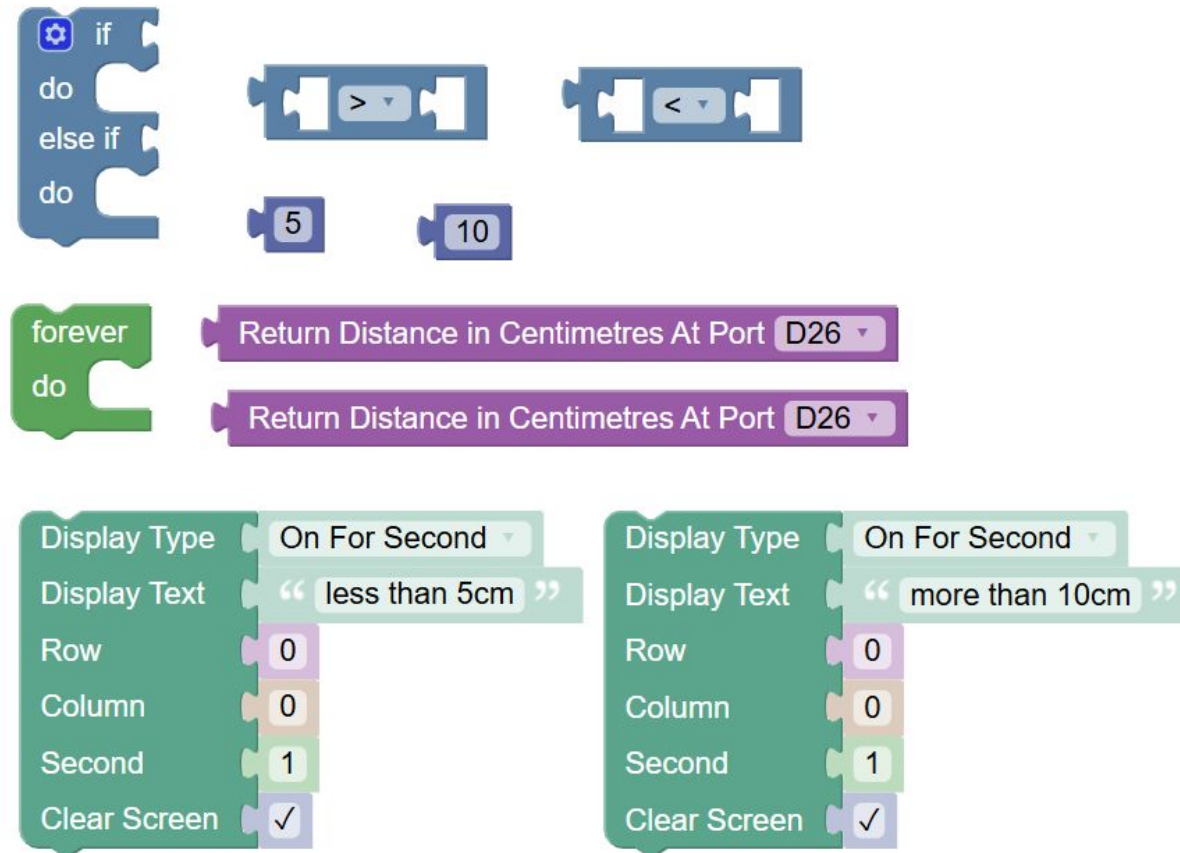
Exercise Q2

Which of the following conditions check that the obstacle distance is **within 5cm to 10cm** away (including 5cm and 10cm)?

- A) 
- B) 
- C) 
- D) 

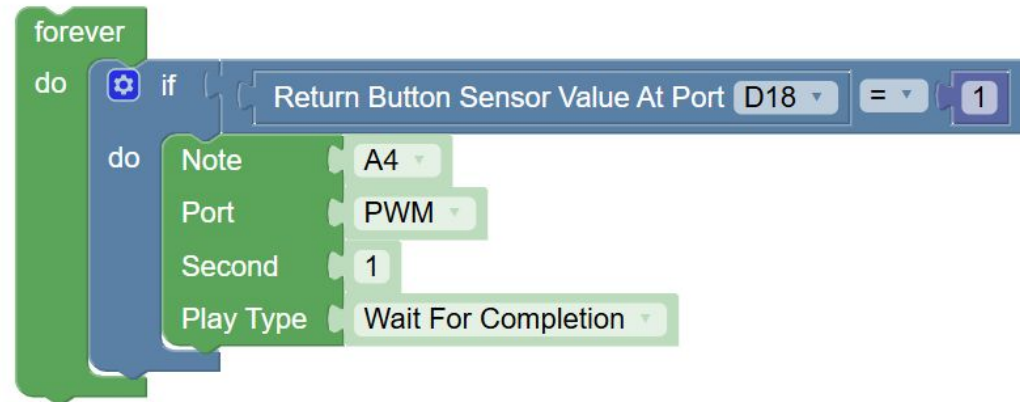
Exercise Q3

Using the blocks below, create a code that detects the obstacle distance and display the correct message.



Exercise Q4

The code below plays a sound when Button D18 is pressed. By adding more blocks, make the code play the sound only when Button D18 is pressed **and also the obstacle distance is 10cm to 20cm away**.



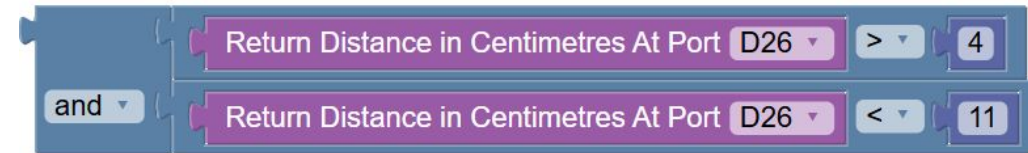
Answers

Q1:

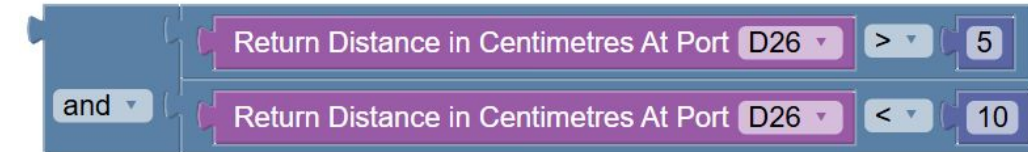
- 1) $9 > 5$ True
- 2) $9 \geq 5$ True
- 3) $5 < 9$ True
- 4) $5 \leq 9$ True
- 5) $3 > 3$ False
- 6) $3 < 3$ False
- 7) $3 \geq 3$ True
- 8) $3 \leq 3$ True
- 9) $0 > -1$ True
- 10) $0 \leq -1$ False
- 11) $-2 < -1$ True
- 12) $-2 \leq -1$ True
- 13) $-4 \geq -4$ True
- 14) $-4 \leq -4$ True

Q2: A and D

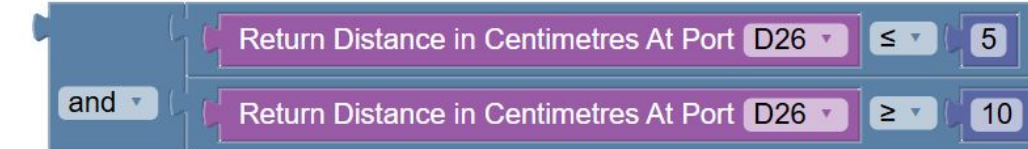
A) 5 to 10



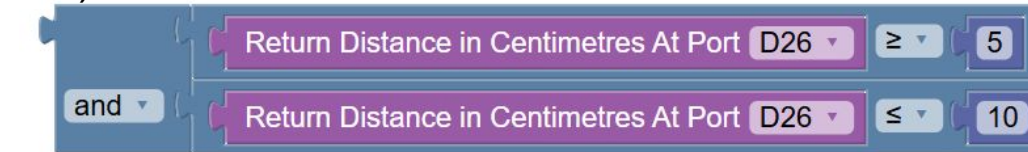
B) 6 to 9



C) 0 to 5 and 10 onwards (not possible)

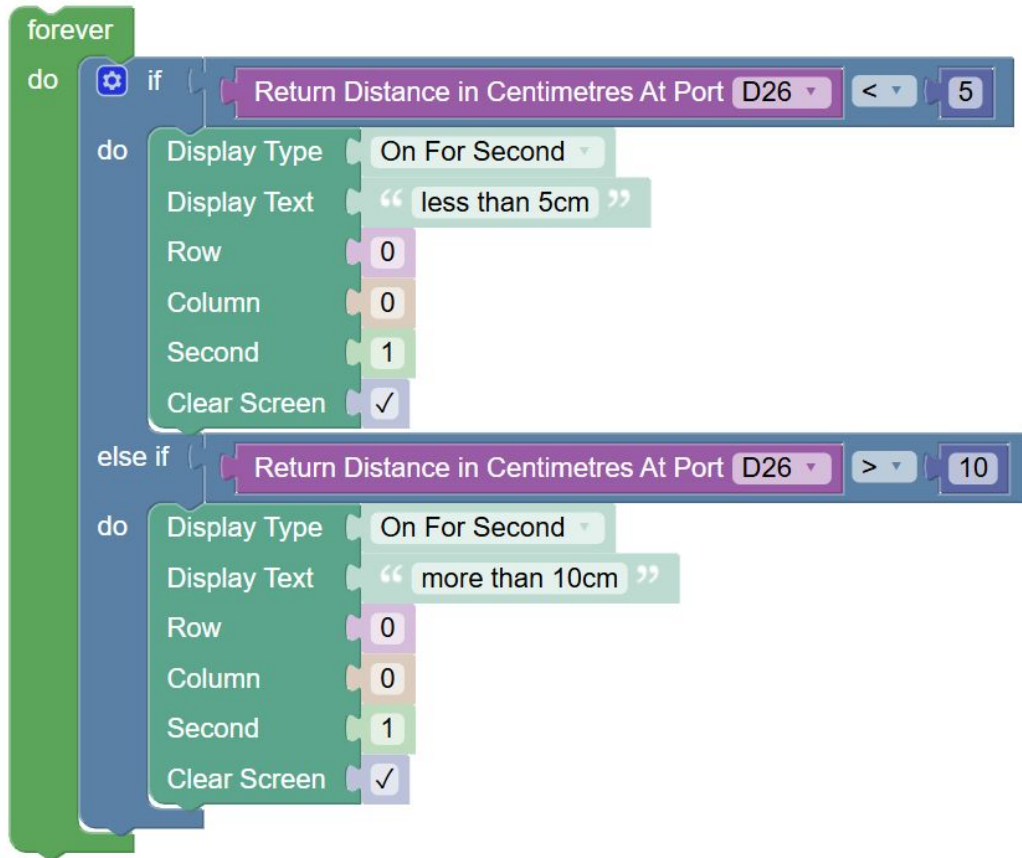


D) 5 to 10

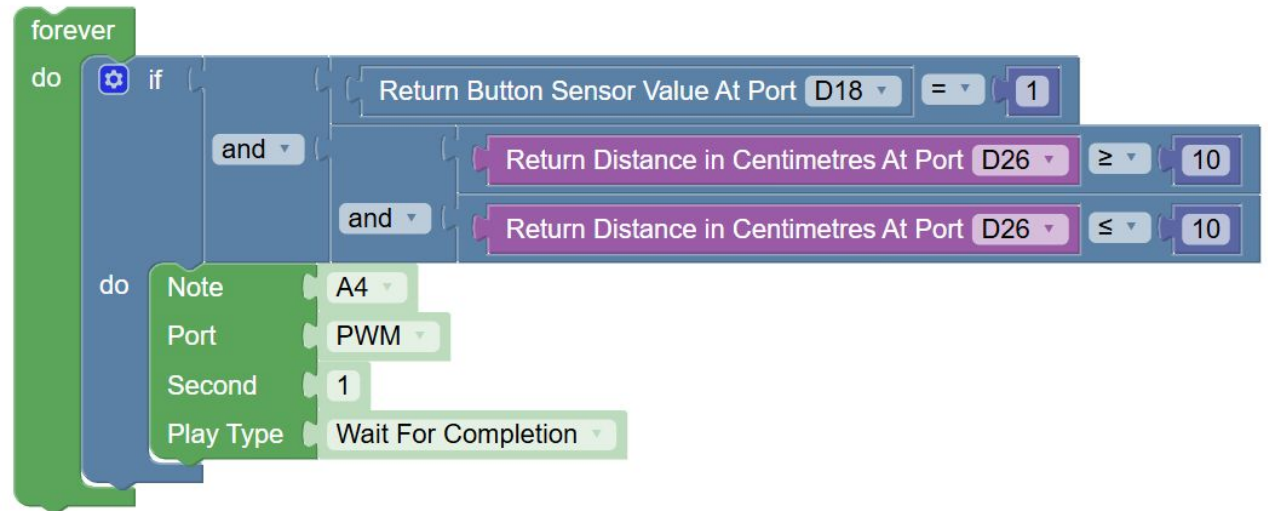


Answers

Q3:



Q4:



2007 Colour Sensor

Learning

The **Colour sensor** is used to detect the colour of an object placed before it.

In class, you will be guided to explore the following:

- Learn how to use the Colour sensor to detect object colour.
- Combining various sensors to make your program more interactive.

2007 Colour Sensor

Colour Sensor

For detecting colour



Return Color Name

Send the name of the colour being sensed to the robot

Return Reflected Light Intensity Value

Send the reflected light intensity value to the robot (not required in curriculum)

Return Ambient Light Intensity Value

Send the ambient (surrounding) light intensity value to the robot (not required in curriculum)

Wait Until Color Sensor Detect Red

Wait until the sensor detects the indicated colour

Red

Colour name: Yellow, Orange, Red, White, Green, Purple, Blue, Black

But only White, Red, Orange, Green and Blue is used.

Return Color Name = Red

Condition to check that the colour.

Return Color Name = Red

You can also use text box in place of the colour block if the colour you wanted is not found in the colour block. Use first letter capitalisation. In your text.

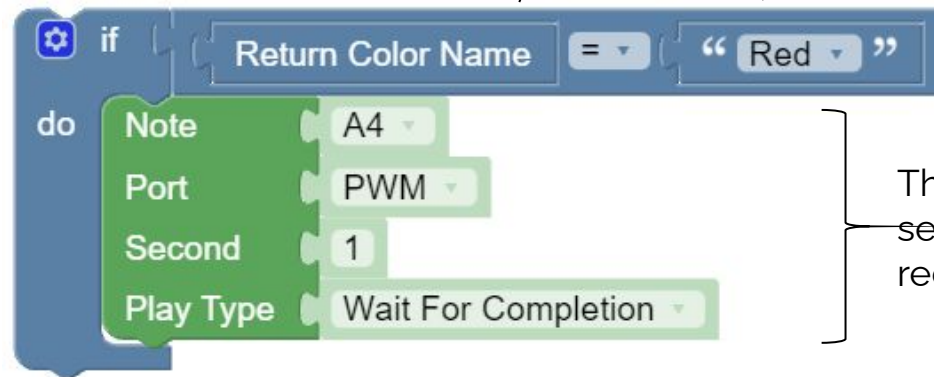
COMPARISON OPERATORS

Equal to =

For checking if a value is the same as another



This condition will be true if the left and right have the same value



This block will run when the colour sensor returns "Red" value (that is, red colour is being sensed)

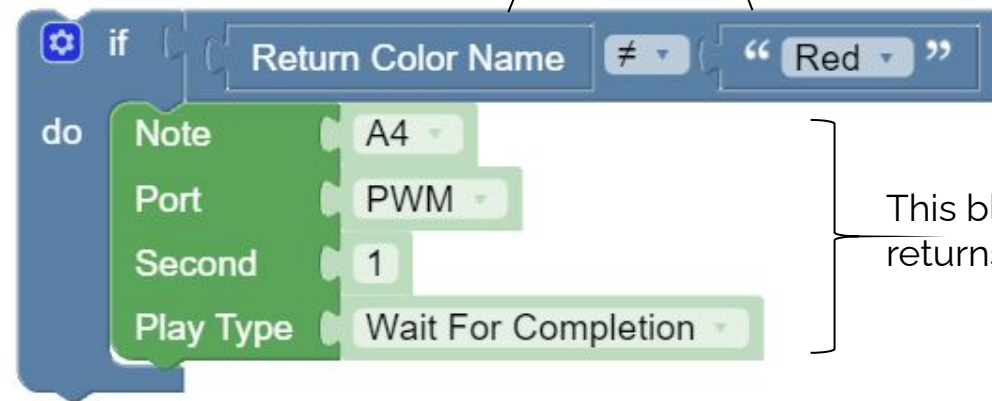
COMPARISON OPERATORS

Not Equal to ≠

For checking if a value is different from another



This condition will be true if the left and right do not have the same value



This block will run when the colour sensor returns any colour except "Red".

Exercise Q1

The Colour sensor on your Raspberry-Pi robot is calibrated to detect **five colours** correctly.
Write down which of the following are the colours.

Yellow

Orange

Red

White

Green

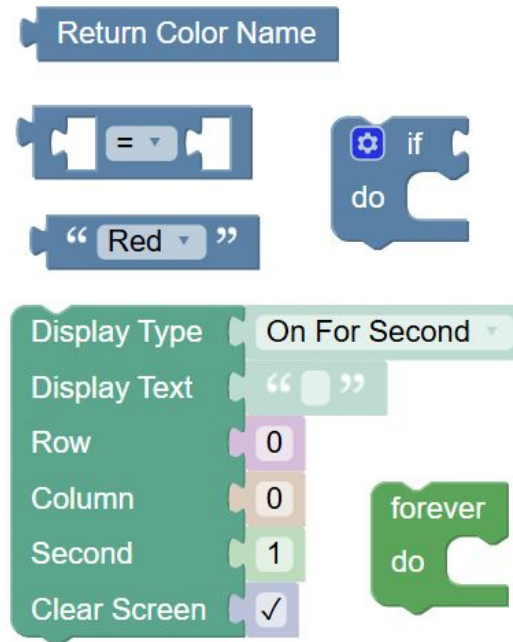
Purple

Blue

Black

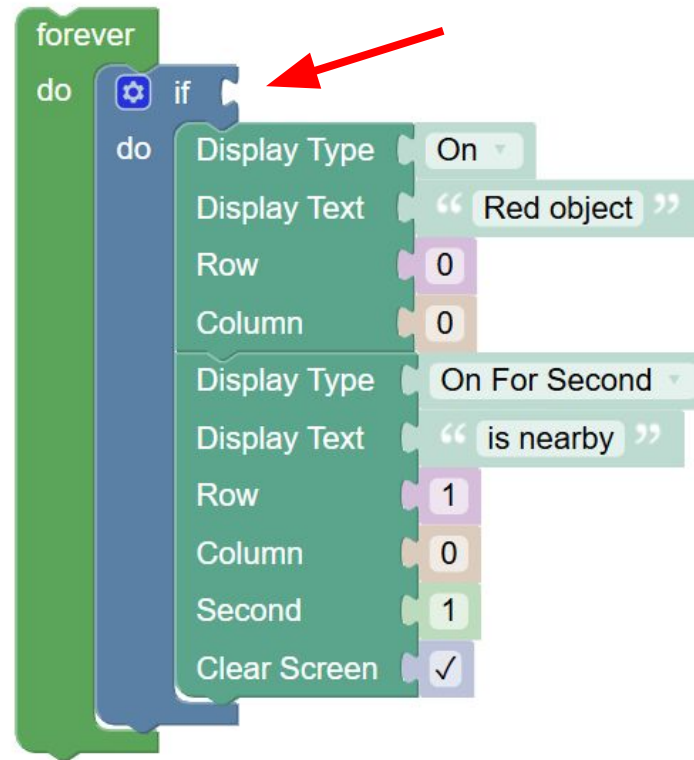
Exercise Q2

Using the blocks below, create a code to display “Go” when green is detected.



Exercise Q3

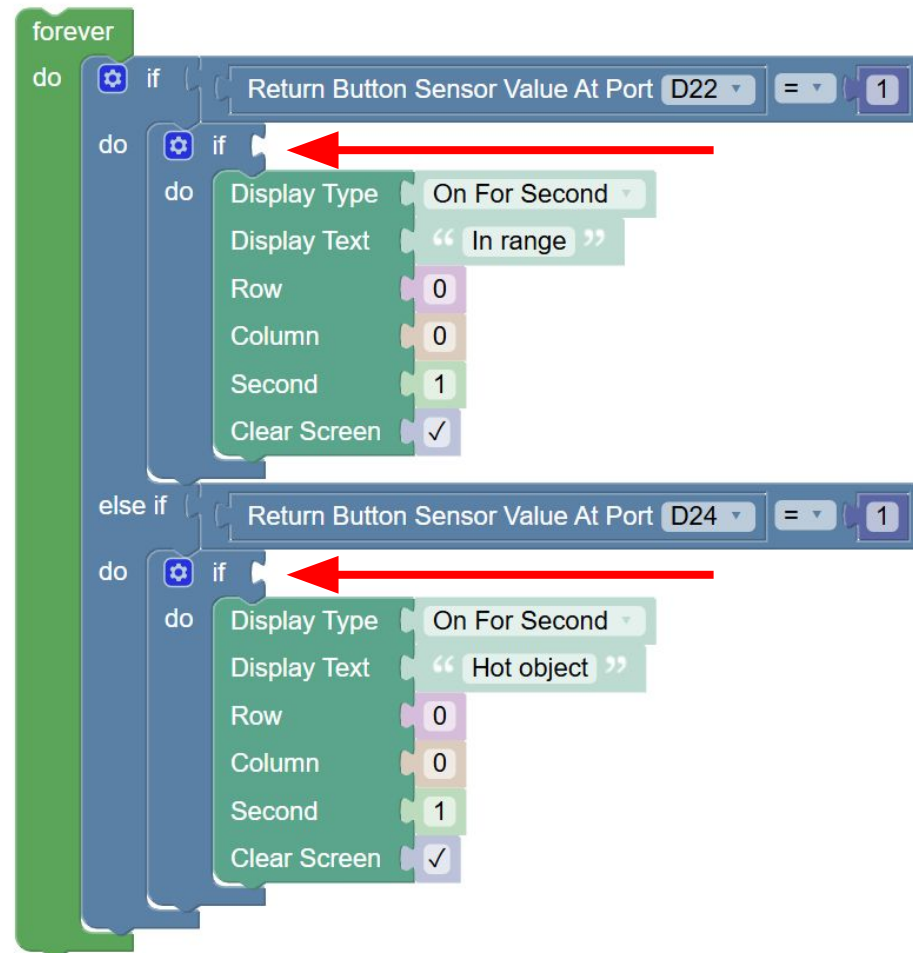
Complete the condition of the IF block below so that when an object is **within 5cm away** and **is red** in colour, the message “Red object is nearby” will appear.



Exercise Q4

Complete the conditions of the IF block so that the program performs the following:

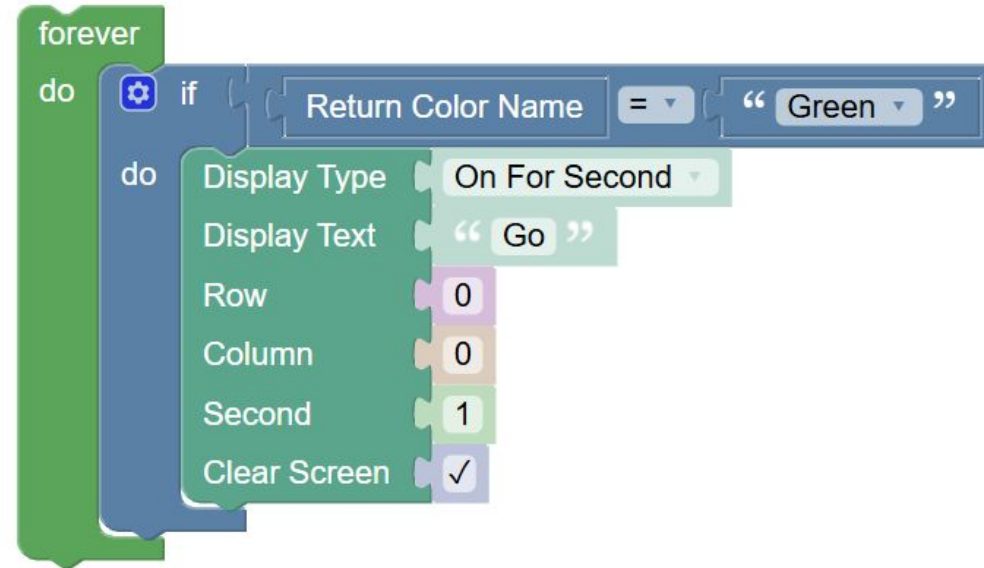
- If Button **D22 is pressed** and object is **within 5 to 10cm** away, display “In range”
- If Button **D24 is pressed** and object is **red or orange** in colour, display “Hot object”



Answers

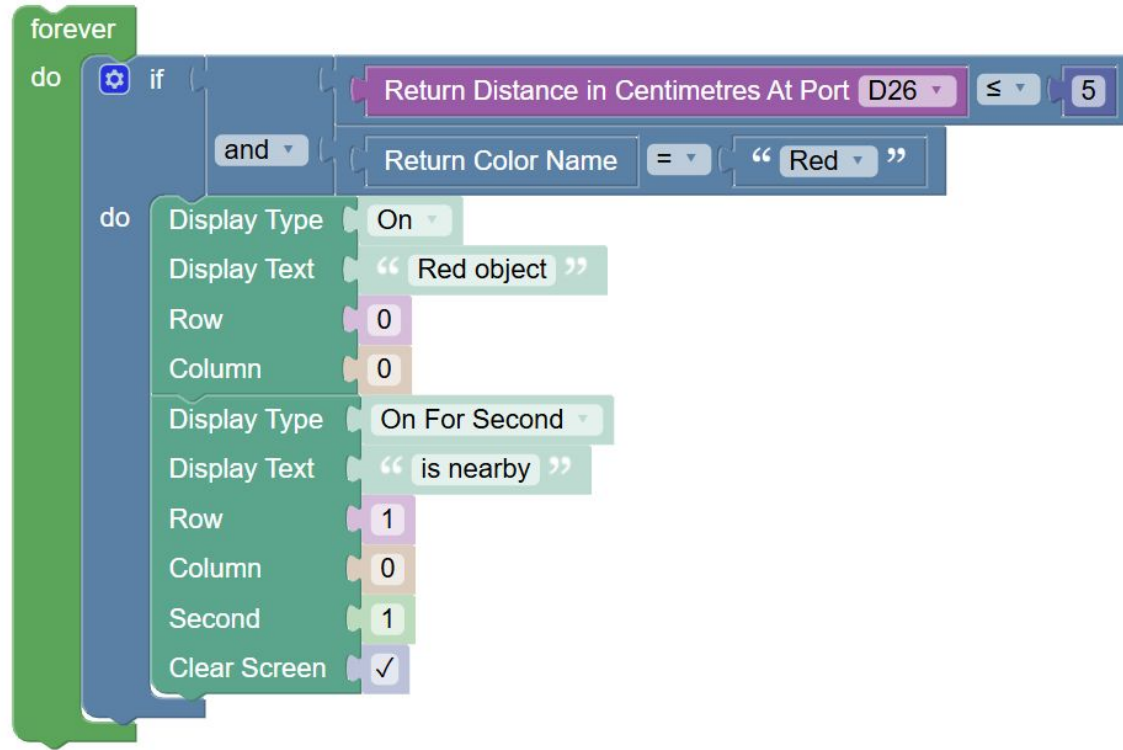
Q1: Orange
Red
White
Green
Blue

Q2:

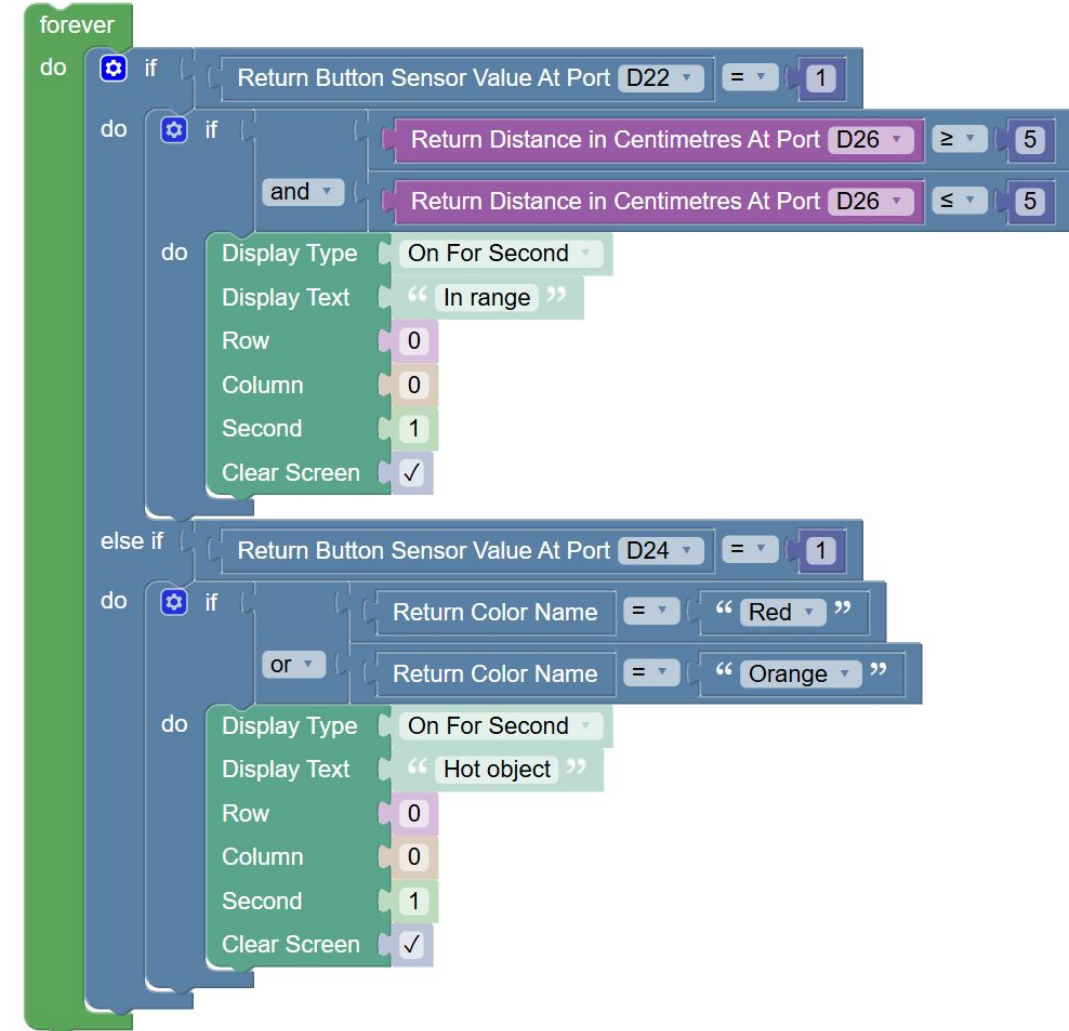


Answers

Q3:



Q4:



2008 Conditional Loop

Learning

Just like **If statements**, conditions are also used in **Repeat-while/Repeat-until loops**.

In Repeat-while loop, a set of code will be repeated when its condition is true. In Repeat-until loop, it repeats when the condition is False and stops when it is True.

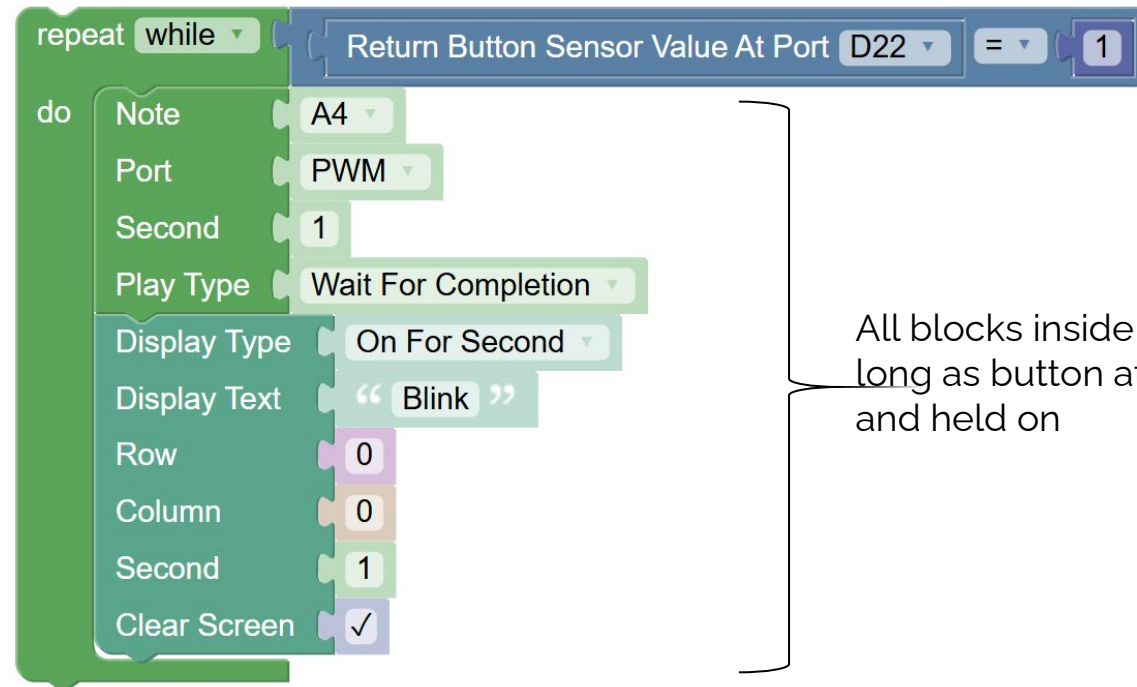
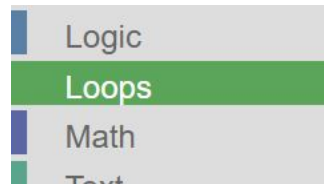
In class, you will be guided to explore the following:

- Learn how to use the Repeat-while/until loops.
- Wait-until blocks as shortcuts to Repeat-until loops.

2008 Conditional Loop

Repeat *while*

Repeat codes while a condition is true



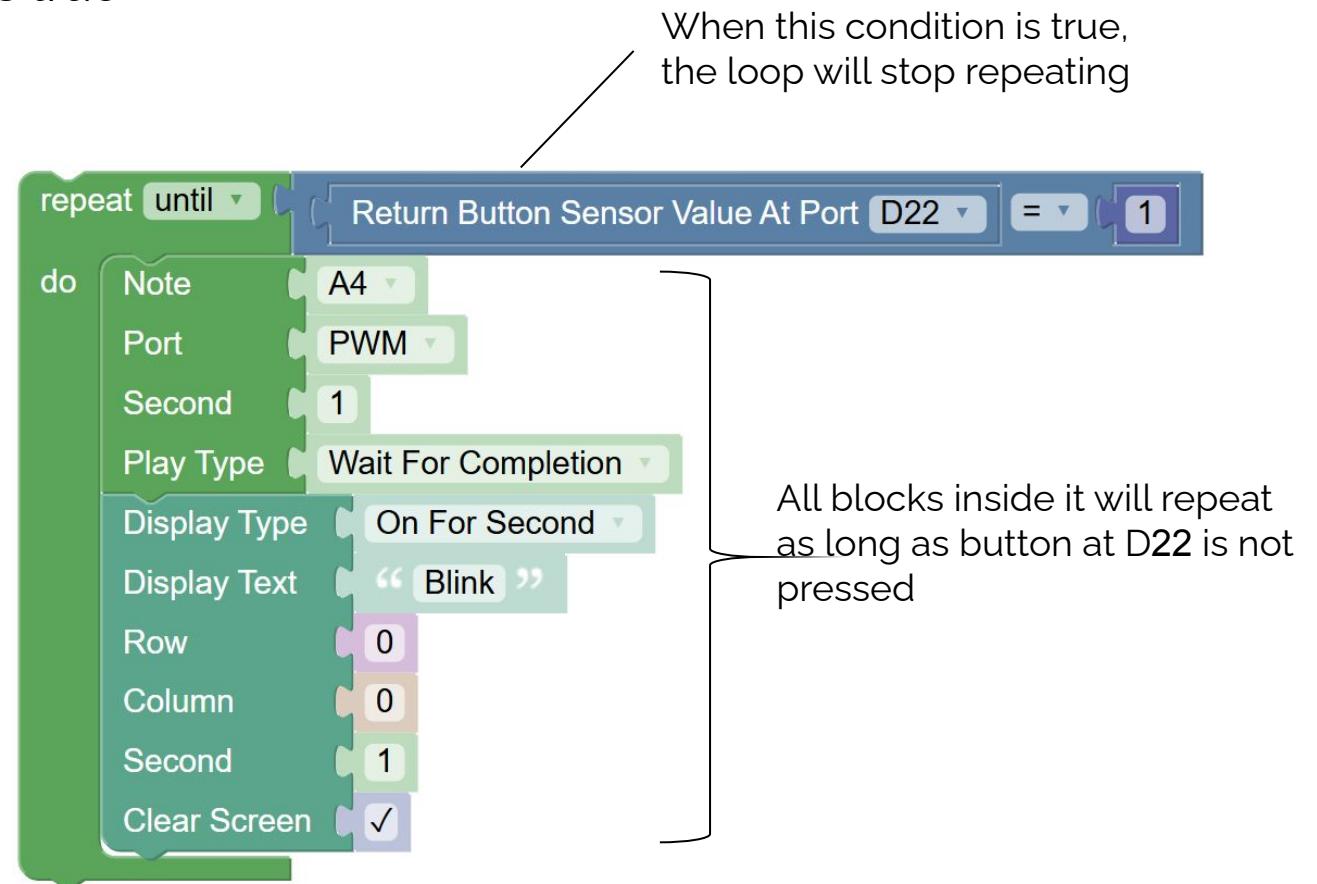
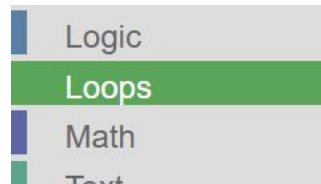
When this condition is true,
the loop will keep repeating

All blocks inside it will repeat as
long as button at D22 is pressed
and held on

2008 Conditional Loop

Repeat *until*

Repeat codes and stop when a condition is true



2008 Conditional Loop

Wait-until

Wait-until blocks are shortcuts to using Repeat-until loops. Below are some examples.



Keep waiting until Button D22 is pressed, then proceed to the next block.

Wait Until Button Sensor at Port D22 Is Pressed

Keep waiting until obstacle distance is more than 5cm, then proceed to the next.

Wait Until Distance at Port D22 > 5 CM

Keep waiting until red is detected then run to next block, then proceed to the next.

Wait Until Color Sensor Detect Red

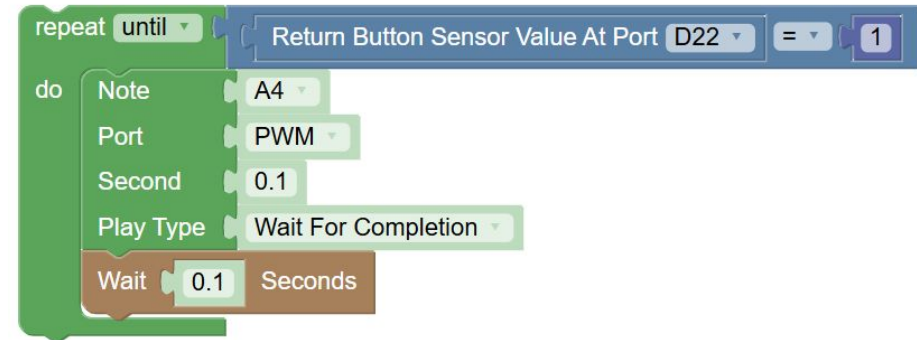
Exercise Q1

Study the codes shown.

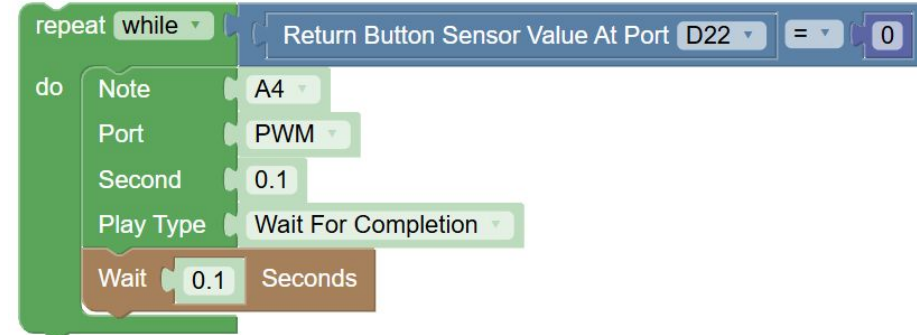
Which statement(s) below is/are true?

- A) Code 1 will keep beeping and stop when a button is pressed
- B) Code 1 will beep only when a button is pressed
- C) Code 2 will keep beeping and stop when a button is pressed
- D) Code 2 will beep only when a button is pressed
- E) Code 3 will keep beeping and stop when a button is pressed
- F) Code 3 will beep only when a button is pressed

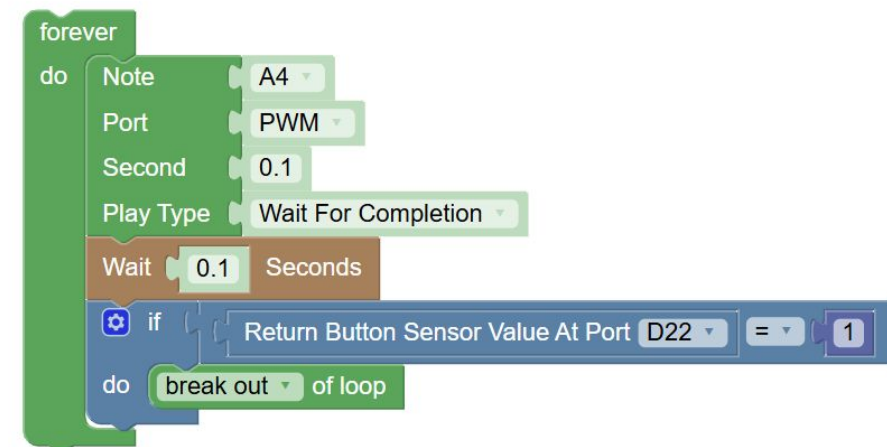
Code 1



Code 2

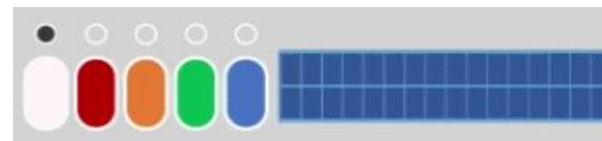
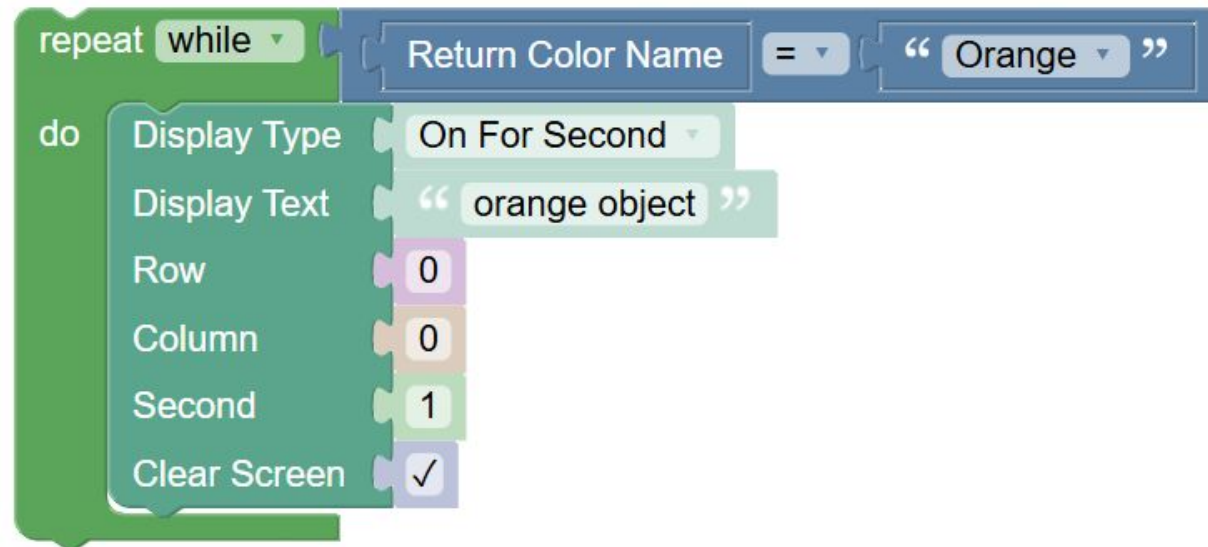


Code 3



Exercise Q3

The code below is supposed to display “orange object” whenever an orange object is flashed to the colour sensor. However, it is not working properly. Add any extra block(s) to make it work.

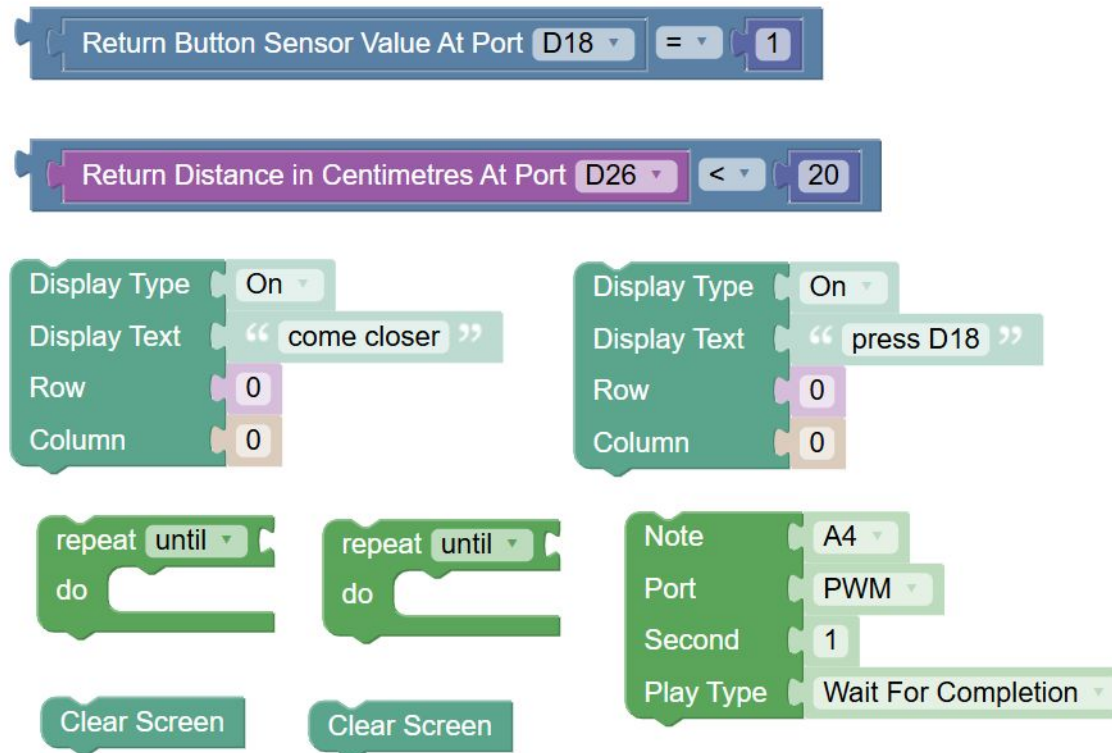


animated clip

Exercise Q3

Using the blocks below, create a program that performs the following:

1. It will display “**come closer**” until an object is **less than 20cm** away.
2. It will then display “**press D18**” until you **press Button D18**.
3. The program will clear screen and end.

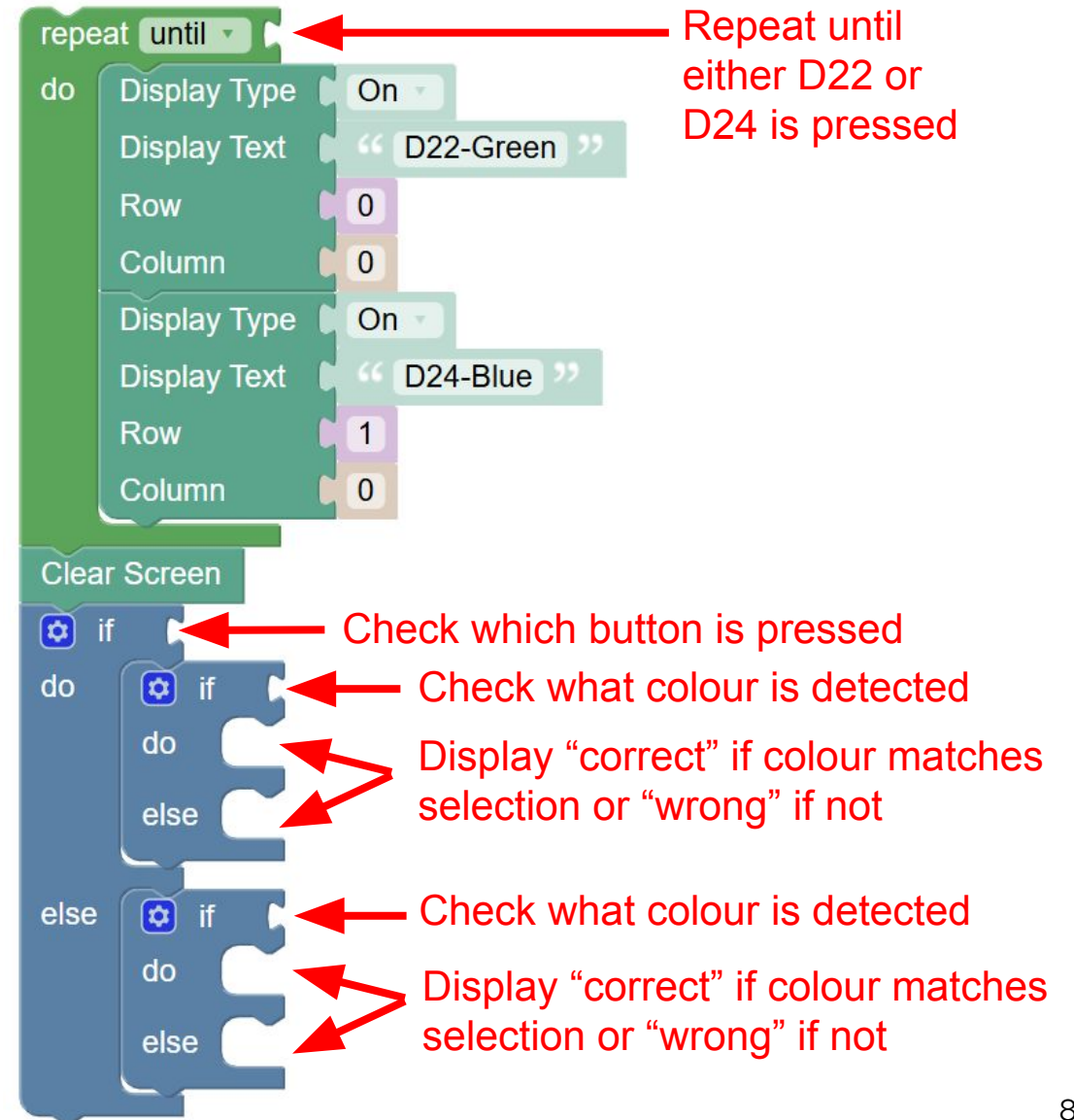


Exercise Q4

The program is supposed to ask the user to press a button to indicate what colour is the Colour sensor detecting.

- If D22 is pressed and Colour sensor detects green, display “correct”
- If D24 is pressed and Colour sensor detects blue, display “correct”
- Otherwise, display “wrong”

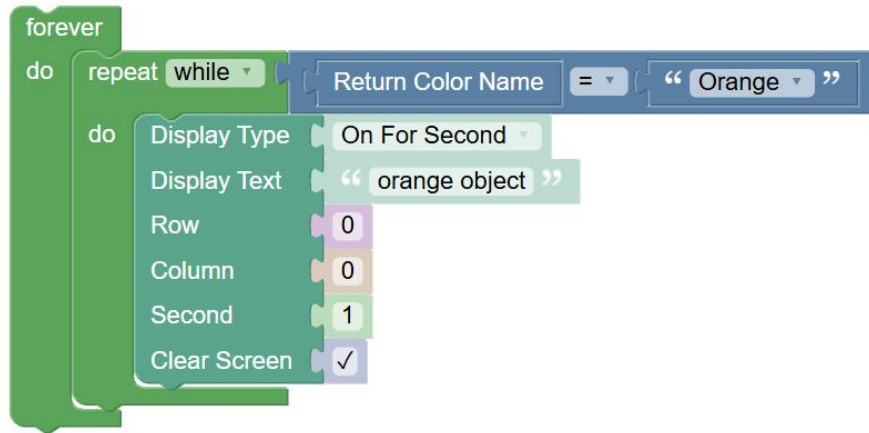
Complete the code to make it work.



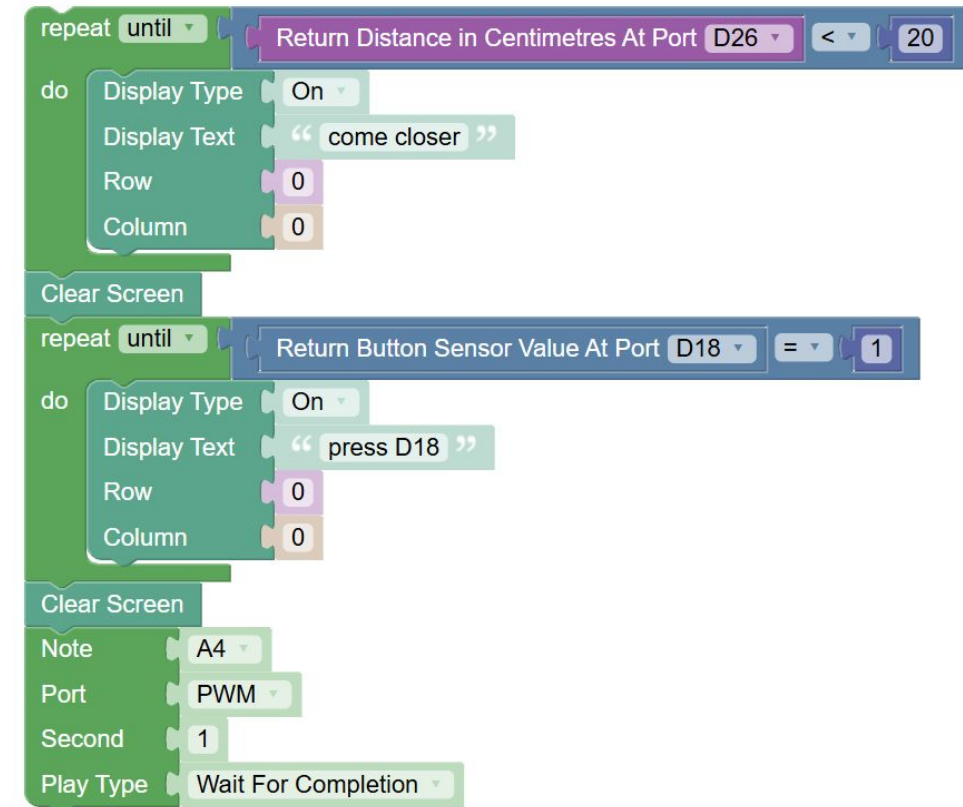
Answers

Q1: A, C and E are true

Q2:

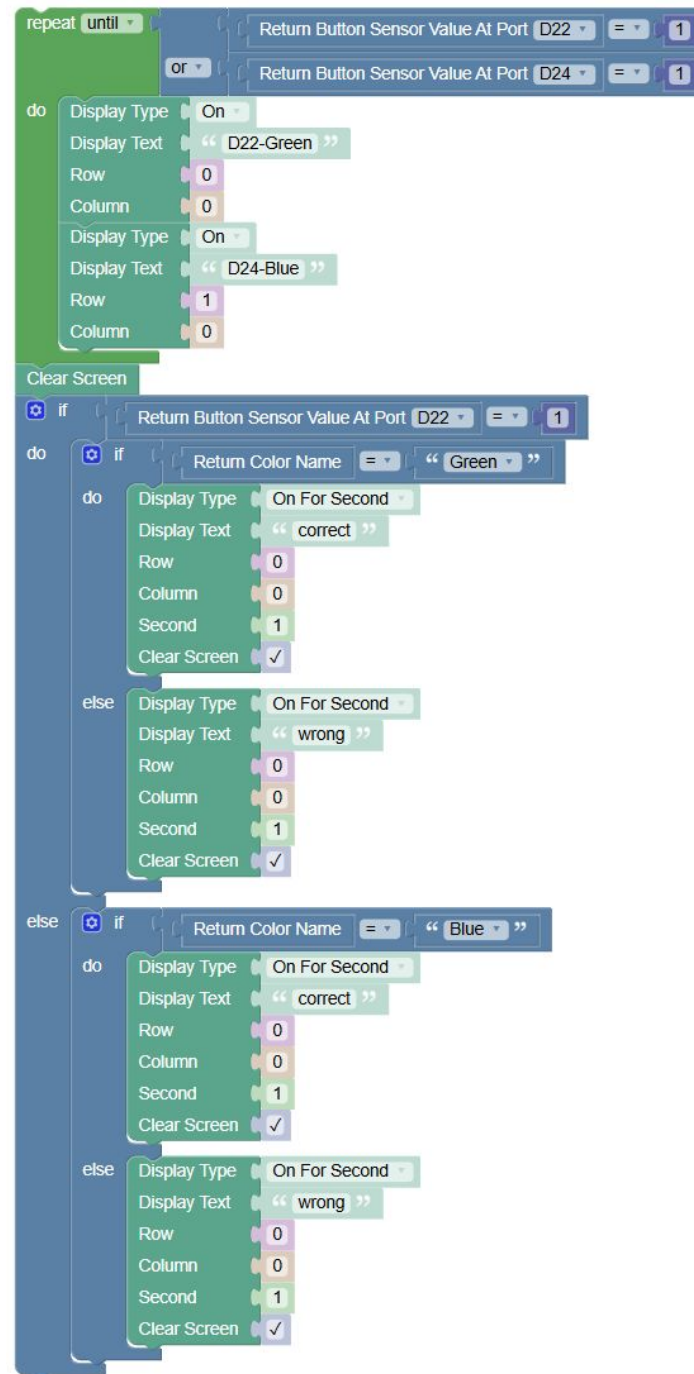


Q3:



Answers

Q4:



2009 Project 1

Learning

When you're passionate about programming, you'll naturally aspire to design and build your own remarkable application. To support that journey, you'll be guided through a project in class that lets you experience what it is like to develop a more extended program.



2010 Count Loop

Learning

A **Count loop** is excellent for counting and forming number sequences. There are many applications that rely on number sequences to work.

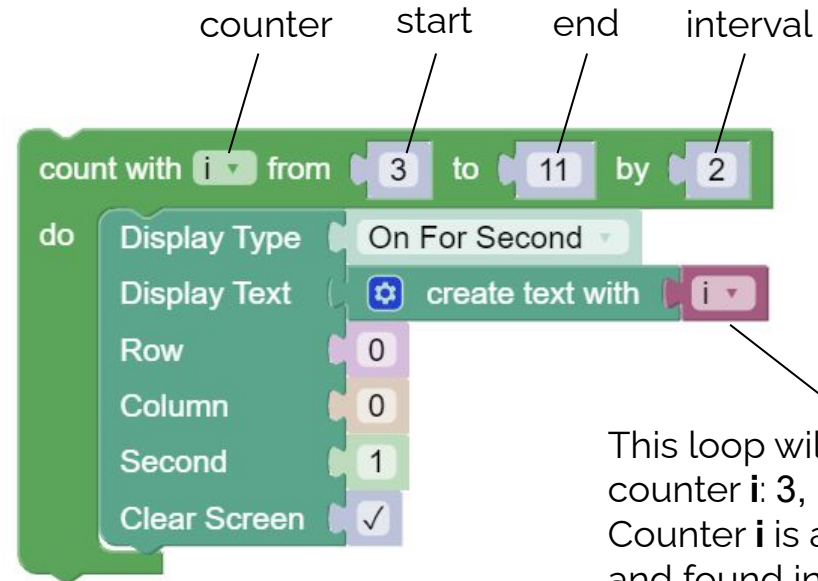
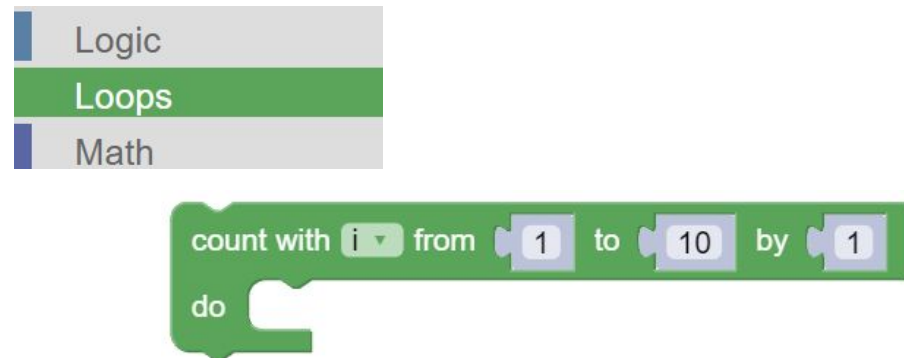
In class, you will be guided to explore the following:

- Understanding the application of Count loop in repetition, generating number sequences, animation, etc.
- Combining Count loops with Math Operators to generate different number sequences using the same loop.

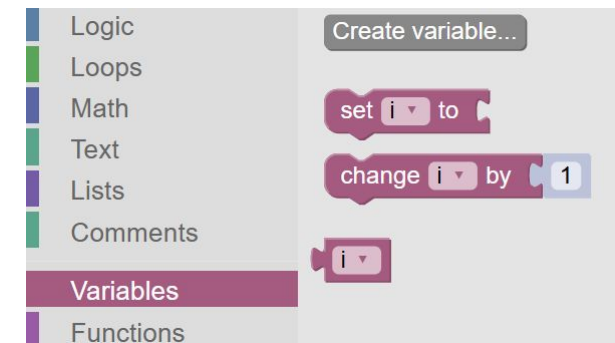
2010 Count Loop

Count loop

Repeat codes within using a counting loop.



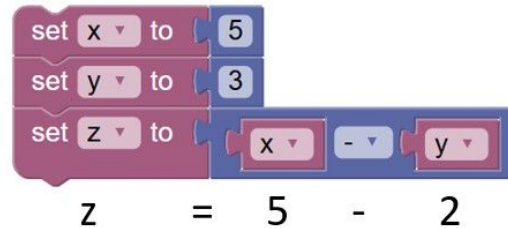
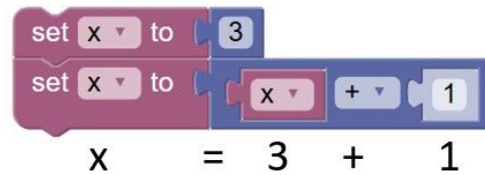
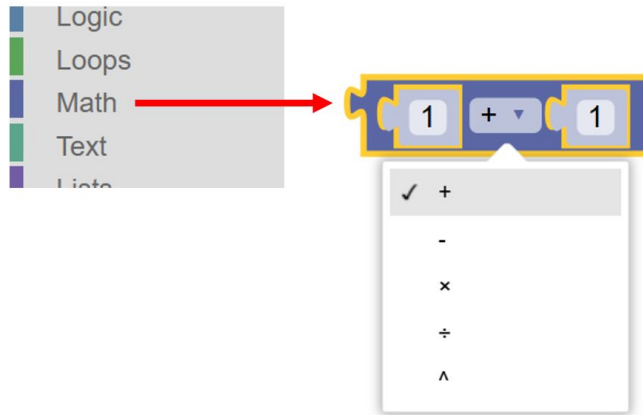
This loop will display the values of counter **i**: 3, 5, 7, 9, 11 (total 5 times). Counter **i** is a variable that is created and found in the variable section



2010 Count Loop

Math Operators

The Math operators are used to perform mathematical operations with the variables. You will learn more about variables soon. Below shows some examples.

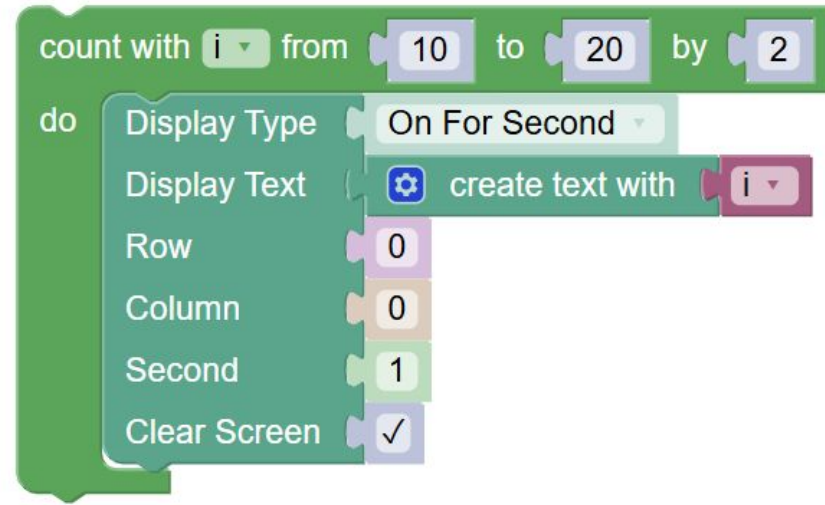


When using multiple Math operator blocks to create formula, be careful how you place them together.



Exercise Q1

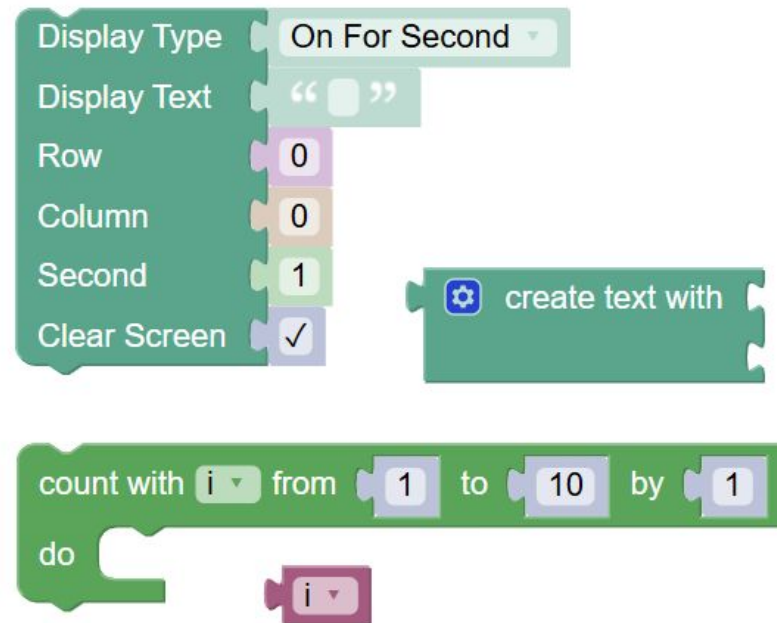
Study the code below and write down the numbers that appear on the LCD.



Exercise Q2

Using the blocks below, display the following numbers one by one:

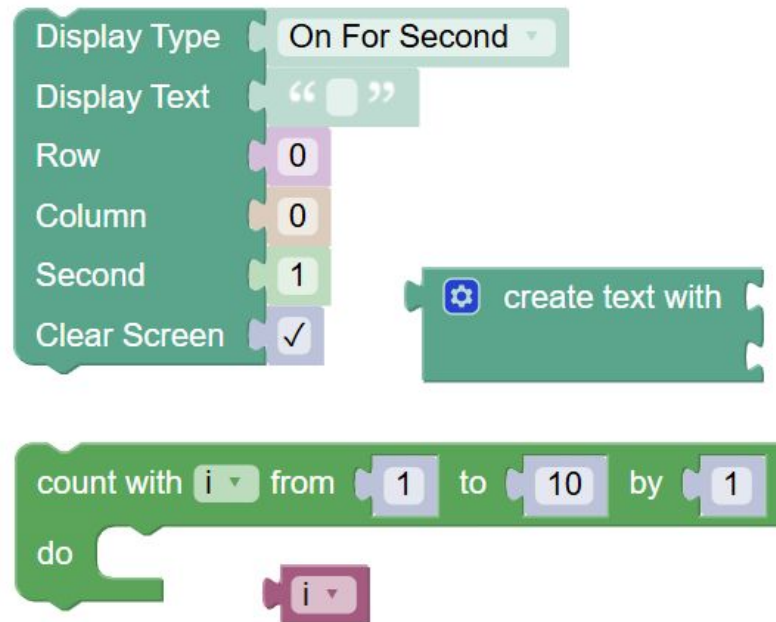
10, 7, 4, 1, -2



Exercise Q3

Using the blocks below, display the following running numbers one by one.
You will need to duplicate some blocks:

5, 6, 7, 8, 9, 8, 7, 6, 5



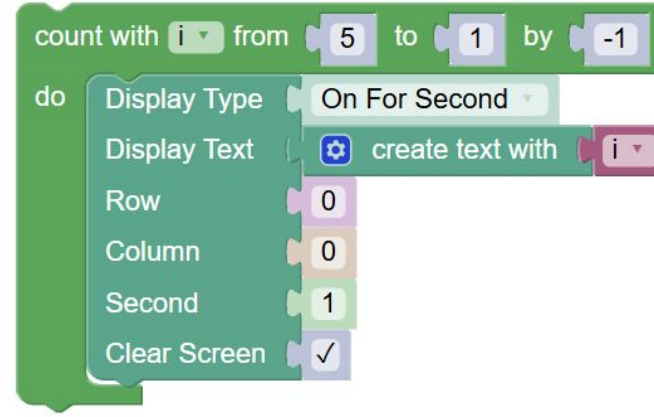
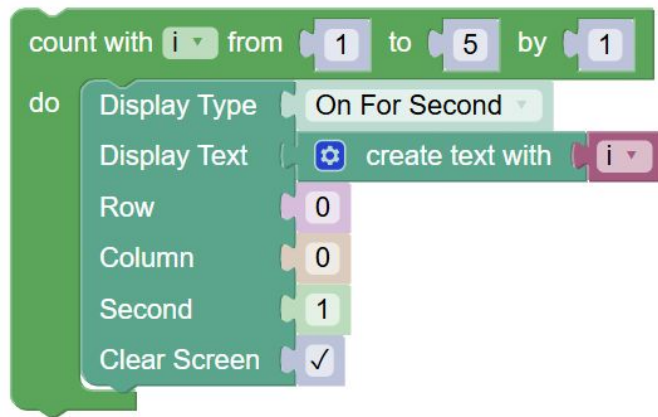
Exercise Q4

The code below shows two counting snippets.

One counting from 1 to 5 and the other from 5 to 1.

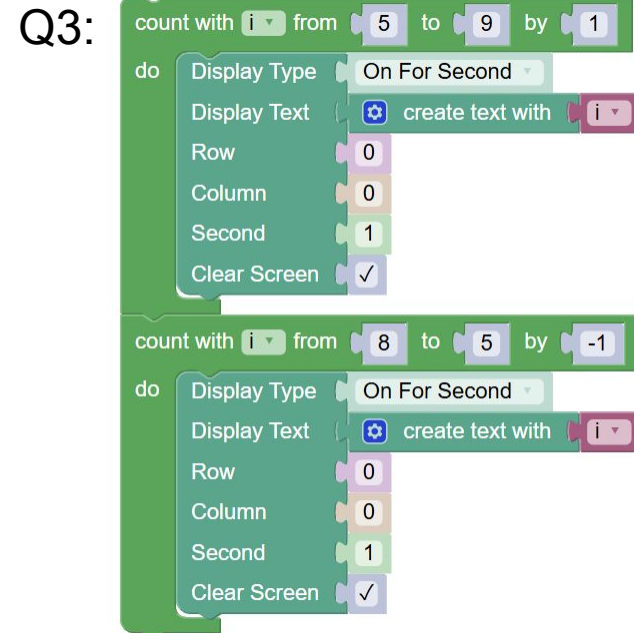
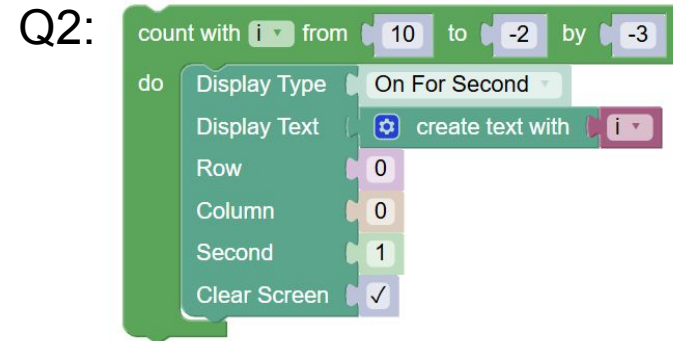
Add in more blocks to form a code that:

- Displays counting number 1 to 5 whenever Button D22 is pressed.
- Displays counting number 5 to 1 whenever Button D24 is pressed.

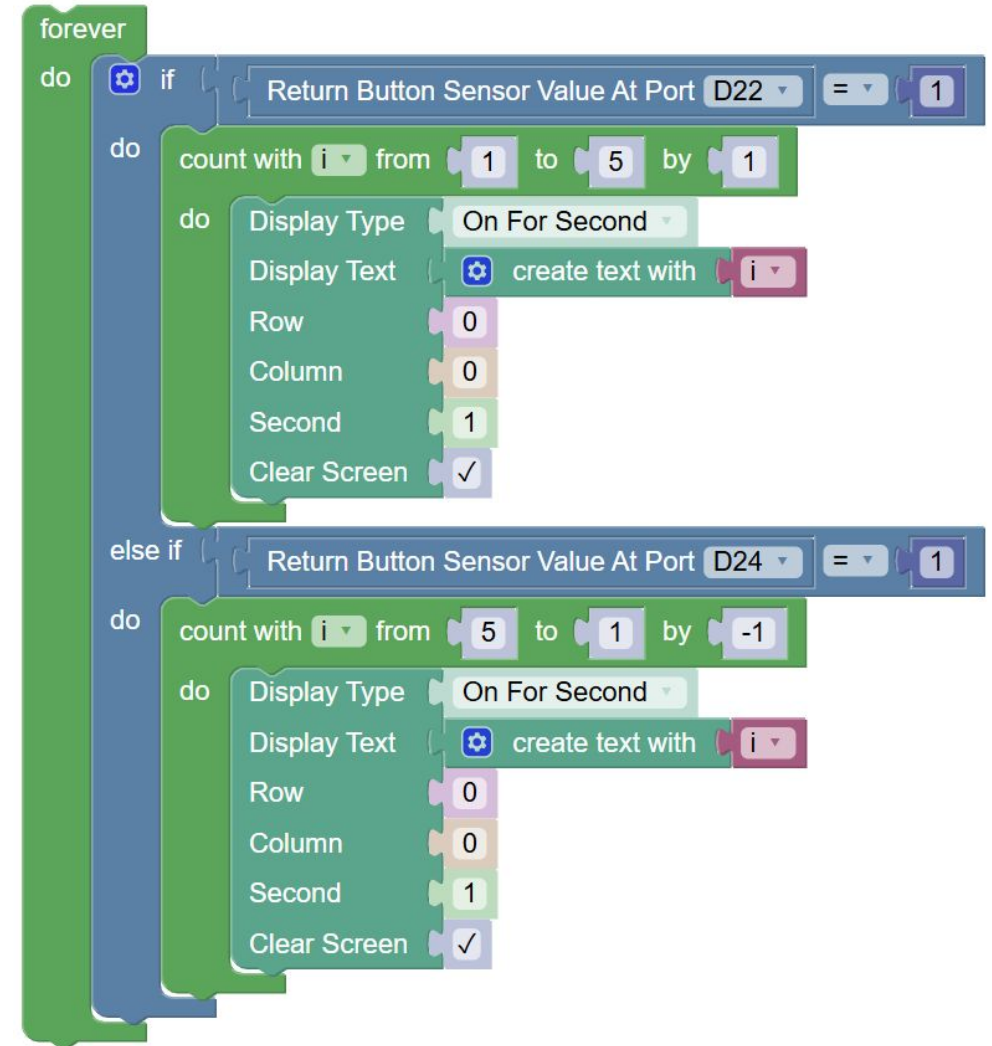


Answers

Q1: 10, 12, 14, 16, 18, 20



Q4:



2011 Variable

Learning

A **variable** stores a value that can be reused and modified during program execution. Using variables allow the program to retain information and control its behavior.

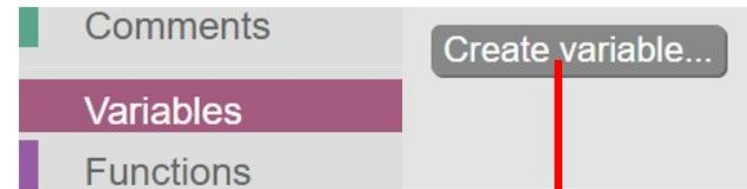
In class, you will be guided to explore the following:

- Create and manipulate number and string (text) variables..
- Generate random numbers and apply logic with variables.
- Explore the possibilities of applications with variables.

2011 Variable

Creating variable

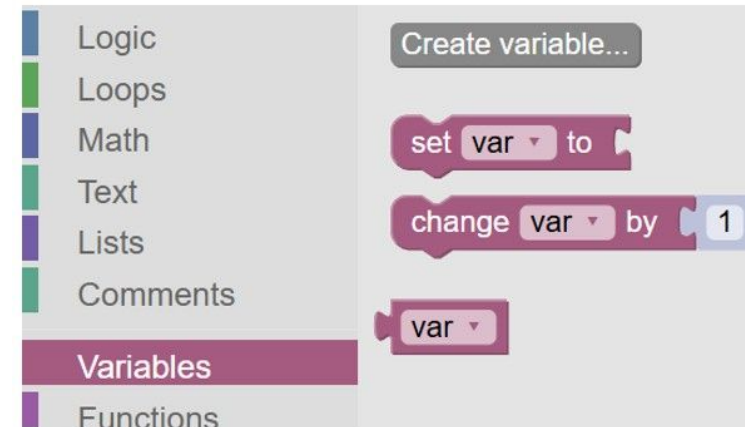
A variable is used to store a value that will be used subsequently. When you can create a variable, give it a meaningful name (what it stores, e.g. name, age, superpower, etc).



A dialog box titled 'web.thelab.sg says' with the text 'New variable name:'. Below the text is a text input field containing the text 'var'. A red arrow points from the text 'key in your variable name here' to the input field. At the bottom of the dialog are two buttons: 'OK' and 'Cancel'.

key in your variable name here

Once a variable is created, it will show up in the menu.



2011 Variable

Setting and Modifying variable

A variable can store either a number (eg. 1, 9, 22, 5.47, etc), a text (eg. "ABC") or other **data type**. It stores only one value at any time.

It is important to know what data type the variable is storing so that you can perform the right operation. Below shows some examples using the **change block** and **create-text block**.

Assign a **text** to the variable



Assign a **number** to the variable

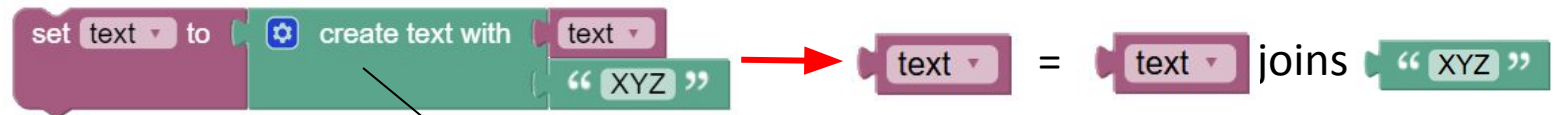


Add a **number** to the variable



This only works when the variable is currently storing a **number** (not text or empty)

Append values behind the current value



this block joins the values into one row of text

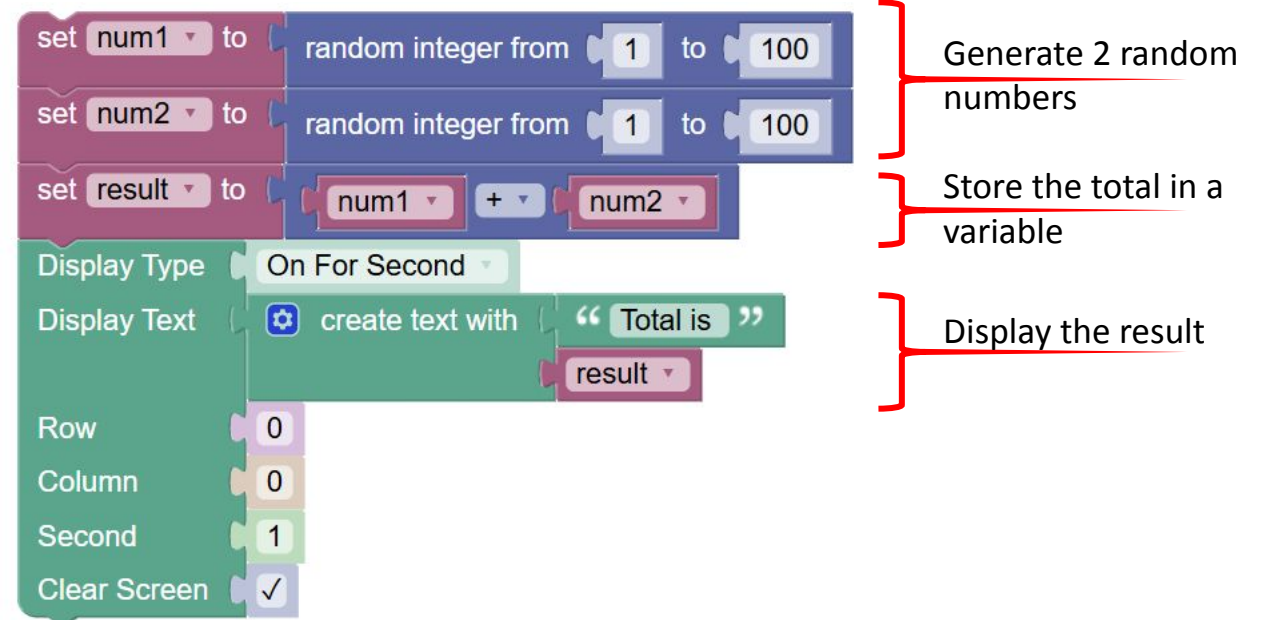
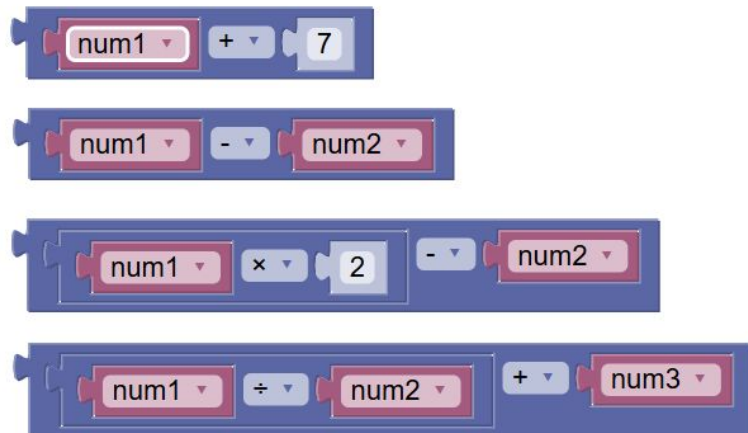
Add a **negative number** decreases the variable value



2011 Variable

Arithmetic Operations with Variables

You can perform mathematical operations (arithmetic) with variables. Below shows some examples.



2011 Variable

Generate Random Number

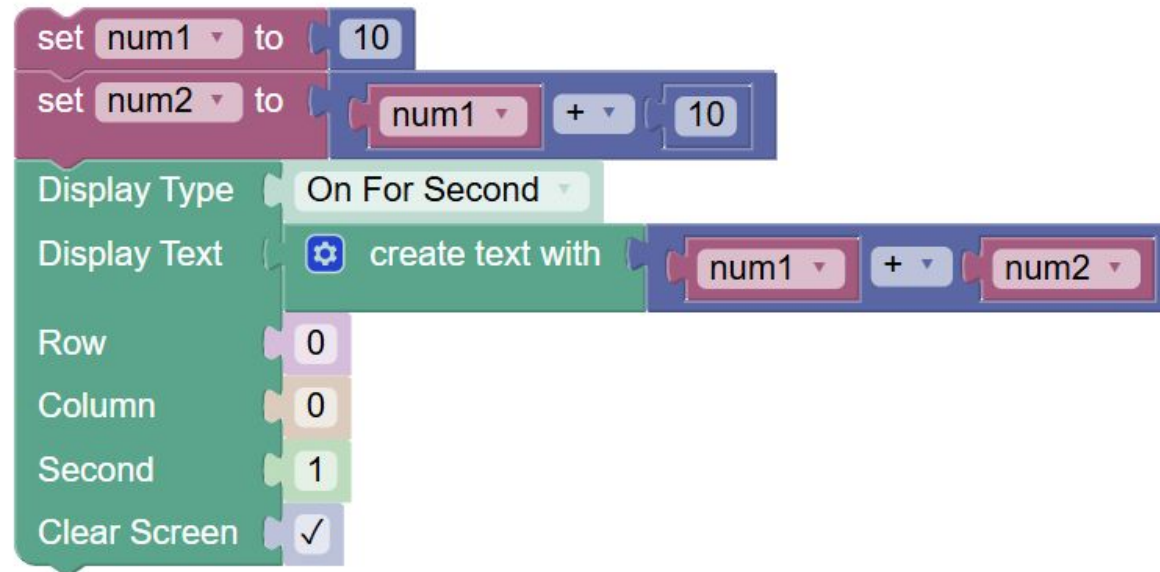
The ***random block*** can generate a random number within a range. Each time the block is used, a new random number is generated (the number may repeat). Below shows an example.



a random number within the range 5 to 10 is generated and assigned to the variable *number*. That is, the random number can be 5, 6, 7, 8, 9, 10.

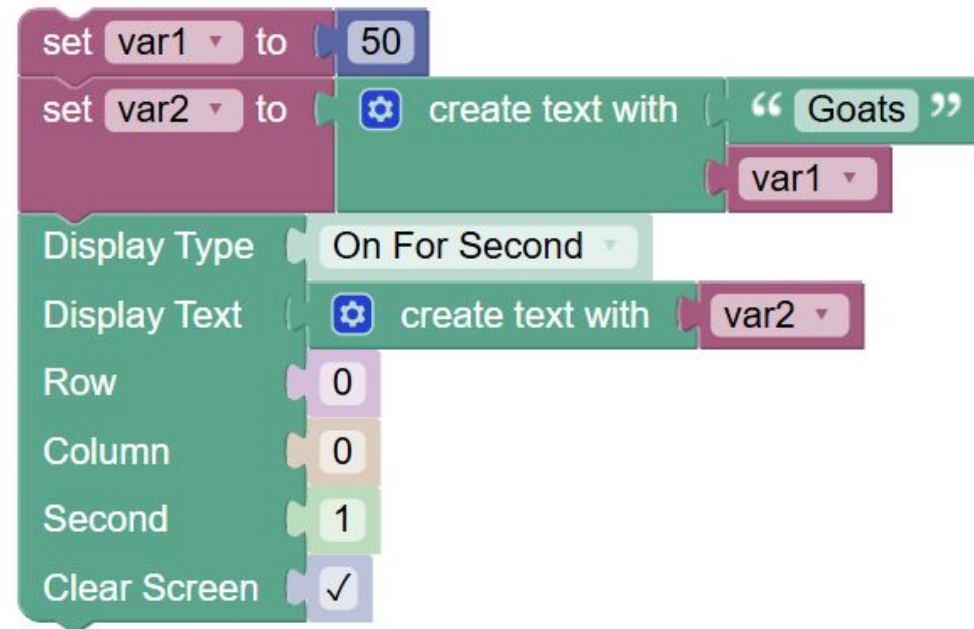
Exercise Q1

Study the code without running it and write down what will be displayed.



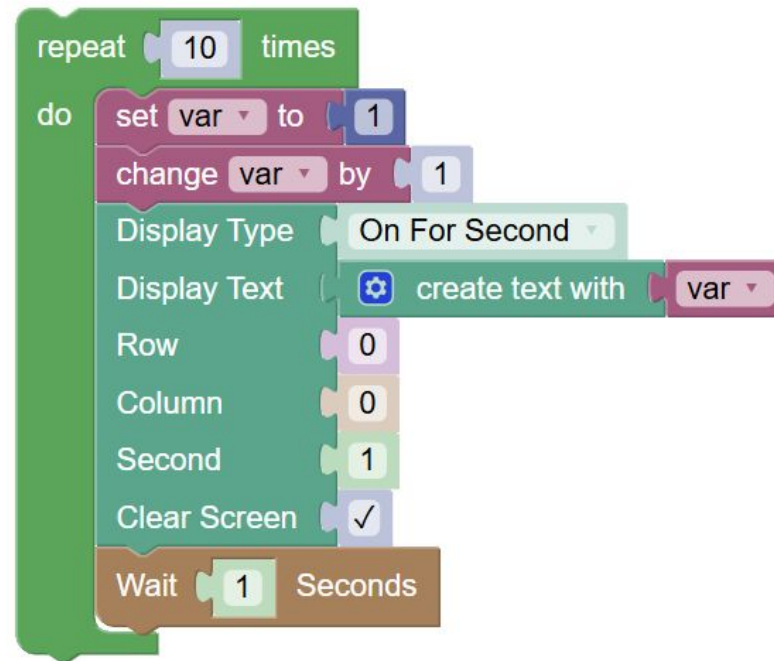
Exercise Q2

Study the code without running it and write down what will be displayed.



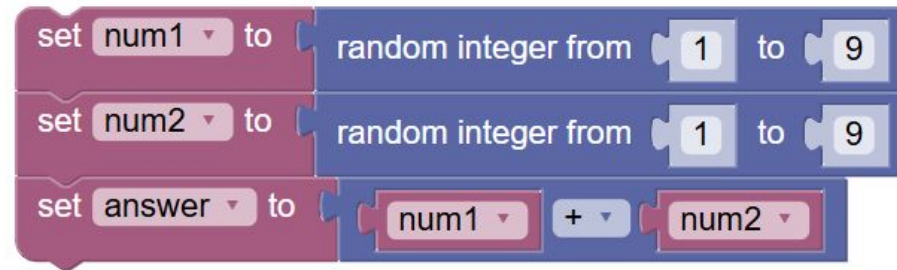
Exercise Q3

The code below is supposed to display 1 to 10 but it is not working. Test out the code and fix the error without adding more blocks.

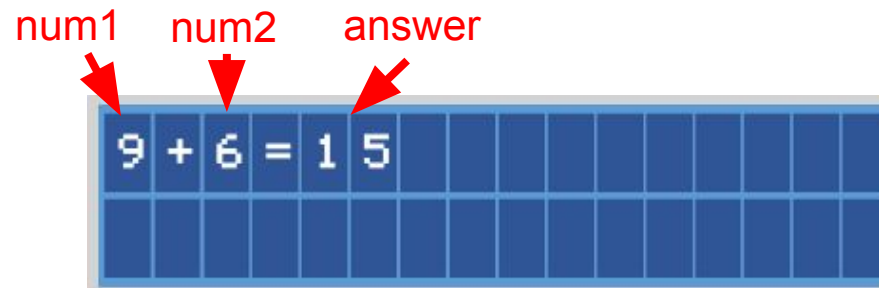


Exercise Q4

The following generates **2 random numbers** and their **sum**.



Add additional locks to the code above so that the output matches the format shown below.

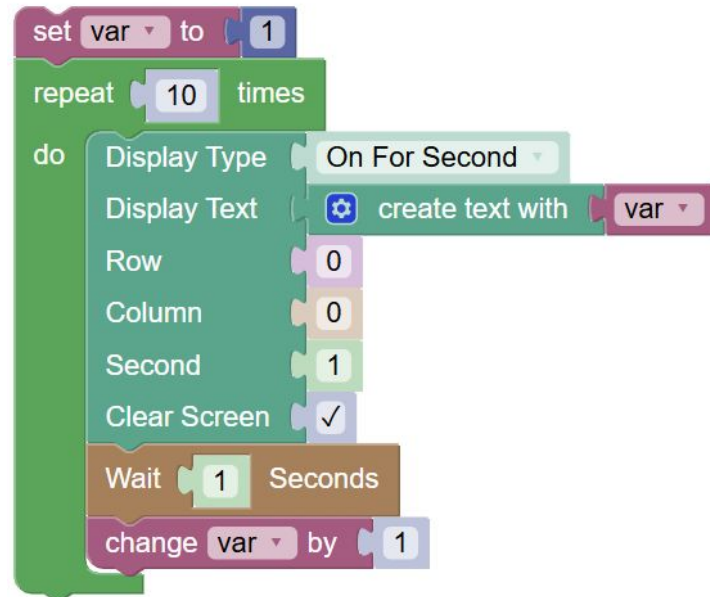


Answers

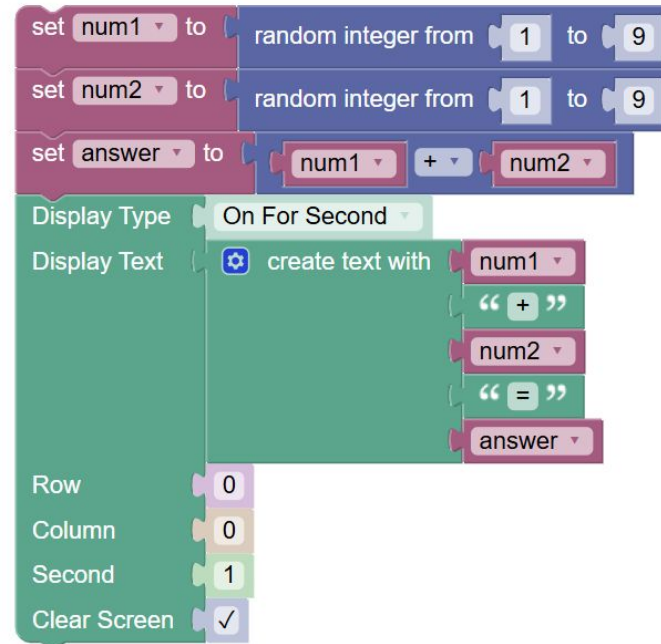
Q1: 30

Q2: Goats50

Q3:



Q4:



2012 Module Control with Variable

Learning

Variables can control different parts of the robot while it is running. That allows you to build way cooler applications!

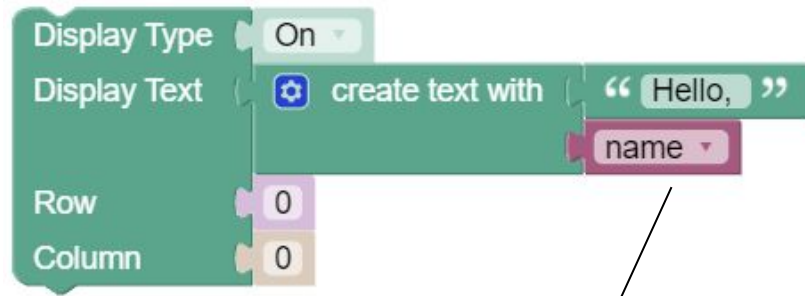
In class, you will be guided to explore the following:

- Controlling of buzzer and LCD modules using variables.

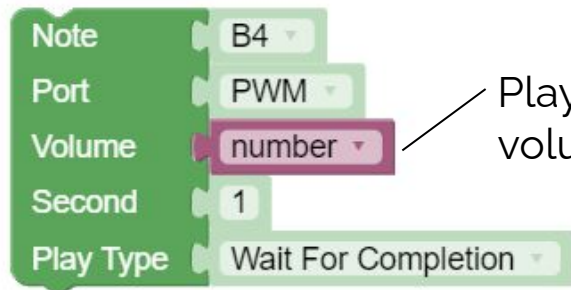
2012 Variable

Using variable

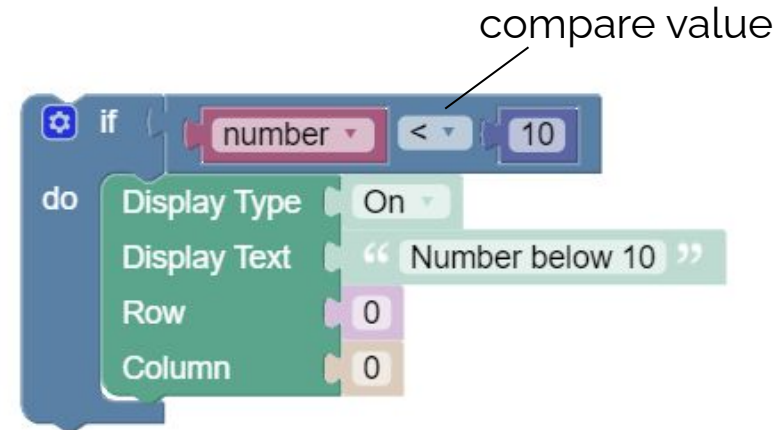
Variables can be used in many ways. Below shows some examples.



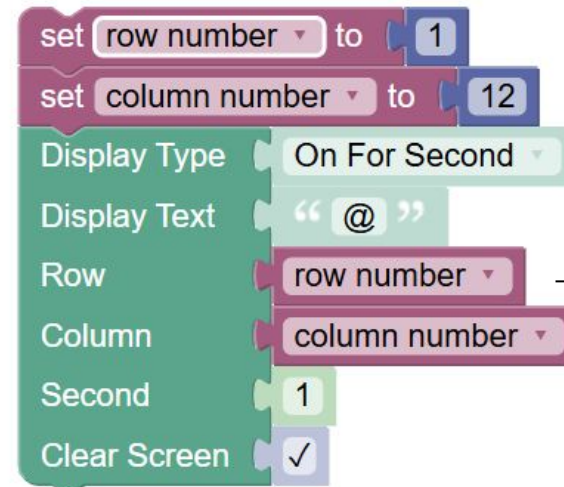
greet with a name



Play note with a volume



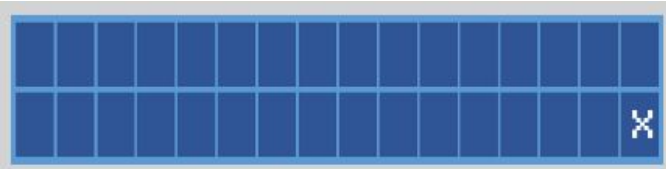
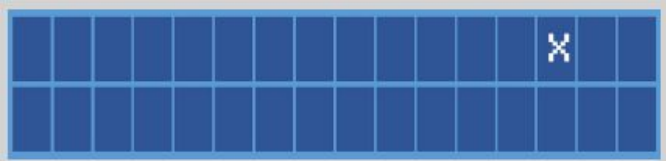
compare value

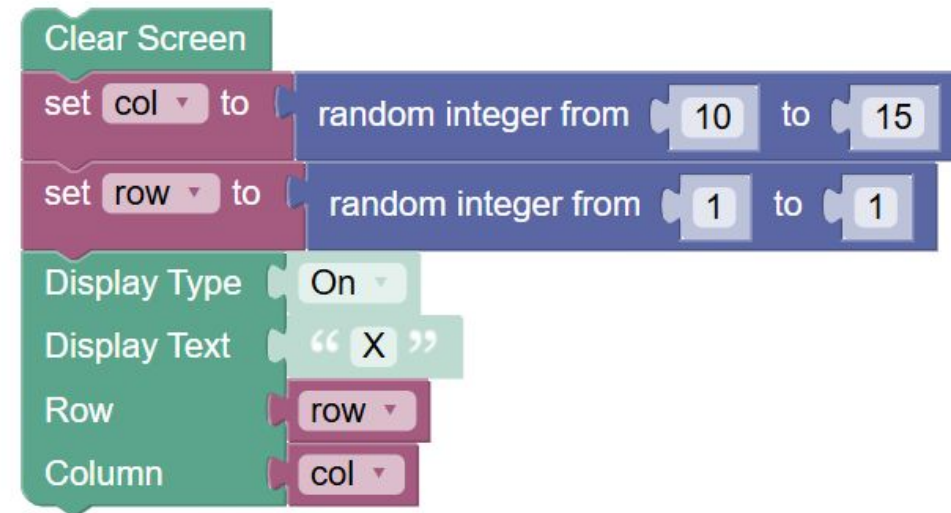


Position a text

Exercise Q1

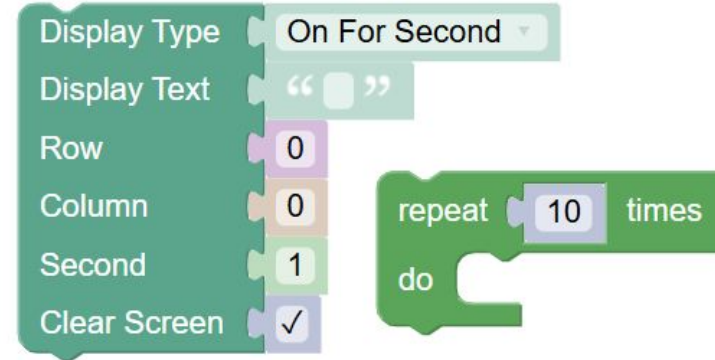
Which of the following output(s) is/are possible when the code is run?

- A) 
- B) 
- C) 
- D) 
- E) 



Exercise Q2

Using the blocks below, form a code that display the animation shown.



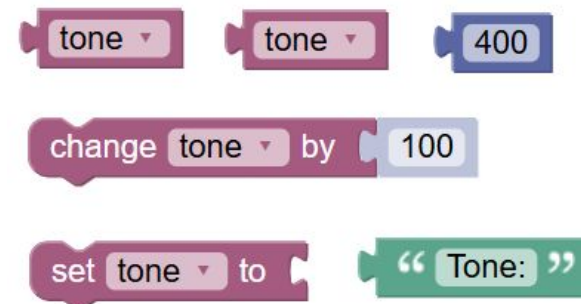
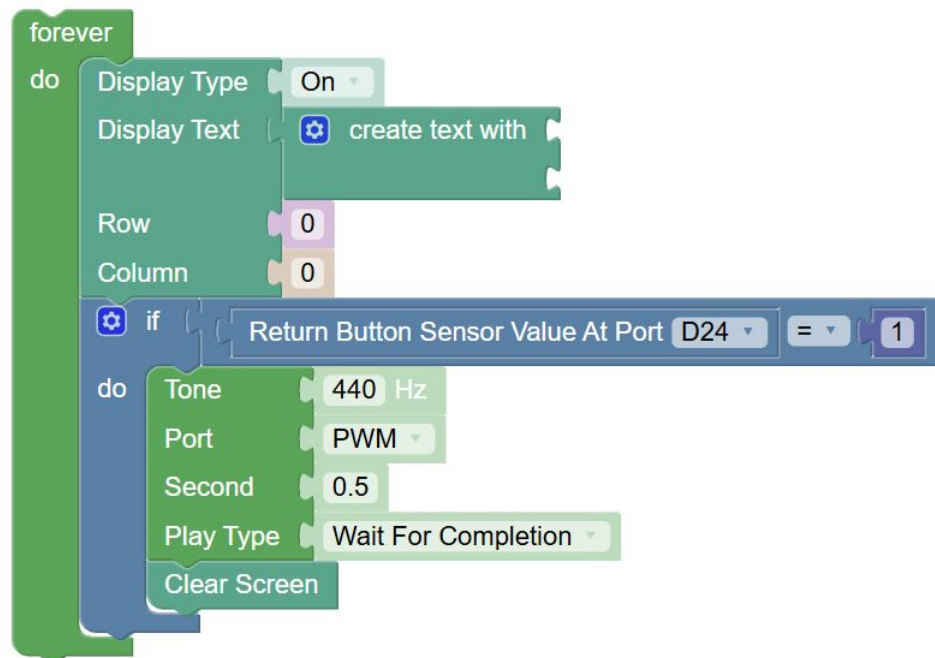
animated clip

Exercise Q3

You are building a code that performs the following:

1. The LCD shows the tone level, starting at 400Hz.
2. Whenever Button D24 is pressed, it will play the tone and then increase it by 100Hz.

Use the blocks below to complete the code.

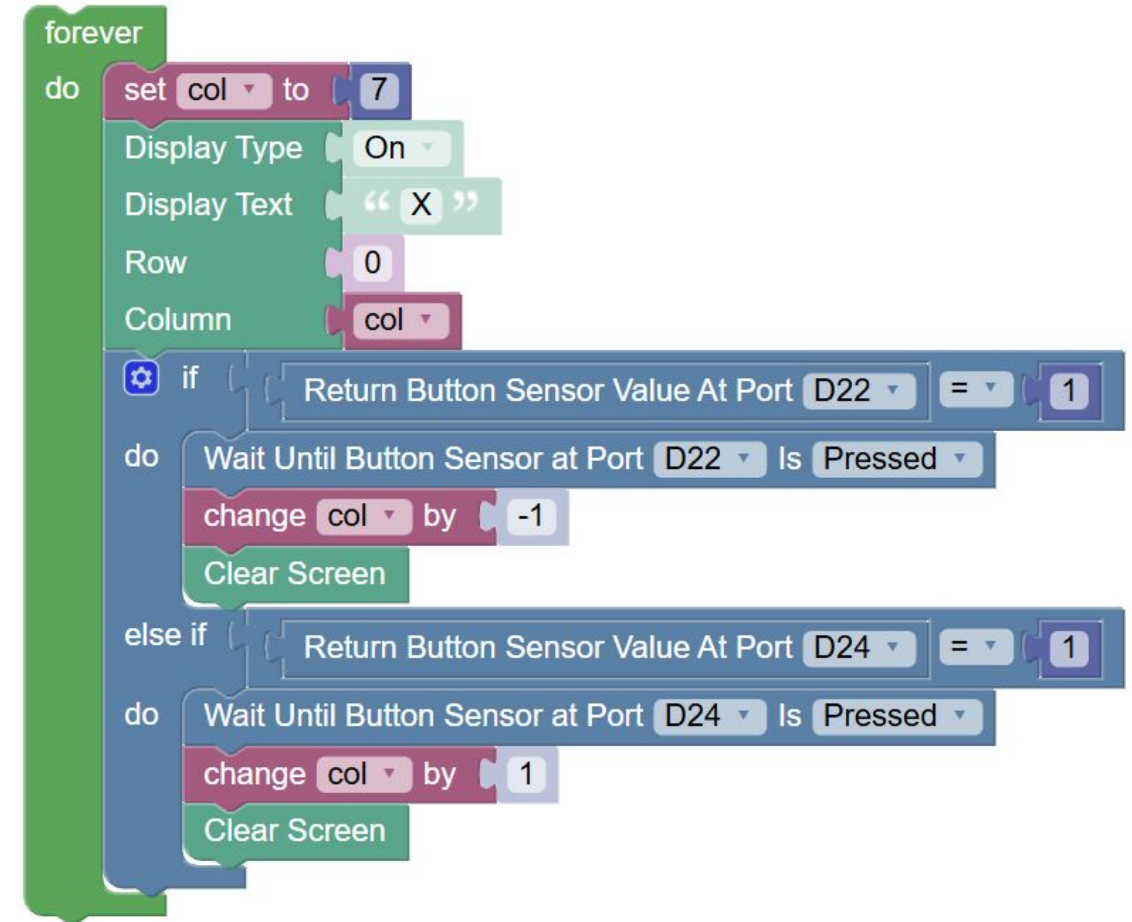
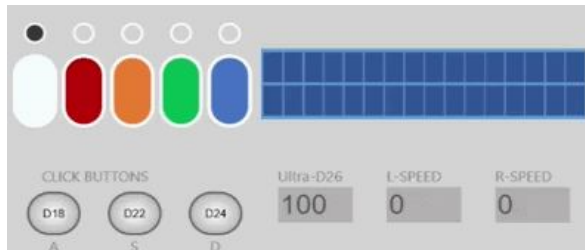


Exercise Q4

The code below is supposed to perform as follow:

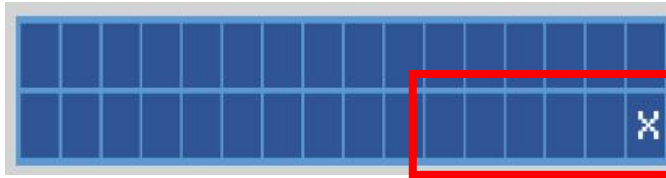
1. When Button D22 is pressed and released, the 'X' shifts left by one column.
2. When Button D24 is pressed and released, the 'X' shifts right by one column.

However, it is not working. **Fix the bug without adding more blocks.**



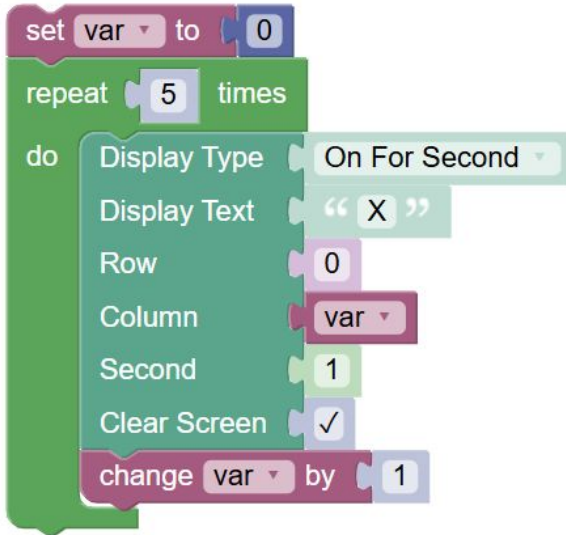
Answers

Q1: C and E

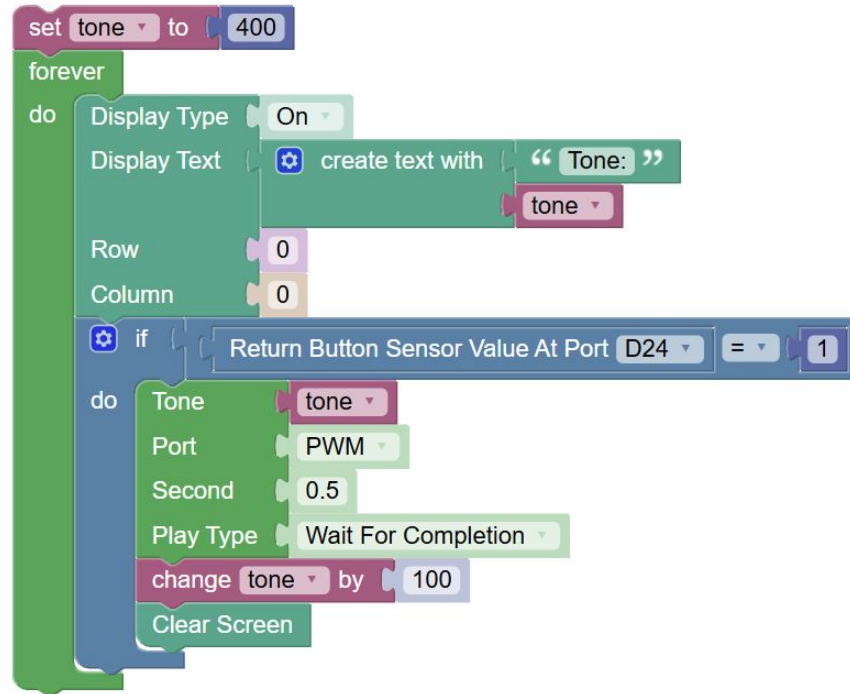


Within this region

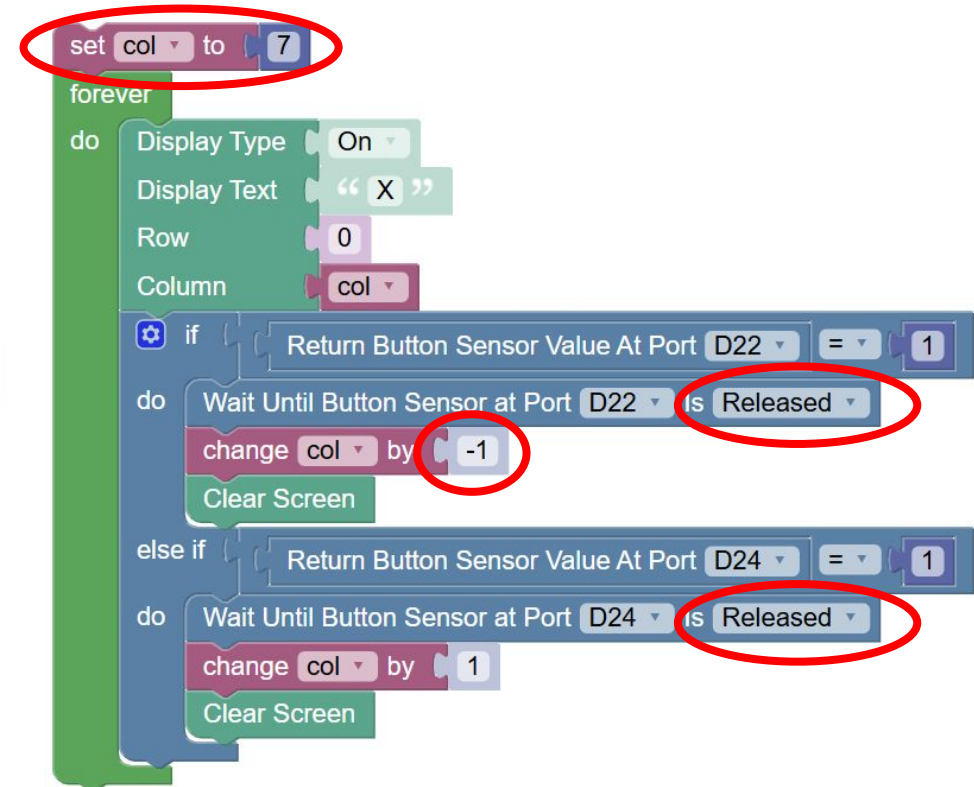
Q2:



Q3:



Q4:



2013 Applications

Learning

There are unlimited possibilities of what we can imagine and build with variables.

In class, you will be guided to explore the following:

- Applying variables to a wider range of applications including games, animation control and simulation.

2014 Variables and Sensors

Learning

Variables make it possible to store sensor inputs. This helps in processing sensor data and makes it easier to build more types of sensor-based applications.

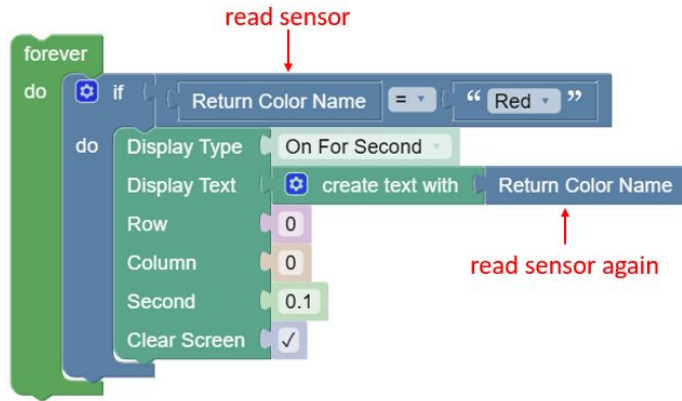
In class, you will be guided to explore the following:

- How to increase the reliability of sensor readings through variables.
- Using variables with button, ultrasonic distance and colour sensors.
- Exploring more interactive applications.

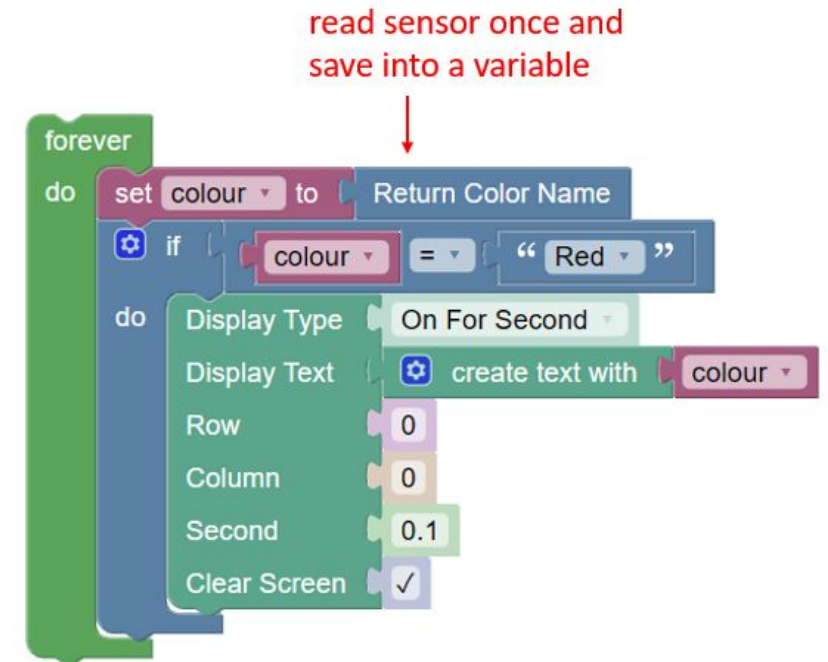
2014 Variables and Sensors

Processing sensor values

The code below will display the Colour sensor value when it is "Red".

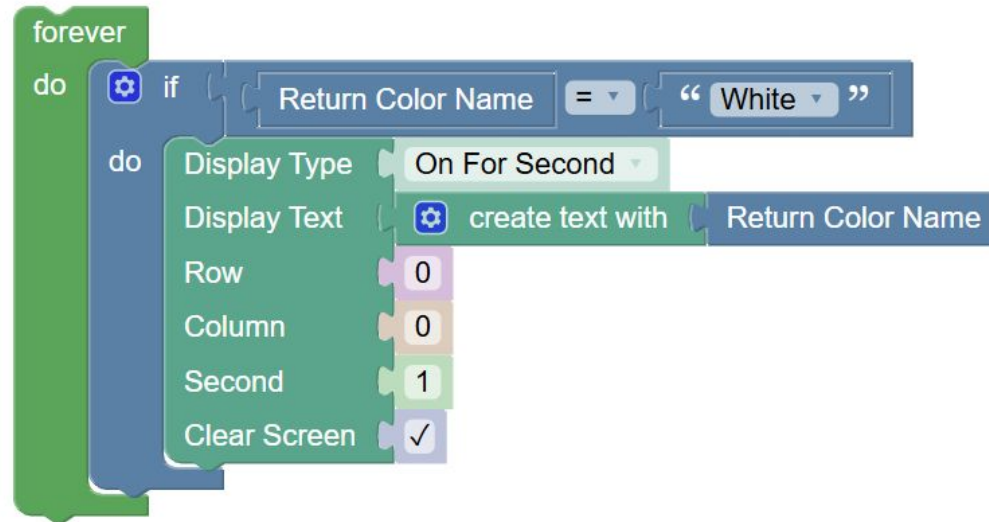


Although not often, it is possible to see other colours being displayed. This is because the code reads the sensor twice. The first time for checking that it is "Red", the second time to display it. These two values may differ and the second reading can be colours other than red. The code can be more reliable using a variable as shown. The variable value will be consistent for checking and displaying.



Exercise Q1

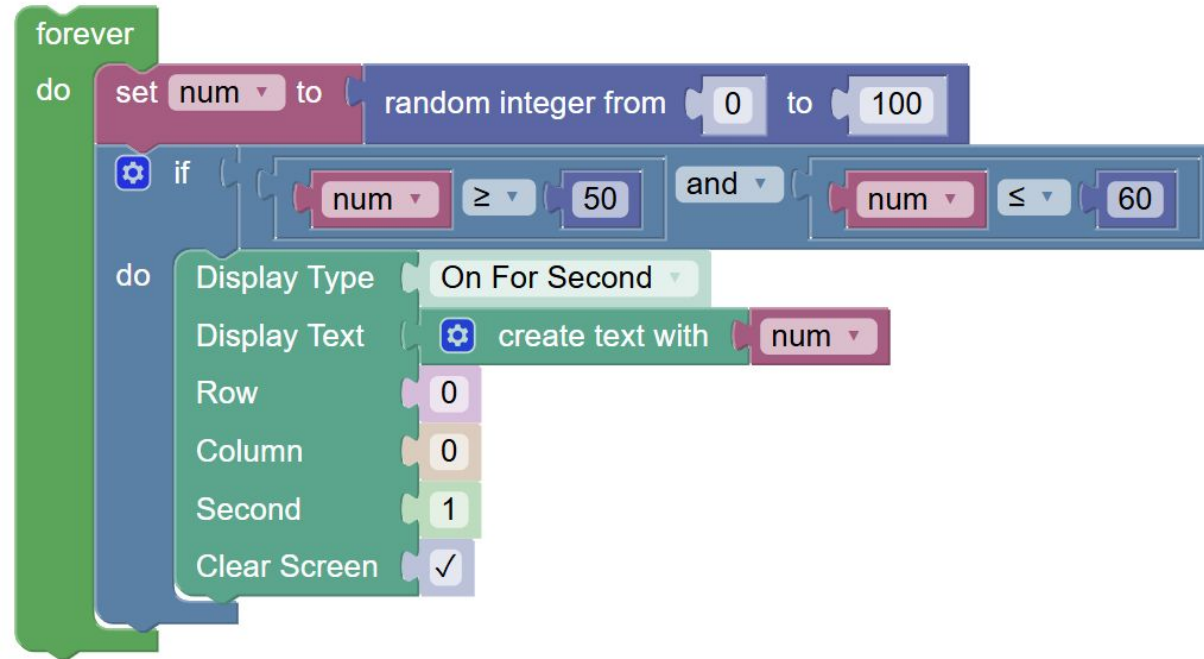
When the code is run, what will be the possible output?



- A) White
- B) Orange
- C) Red
- D) Green
- E) Blue

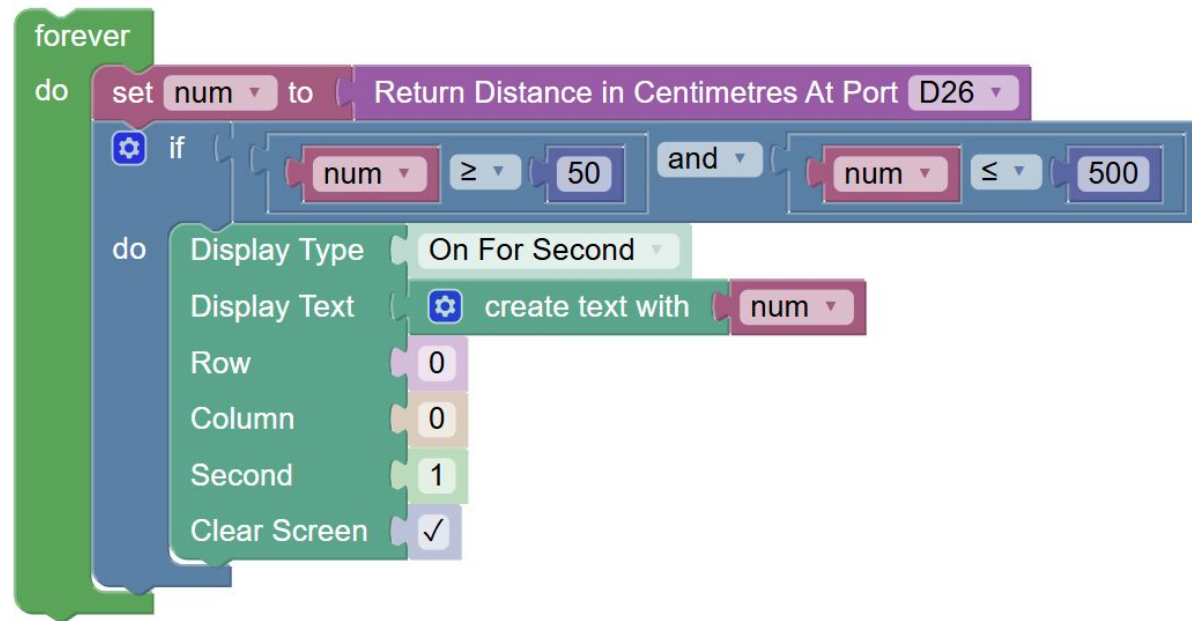
Exercise Q2

Write down the range of numbers that may appear when the code is run.



Exercise Q3

Assuming that the Ultrasonic sensor block can detect a distance from 0 to 100cm, what will be the possible number range that can be displayed by the code?

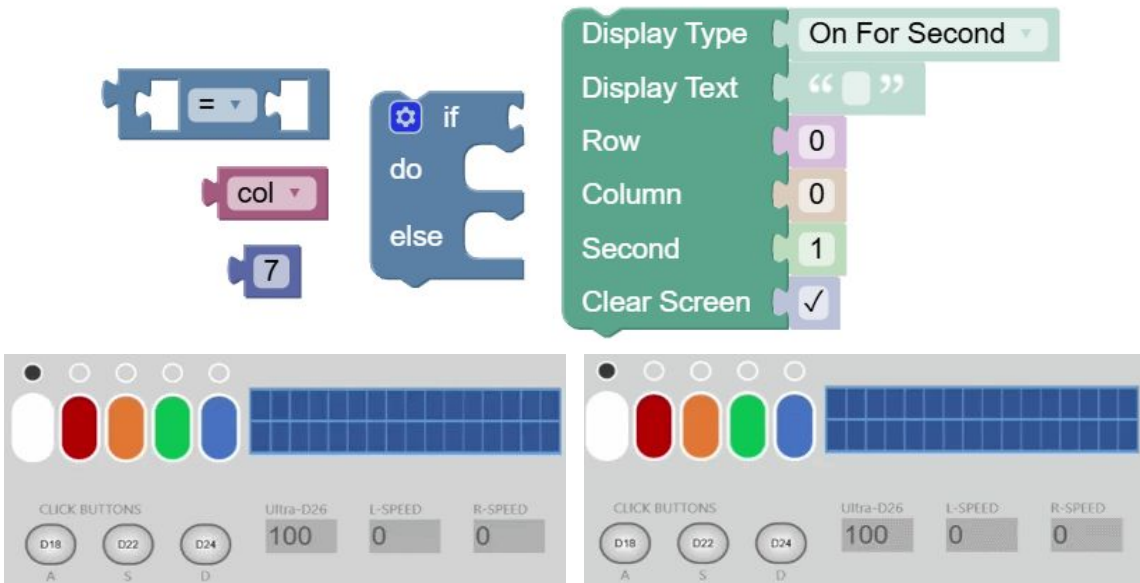


Exercise Q4

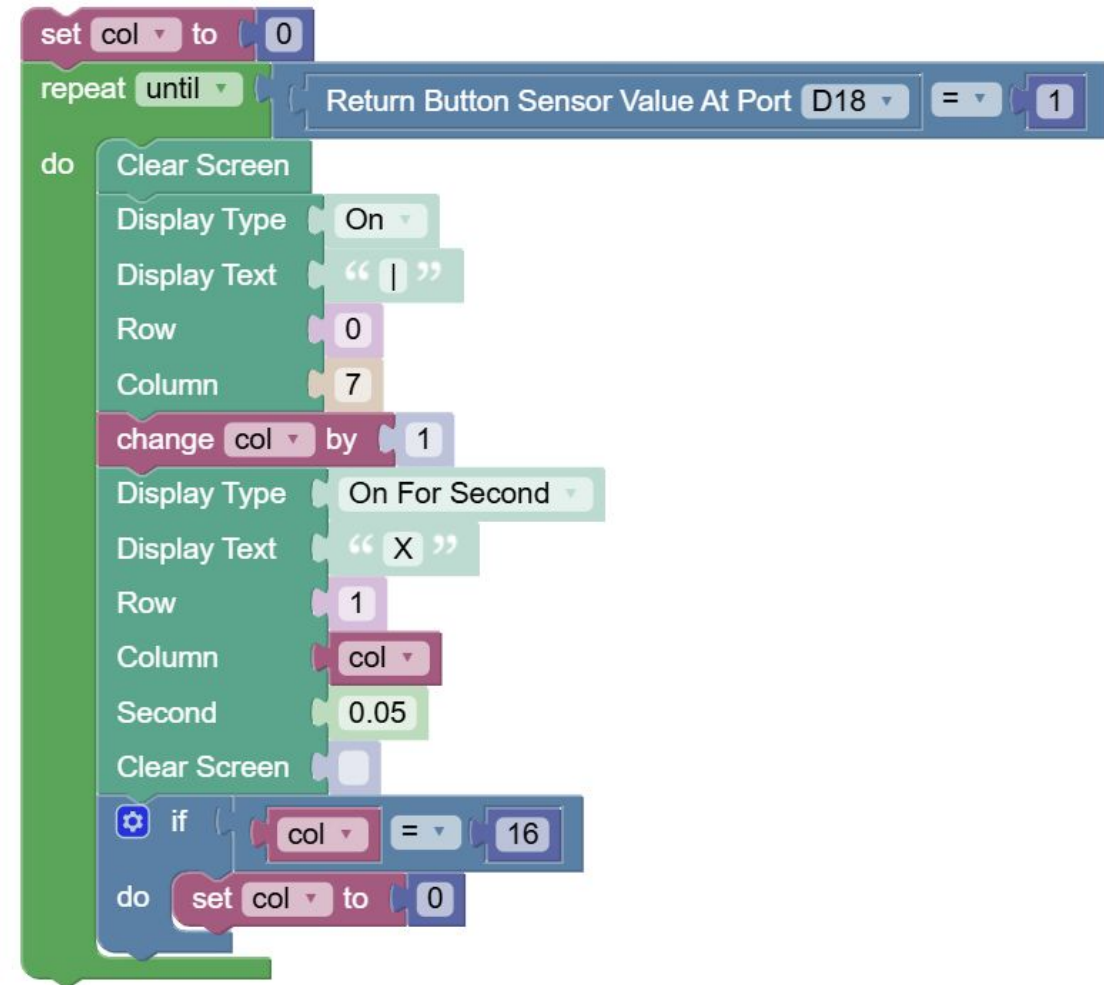
You are building a reflex game that works as follows:

- The “X” will move across the bottom row repeatedly.
- When Button D18 is pressed, it will stop moving.
- If X is aligned with “|”, “hit” will be displayed.
- If X is not aligned with “|”, “miss” will be displayed.

Add the blocks below to the code to make it work. You may need to duplicate some blocks.



animated clips



Answers

Q1: A, B, C, D, E.

The colour is read twice and the second time may not be white.

Q2: 50 to 60

Q3: 50 to 100

Q4:



2015 Boolean

Learning

A Boolean is a type of data that can only be either true or false. It's used in programming to make decisions and control how things work.

In class, you will be guided to explore the following:

- How to manipulate and use Boolean.

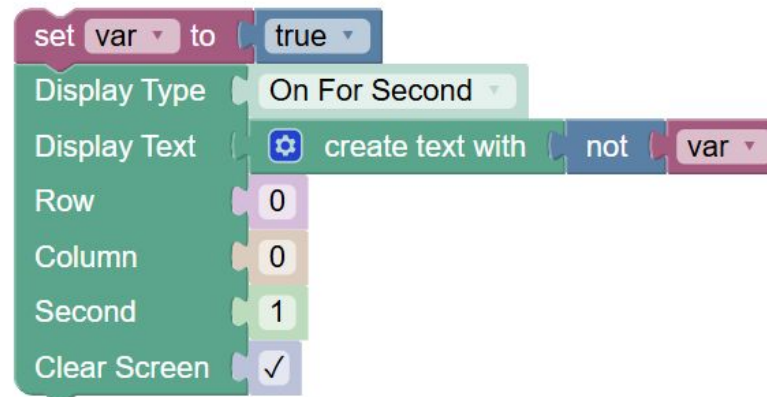
2015 Boolean

Booleans

A Boolean is like a yes-or-no switch in your code. It can only be true (yes) or false (no), and it helps your program figure out what actions to take, like whether a door should open or a character should jump.



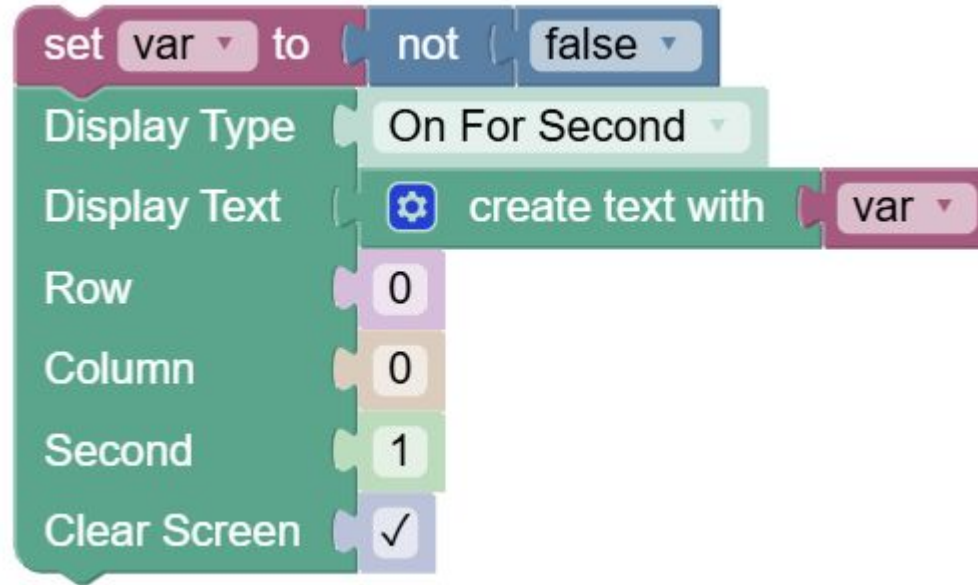
A Not operator reverses the logic.



False will be displayed

Exercise Q1

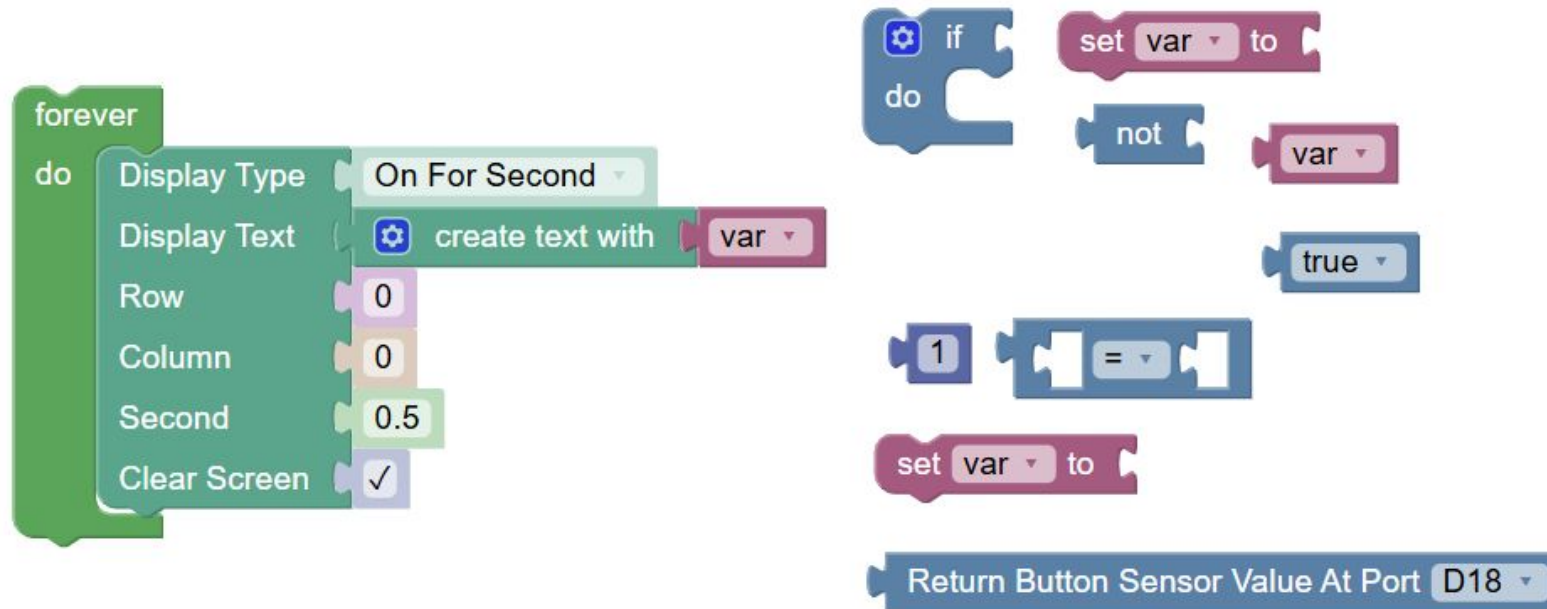
Write down what will be displayed on the LCD when the code is run.



Exercise Q2

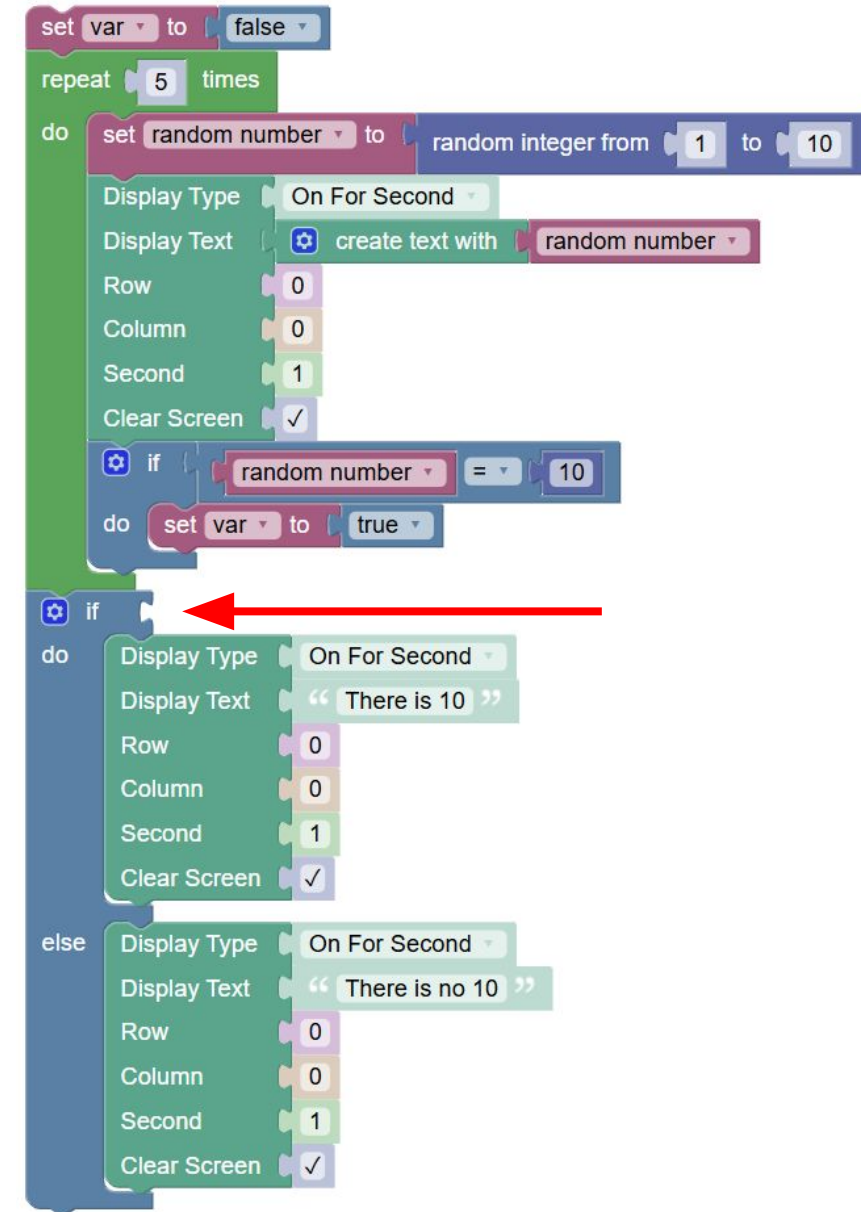
Using the blocks below, create a code that performs as follows:

1. It displays **True** at first.
2. When Button D18 is pressed, it will switch to **False**.
3. When pressed again, it will switch back to **True**.
4. Each time Button D18 is pressed, it will keep switching between **True** and **False**.



Exercise Q3

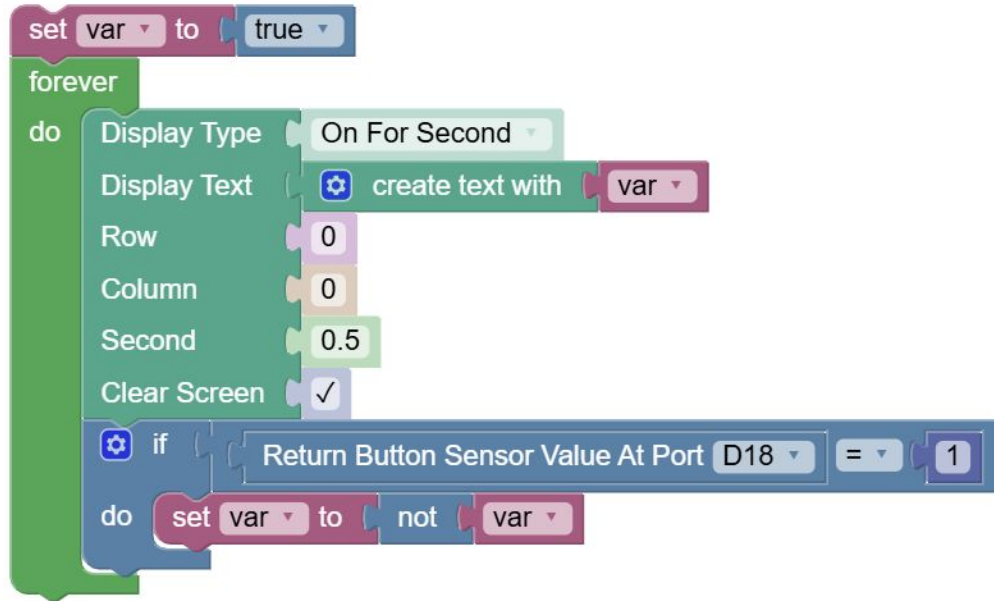
The program generates 5 random numbers and displays them one by one. It will identify whether number 10 has appeared. Complete the IF block condition to make it work.



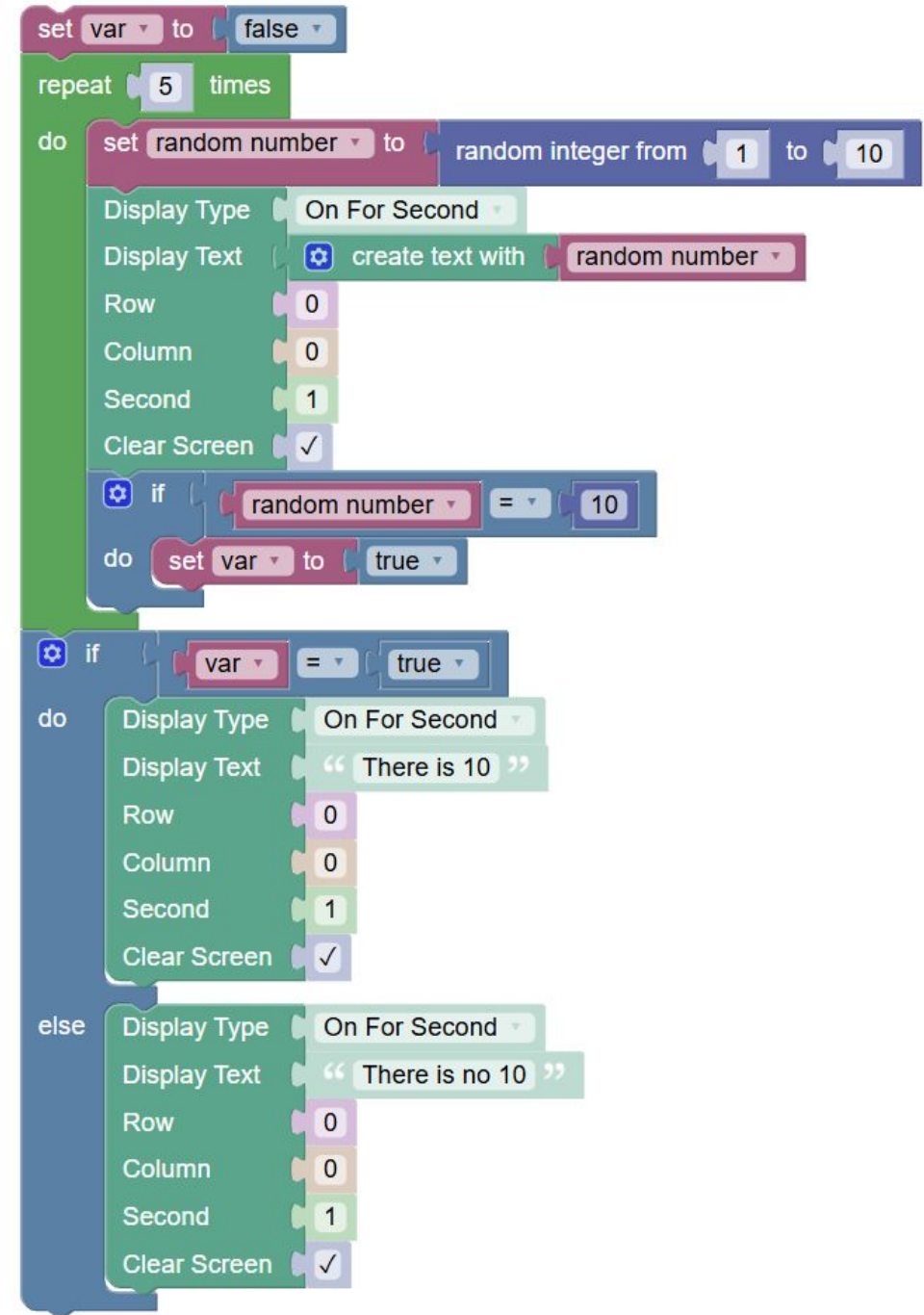
Answers

Q1: True

Q2:



Q3:



2016 Applications

Learning

This unit prepares you for developing your final project by exploring more examples of applications made possible through the use of variables.

In class, we will explore the following:

- A wider range of applications using what you have learnt.

2017 Project 2

Learning

As you delve deeper into programming, your growing expertise unlocks the ability to build more ambitious and creative projects. You'll be guided through the development of a playful animated application, an exciting addition to your coding portfolio!



END